

华宸AI报告单：AIGC 原创性检测报告单

报告单编号：1779107954888

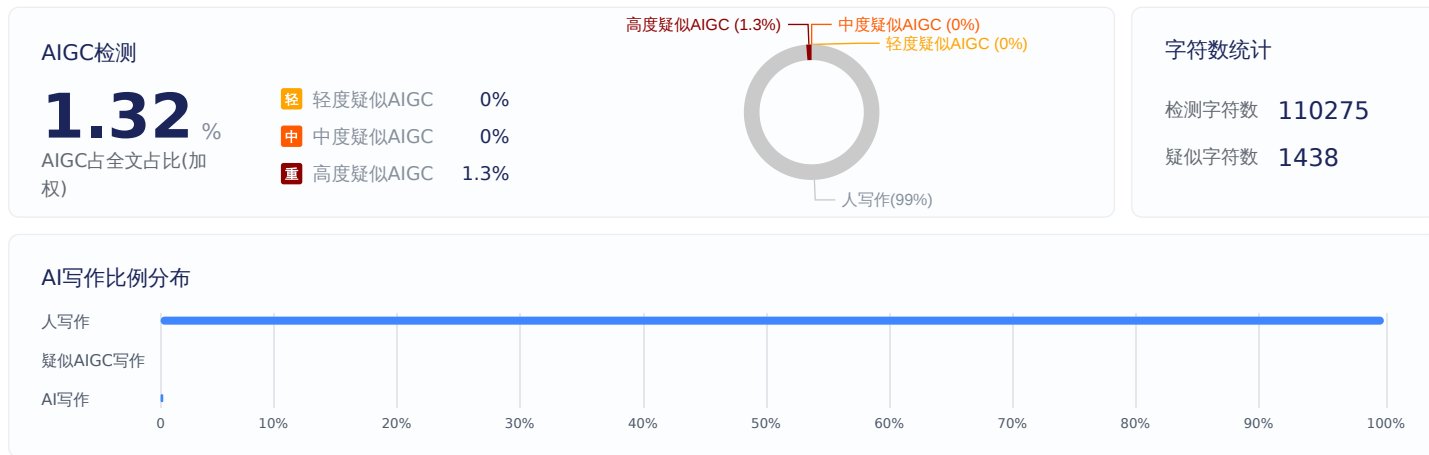
上传时间：2026-05-18

采用先进的深度学习和自然语言处理技术，以大语言模型为核心，结合特征向量分析、模型匹配算法和语义相似度测算，多层次语义分析和对抗性样本筛查等技术手段，精准识别区分人类撰写与模型生成内容，确保信息真实性

论文篇名 基于AI Agent 的建筑幕墙韧性评估软件系统设计与实现

作者 林继申

AIGC检测结果



论文原文

TONGJI UNIVERSITY 毕业设计（论文） 课题名称 基于AI Agent 的建筑幕墙韧性评估软件系统设计与实现副标题 学院 计算机科学与技术学院专业 软件工程 学生姓名 林继申 学号 2250758 指导教师 黄杰 日期 2026年5月22日 同济大学本科毕业设计（论文）信息说明页 课题名称：基于 AI Agent 的建筑幕墙韧性评估软件系统设计与实现成果类型：☒ 毕业设计 ☐ 毕业论文 学科专业：软件工程 内容简述（请用 300 字以内简要概述）：本毕业设计面向建筑幕墙人工巡检效率低、主观性强、难以处理海量图像等问题，设计并实现了一套基于 AI Agent 的建筑幕墙韧性评估软件系统。系统亮点包括面向建筑幕墙多材质图像的 Agent 自主任务调度方法和 LLM 分层信息整合策略：前者由分类 Agent 根据图像内容自主判断需要执行的原子检测能力，并将开放语义判断约束为工程系统可消费的任务代码；后者将大语言模型从简单文本生成工具扩展为“路由、压缩、汇总”的智能节点，有效解决了海量数据直接输入 LLM 模型带来的上下文过载和重点淹没问题。作者现已完成原型系统的部署和实际场景试用，获得同济大学土木工程学院科研团队认可，为建筑幕墙数字运维提供了一种兼具工程可实施性和智能决策能力的解决方案。 毕业设计：主要图纸 26 张，正文总字数 52547 字随附资料：建筑幕墙韧性评估软件系统源代码 系统编译、镜像构建、部署相关配置与脚本文件 系统编译产物（可执行二进制文件或部署包） 大厦无人机拍摄的同济大学地震工程馆建筑幕墙数据集（请列出所有提交的支撑材料名称，如：毕业设计-全套图纸、毕业作品、计算书、程序代码、附录等） 基于 AI Agent 的建筑幕墙韧性评估软件系统设计与实现 摘要

建筑幕墙是现代建筑的重要外围护结构，其服役状态直接影响建筑安全和运维质量。现实工程中，幕墙巡检长期依赖人工经验，存在效率低、主观性强、难以处理海量图像等问题。已有计算机视觉方法能够完成裂缝、锈蚀、破损等局部缺陷检测，但多数方案仍停留在单模型、单图像、单缺陷识别层面，只能检测某张图像是否存在某类缺陷，缺少多组巡检图像、多种幕墙材质、多类检测结果之间的汇总融合。对于土木工程领域用户而言，实际需求并不是孤立的单图检测结论，而是面向整栋建筑形成的多维度装 韧性评估结论。因此，为打通从项目级图像资产管理、单图检测结果到建筑级韧性评估 报告的完整链路，本文设计并实现了一套基于 AI Agent 的建筑幕墙韧性评估软件系统。本文的研究重点是 AI Agent 参与工程任务规划、能力调度与信息整合的系统性方法。本文工作的主要亮点包括两个方面：其一，提出面向建筑幕墙多材质图像的 Agent 自主任务调度方法。该方法由分类 Agent 根据图像内容自主判断需要执行的原子检测能力，并将开放语义判断约束为工程系统可消费的任务代码；其二，提出 LLM 分层信息整合策略。该策略将所有多源检测结果、项目背景、图像分组和人工复核意见等信息 线 进行分层整合，将大语言模型从简单文本生成工具扩展为“路由、压缩、汇总”的智能节点，有效解决了海量数据直接输入 LLM 模型带来的上下文过载和重点淹没问题。在工程实现上，原型系统主要使用Golang、Python 和 TypeScript 编程语言、React、Gin 和PyTorch 框架、gRPC 服务通信协议、RocketMQ 消息队列、WebSocket 实时消息推送、MySQL 关系型数据库、Redis 非关系型数据库、MinIO 对象存储和 Docker 容器化等技术。工程链路为 Agent 提供高可用、高性能、高并发的运行环境，原型系统提供从建筑图像资产构建、Agent 智能检测、人工复核到韧性评估报告的完整闭环工作链。作者现已完成原型系统的部署和实际场景试用，获得同济大学土木工程学院科研团队认可，为建筑幕墙数字运维提供了一种兼具工程可实施性和智能决策能力的解决方案。

关键词：AI Agent，建筑幕墙，任务编排，韧性评估 Design and Implementation of an AI Agent-Based Software System for Building Curtain Wall Resilience Assessment ABSTRACT Curtain walls are a common exterior

envelope in modern buildings. Their condition matters for safety, maintenance, and long-term operation, but routine inspection is still largely a manual job. In field work, engineers may already have a large number of UAV or camera images. The slow part often comes after that: sorting the images, checking the curtain wall material, finding defects, comparing results, and writing the final assessment. Computer vision can help with local defects such as cracks, corrosion, stains, or glass breakage, but many existing methods still work on a single image and a single defect type. What civil engineering users need is usually not a list of isolated image-level results. They need a building-level judgement: where the risks are, what types of defects are more serious, and how the curtain wall performs as a whole. Based on this problem, this thesis designs and implements an AI Agent-based software system for building curtain wall resilience assessment. The work mainly focuses on how to put AI Agents into an engineering workflow while keeping the process controllable. Before detection, a classification Agent reads the image content and chooses the required atomic detection tasks, such as corrosion, crack, stain, flatness, and spalling. Its output is limited to task codes, so the backend can schedule the following jobs in a clear and stable way. After detection, LLMs are used in stages. Single-image results are first turned into short summaries, and the project report is then generated from detection facts, project background, image groups, and human review comments. This design avoids sending a large amount of raw data directly to an LLM. It also makes the report easier to trace back to concrete evidence. The prototype is implemented with Golang, Python, and TypeScript. The main stack includes React, Gin, PyTorch, gRPC, RocketMQ, WebSocket, MySQL, Redis, MinIO, and Docker. In the implemented workflow, users can create a project, maintain building information, upload and group inspection images, start Agent-based detection, review the results, and generate a curtain wall resilience assessment report. The system has been deployed and tried in a real scenario, and the trial received recognition from a research team in the College of Civil Engineering at Tongji University. The result shows that the proposed approach is not only a set of detection scripts, but a usable software workflow for curtain wall digital operation and maintenance.

Key words: AI Agent, Building Curtain Wall, Task Orchestration, Resilience Assessment

目录

绪论 1 研究背景 1 研究问题 2 Agent 范式的场景适用性 3 系统技术定位 5 研究目标 5 研究内容与贡献 6

1.7 论文组织结构 7 相关研究基础 8 建筑幕墙巡检与数字化运维 8

2.2 幕墙缺陷检测技术 9 金属锈蚀检测原理 10 石材裂缝检测原理 11 石材污渍检测原理 11

2.2.4 玻璃平整度检测原理 12 2.2.5 玻璃爆裂检测原理 12 从单点检测到项目级评估的差距 13 AI Agent 的发展与工程范式 14 Agent 工程化的可控性问题 15 工程软件中的异步任务编排 17 LLM 信息整合与报告生成 18 本章小结 18 系统需求分析 20 业务闭环中的角色与责任 20 典型用例分析 22 3.3 功能需求分析..... 23 3.3.1 项目与权限需求..... 23 3.3.2 图像资产需求..... 24 3.3.3 智能检测需求..... 24 3.3.4 复核与报告需求..... 24 3.4 阶段化需求分析..... 25 3.5 非功能需求分析..... 26 3.5.1 可扩展性..... 27 3.5.2 一致性..... 27 3.5.3 可观测性..... 27 3.5.4 可部署性..... 28 3.5.5 用户体验..... 28 3.6 关键挑战..... 28 3.7 数据流分析..... 29 3.8 约束条件..... 31 3.9 本章小结..... 31 4 技术方案设计..... 33 4.1 关键术语..... 33 4.2 总体方法框架..... 33 4.3 Agent 在系统中的角色边界..... 36 4.4 技术路线取舍分析..... 37 4.4.1 方案评价维度..... 37 4.4.2 技术路线比较..... 38 4.4.3 技术路线组合逻辑..... 39 4.4.4 工业界实践启示..... 39 4.5 Agent 自主任务调度方法..... 40 4.5.1 任务调度问题建模..... 41 4.5.2 材质语义与任务代码映射..... 41 输出契约与服务端归一化 42 任务调度闭环设计 42 异常场景与边界处理 43 4.6.1 统一注册机制..... 43 4.6.2 统一执行协议..... 44 4.6.3 统一结果契约..... 44 4.6.4 工具化与 Agent 调度的关系..... 45 4.7 LLM 分层信息整合策略..... 46 4.7.1 信息分层整合的必要性..... 47 4.7.2 图像级总结设计..... 47 4.7.3 项目级报告源数据设计..... 48 4.7.4 项目级报告输出契约..... 49 4.7.5 幻觉与上下文漂移防止机制..... 49 4.7.6 信息分层整合与工程认知路径..... 50 4.6.1 统一注册机制..... 50 4.6.2 统一执行协议..... 50

.444.6.3统一结果契约	
.454.6.4工具化与 Agent 调度的关系	.454.7 LLM分层
信息整合策略	464.7.1信息分层整合的必要性
	474.7.2图像级总结设计
	474.7.3项目
级报告源数据设计	.484.7.4项目级报告输出契约
	494.7.5幻觉与上下文漂移防止机
制	494.7.6信息分层整合与工程认知路径
	504.6 原子检测能力工具化 43 装 订 4.8 任务状态流转与人机协同机制 50 4.9 本章小结 52 线 5 系统架
构与工程实现 53 关键术语 53 总体架构设计与实现 53 通信与协议设计 55 模块设计与实现 56 Core API 服务 56 Core	
BIZ 服务 57 Classification 服务 59 Reasoning 服务 60 Summary 服务 61 Core Web 服务 62 ICW Common 模块	
63 数据实体设计与实现 64 数据库设计与实现 65 缓存设计与实现 66 对象存储设计与实现 66 系统状态机设计与实现 67	
安全性设计与实现 68 稳定性设计与实现 70 部署设计与实现 72 本章小结 73 装 6 系统验证与应用分析 75 验证目标 75	
功能验证 75 订 6.3 Agent 自主任务调度方法验证 76 LLM 分层信息整合策略验证 77 状态一致性验证 77 部署验证 78	
线 6.7 验证结果展示 78 功能验证结果展示 79 Agent 自主任务调度方法验证结果 81 LLM 分层信息整合策略验证结果	
84 状态一致性验证结果 86 部署验证结果 86 应用价值分析 86 本章小结 87 总结与展望 88 全文总结 88 主要成果 88 未	
来工作展望 89 参考文献 91 附录 95 系统模块与 PSM 划分 95 HTTP 与 RPC 接口清单 96 icw.core.api 服务接口清单	
96 icw.core.biz 服务接口清单 97 icw.activity.classification 服务接口清单 99 icw.activity.reasoning 服务接口清单	
99 icw.activity.summary 服务接口清单 99 装 C 环境变量配置 99 icw.core.web 服务环境变量配置 100 icw.core.api	
服务环境变量配置 100 订 C.3 icw.core.biz 服务环境变量配置 100 icw.activity.classification 服务环境变量配置 101	
icw.activity.reasoning 服务环境变量配置 102 icw.activity.summary 服务环境变量配置 102 线 谢辞 104	

绪论 研究背景 建筑幕墙是现代建筑的重要外围护结构,承担围护、采光、隔热、防水、防风压、节能和外观表达等多重功能。随着城市中大量公共建筑、商业建筑和高层建筑进入长期服役阶段,幕墙系统的安全性、耐久性和可维护性逐渐成为建筑运维中的重要问题。幕墙材料长期暴露在风、雨、温差、紫外线、污染物和偶发冲击环境中,可能出现玻璃爆裂、玻璃平整度异常、金属锈蚀、石材裂缝、石材污渍以及连接构件退化等问题。这些问题如果不能及时发现和评估,不仅会影响建筑外观,还可能对围护性能、使用安全 和维护成本产生持续影响。装 在建筑运维数字化和 BIM 1 技术发展的背景下,幕墙评估的目标正在从“发现某个缺陷”转向“理解建筑外围护系统的整体状态”。韧性概念强调系统在扰动或退化条件下维持关键功能、恢复功能和适应环境变化的能力[2]。对于建筑幕墙而言,韧性评估 订 不仅需要知道某张图像中是否存在缺陷,还需要回答缺陷分布在哪些区域、不同材料上的损伤类型是否具有聚集趋势、局部损伤对整体运维决策有何意义、后续维护应优先关注哪些问题等更高层次的问题。现实工程中,幕墙巡检仍然大量依赖人工经验。检测人员需要面对高空作业、大面 线 积立面、海量图像、复杂材料和不一致的现场环境。即便无人机和高清相机降低了图像采集难度,后续的图像整理、材质判断、缺陷识别、结果汇总和报告撰写仍然消耗大量人工。实际业务中经常出现“图像采集已经完成,但评估报告迟迟难以形成”的情况:现场已经获得大量巡检图像,但图像缺少面向建筑区域的分类整理,单张图像的检测结果彼此割裂,缺陷类型、分布区域、风险倾向和维护优先级需要工程人员反复比对、筛选和汇总。已有计算机视觉方法能够完成裂缝、锈蚀、破损等局部缺陷检测,但多数方案仍停留在单模型、单图像、单缺陷识别层面,只能检测某张图像是否存在某类缺陷,缺少多组巡检图像、多种幕墙材质、多类检测结果之间的汇总融合。如图1.1 所示,对于土木工程领域用户而言,实际需求并不是孤立的单图检测结论,而是面向整栋建筑 1 BIM (Building Information Modeling),即建筑信息模型。它通过数字化方式组织建筑的几何形态、构件属性、空间关系和运维信息,使建筑全生命周期中的设计、施工、检测和维护数据能够被统一表达和管理[1]。形成的包括幕墙整体状态、主要缺陷类型、缺陷集中区域、风险倾向等多维度韧性评估结论。传统幕墙巡检流程无人机或人工采集图像人工整理图像文件单图单缺陷检测人工比对汇总结果评估报告形成滞后传统幕墙巡检流程无人机或人工采集图像人工整理图像文件单图单缺陷检测人工比对汇总结果评估报告形成滞后用户真实评估需求项目级图像统一管理按图像内容自动分流多类缺陷协同检测单图结果可读总结建筑级韧性评估报告用户真实评估需求项目级图像统一管理按图像内容自动分流多类缺陷协同检测单图结果可读总结建筑级韧性评估报告从单图割裂 到建筑级融合 装 订 图 1.1 传统幕墙巡检流程与用户真实评估需求对比 因此,本文关注的核心矛盾并不仅是单个检测模型能否识别某类缺陷,而是如何把图像采集、资产组织、任务调度、检测结果解释和建筑级报告生成连接为完整闭环。线 研究问题 本文关注的研究问题可以概括为三个层次。第一个层次是数据规模与组织问题。一个建筑项目可能包含大量幕墙图像,图像之间还需要按照立面、楼层、区域或采集批次进行组织。若缺少图像资产管理和分类整理能力,图像只能作为普通文件存在,难以支撑后续检索、复核、状态追踪和报告生成。更重要的是,建筑幕墙评估不能只停留在“图像数量很多”的层面,还需要从建筑整体维度理解不同区域的缺陷类型、空间分布、风险倾向和维护优先级。例如,同样是锈蚀缺陷,如果集中出现在某一立面或特定高度区间,其工程意义与零散出现完全不同;同样是污渍问题,如果与局部渗水或排水异常相关,也需要在报告中形成区域化判断。因此,系统需要同时支持项目级图像资产管理和建筑维度的区域化汇总能力。第二个层次是任务选择与调度问题。幕墙图像中可能包含玻璃、金属、石材或混凝土等不同材料,不同材料对应的检测能力不同。传统固定流程会对每张图像执行全部检测模型,这会造成计算浪费和语义噪声。例如,对玻璃幕墙图像执行石材裂缝检测并无实际意义,对石材图像执行玻璃平整度检测也难以产生可信结论。另一种做法是由人工先判断图像材质,再手动把图像分配到不同检测模型中,但在批量巡检场景下,这种流程需要大量重复性人工分类,效率低、成本高,并且不同人员之间判断标准不一定一致。因此,系统需要一种能够根据图像内容自主选择检测能力的机制,将开放视觉语义判断转化为稳定的工程任务代码。第三个层次是韧性评估与报告生成问题。检测模型通常输出区域坐标、像素占比、置信度、掩码图像和浮层图像等底层结果。这些结果对算法复核有价值,但不能直接构成建筑运维人员可读的韧性评估报告。如果直接把所有检测结果拼接给大语言模 装 型,海量数据内容会造成上下文过长、信息重复、重点不清和报告生成不稳定等问题。因此,系统需要一种从原子检测结果到图像级总结,再到项目级报告的中间组织机制,使单图单点检测结果能够逐步转化为面向建筑整体的缺陷概述、风险判断和维护建议。

研究问题层次系统能力指向风险判断、项目整体评估、维护建议材质识别、检测能力选择、任务代码生成项目 / 图像组 / 图像资产管理韧性评估与报告生成问题任务选择与调度问题数据规模与组织问题研究问题层次系统能力指向风险判断、项目整体评估、维护建议材质识别、检测能力选择、任务代码生成项目 / 图像组 / 图像资产管理韧性评估与报告生成问题任务选择

与调度问题数据规模与组织问题订 线图 1.2 建筑幕墙韧性评估的三层研究问题模型 如图1.2 所示, 上述三个层次共同构成本文的研究问题: 如何设计一套面向建筑幕墙韧性评估的软件系统, 使其能够管理项目级图像资产, 基于图像内容自主选择检测能力, 调度原子检测工具, 并将多源检测结果逐层整合为项目级评估报告。Agent 范式的场景适用性 传统软件系统擅长执行确定流程, 但面对“先判断场景, 再选择工具, 再综合输出结论”的任务时, 往往需要大量规则和人工配置。建筑幕墙检测正属于这种情况。图像 是否属于幕墙、主要材料是什么、应该运行哪些检测能力、检测结果如何转化为工程语言, 这些问题具有一定语义判断和上下文理解需求。单纯依靠硬编码规则难以覆盖复杂场景, 单纯依靠检测模型又无法完成任务规划和报告表达。AI Agent 范式为该问题提供了新的解决思路。近年来, 学术界和工业界逐渐将大语言模型从“文本生成器”扩展为“任务规划和工具调用中心”。ReAct 方法强调推理和行动交替进行, 使模型能够在推理过程中调用工具并利用工具反馈更新计划[3]; Toolformer讨论语言模型学习使用外部API 的能力[4]; AutoGen 等框架进一步展示了多Agent 协作和可编程智能工作流的工程价值[5]。这些研究表明, Agent 的核心价值不只是输出自然语言, 而是把理解、规划、工具调用和结果整合连接起来。如图1.3 所示, AI Agent 范 式通常不是一次性生成答案, 而是在任务输入、语义理解、任务规划、工具调用、观察 装 任务输入与上下文语义理解与目标识别任务规划与工具选择工具调用与外部执行结构化输出与状态反馈观察反馈与结果校验任务输入与上下文语义理解与目标识别任务规划与工具选择工具调用与外部执行结构化输出与状态反馈观察反馈与结果校验反馈和结构化输出之间形成闭环。订 线图 1.3 AI Agent 范式的一般工作流 本文将Agent 引入建筑幕墙韧性评估, 并不是为了给系统增加一个对话入口, 而是为了让Agent 承担三个具体职责: 在检测前完成任务路由, 在检测后完成图像级语义压缩, 在项目结束时完成报告级信息融合。这样的设计使 Agent 真正嵌入业务链路, 而不是作为孤立的问答功能存在。如果采用传统规则系统, 则需要人工配置材料和检测任务之间的映射关系。例如, 当用户选择“玻璃幕墙”时, 系统固定执行玻璃平整度和玻璃爆裂检测; 当用户选择“石材幕墙”时, 系统固定执行裂缝和污渍检测。但是现实图像往往并不如此简单。一张图像可能同时包含玻璃和金属框架, 也可能包含非幕墙背景、反射、遮挡或局部构件。完全依赖用户手动选择材料, 会降低批量处理效率; 完全依赖硬编码规则, 又难以 处理复杂视觉语义。Agent 的价值正在于它可以在有限工具集合内完成语义判断, 从而把复杂场景判断转化为工程系统可执行的任务代码。如果直接让大语言模型生成最终报告, 也存在明显问题。幕墙项目可能包含多张图像, 每张图像又包含多个检测任务和多个区域结果。原始结果中大量坐标、像素、置信度和图像产物信息会干扰报告生成。本文没有让 Agent 一步完成所有任务, 而是将其拆分为三个受控节点: 分类路由、图像级总结和项目报告。这种设计既符合当前Agent工程化中的工具调用思想, 也符合建筑韧性评估从局部观察到整体判断的认知过程。系统技术定位 本项目是一个以工程系统为载体、以 Agent 能力为核心亮点的毕业设计。工程系统 装 本身采用的是相对成熟的技术组合, 例如前后端分离、gRPC、消息队列、对象存储、数据库事务和容器化部署等。这些工程技术是系统稳定性、可追踪性和可部署性的基础。没有工程链路, Agent 只能处理单次输入; 有了工程链路, Agent 的判断才能驱动任务 创建、状态推进、结果落库、异常恢复和报告生成。订 因此, 本系统的技术定位有别于“实现一个视觉检测模型”或“实现一个通用后台 管理系统”, 而是“构建一个 Agent 驱动的幕墙缺陷检测与韧性评估方法框架”。工程链路提供高可用、高性能和高并发的运行保障, 视觉检测能力提供原子事实来源, Agent 线 自主任务调度方法和 LLM 分层信息整合策略则实现真正的 AI 赋能。换言之, 工程链路实现保证系统可用性, Agent 方法决定系统能否从单点检测提升到建筑维度评估。研究目标 本文的研究目标是设计并实现一套基于 AI Agent 的建筑幕墙韧性评估软件系统。系统需要完成从项目创建、图像资产管理、Agent 智能检测、人工复核确认到评估报告生成的完整流程。在智能检测阶段, 本文重点研究面向建筑幕墙多材质图像的 Agent 自主任务调度方法, 通过分类 Agent 判断每张图像需要执行哪些原子检测能力; 在原子检测阶段, 系统需要稳定调度金属锈蚀、石材裂缝、石材污渍、玻璃平整度和玻璃爆裂五类检测能力; 在信息整合阶段, 本文重点研究LLM 分层信息整合策略, 将单张图像的检测结果生成图像级总结, 再将所有图像总结、项目背景、图像分组和复核信息汇总为项目级韧性评估报告。

从工程目标看, 系统需要具备清晰模块边界和可部署性。前端需要提供完整项目流程和结果展示, 后端需要支持长耗时任务、异步回调、实时状态推送、数据库事务、对象存储和容器化部署。工程实现不是本文的唯一亮点, 但它是 Agent 能力落地的必要保障。没有稳定工程链路, Agent 自主任务调度方法和 LLM 分层信息整合策略只能停留在脚本演示阶段, 无法成为实际可用的软件系统。研究内容与贡献 本文主要研究内容包括: (1) 调研建筑幕墙检测、BIM 与AI Agent 应用范式, 分析现有单点检测方案在建筑维度评估中的不足; 装 (2) 建立幕墙韧性评估软件的问题模型, 明确图像资产、检测任务、人工复核和报告生成之间的关系; 提出面向建筑幕墙多材质图像的 Agent 自主任务调度方法, 使系统能够根据 图像内容动态选择检测能力; 订 提出 LLM 分层信息整合策略, 将原子检测结果先转化为图像级总结, 再融 合为项目级韧性评估报告; 交付可运行软件原型系统, 实现前端交互、服务通信、异步任务、状态推送、线 对象存储、容器化部署和实际场景试用。本文贡献主要体现在以下三个方面: 将 Agent 工具调用思想引入建筑幕墙评估场景, 提出 Agent 自主任务调度方法, 使 Agent 的语义判断能够被工程状态机消费, 降低批量场景下人工材质分类和任务选择成本; 提出面向建筑幕墙多材质图像的 Agent 自主任务调度方法, 使系统能够根据图像内容动态选择检测能力; 提出 LLM 分层信息整合策略, 用图像级总结作为项目级报告生成的中间语义层, 解决海量检测结果直接汇总的上下文压力和信息噪声问题, 使单图检测结论能够上升为建筑维度评估结论; 构建完整的工程闭环, 将 Agent 自主任务调度方法、视觉检测模型、结构化 落库、人工复核和报告生成连接为可部署系统, 体现了工程实践与 Agent 方法结合的价值。论文组织结构 全文共七章。第一章介绍研究背景与问题、Agent 范式的场景适用性、系统技术定位、研究目标、内容与贡献。第二章梳理建筑幕墙缺陷检测、计算机视觉方法、AI Agent范式、建筑幕墙韧性评估的相关研究基础。第三章对问题进行建模, 分析系统需求、角色、数据对象和相关约束条件。第四章详细阐述本文提出的 Agent 自主任务调度方法和 LLM 分层信息整合策略等技术方案设计。第五章介绍系统架构与工程实现, 重点说明如何构建高可用、高性能和高并发的工程链路, 并支撑 Agent 能力稳定落地。第六章进 装 行系统验证与应用分析, 说明系统在功能、流程、部署和试用层面的完成情况。第七章总结全文工作内容并展望后续研究方向。订 线 相关研究基础 本章从建筑幕墙巡检与数字化运维、幕墙缺陷检测技术、AI Agent 工程范式和韧性评估报告生成四个方向梳理本系统的相关研究基础。现有方法在单类缺陷检测能力上已经具有一定积累, 但在项目级任务编排、Agent 自主任务调度和 LLM 分层信息整合方面仍存在一定空白。建筑幕墙巡检与数字化运维 建筑幕墙巡检长期以来依赖人工现场勘查。人工巡检能够结合经验进行综合判断, 但在高层建筑、大面积幕墙和复杂环境下效率较低, 且存在安全风险。随着无人机技 装 术和图像采集设备的发展, 基于无人机的建筑立面巡检逐渐成为重要方向。Freimuth 和 König 对无人机建筑检查的规划和执行流程进行了系统研究, 说明无人机能够降低高空检测成本并提升数据采集效率[6]。在此基础上, 红外热成像、图像配准和 BIM、AI 结 合等方法进一步扩展了建筑立面检测的数据来源、空间定位能力和智能分析能力[7-9]。订 但是, 数字化采集并不等于智能化评估。无人机可以快速采集图像, 但图像采集之 后仍需要图像管理、缺陷检

测、结果解释和报告生成。若缺少后续软件系统，采集到的图像仍然是离散资料，难以转化为运维决策。因此，建筑幕墙评估系统需要将采集后的 线 图像组织为项目资产，并建立从图像收集到评估报告生成的完整处理流程。从工程应用角度看，采集设备能够提高图像获取效率，检测模型能够提高局部缺陷识别效率，但真正面运维单位和工程师的交付物通常是带有结论、依据和建议的评估报告。这意味着系统需要同时处理视觉证据、结构化指标、项目语义和文本表达。若系统只停留在缺陷框或掩码图层面，仍然需要人工把这些结果重新解释、筛选、汇总和撰写。因此，图像检测之后的组织、解释和报告生成，是建筑幕墙从数字化运维走向智能化运维的关键环节。幕墙工程对象同时具有明显的项目属性。同一种缺陷出现在不同区域，其工程意义可能不同；同一张图像中的不同材料，也可能对应不同检测任务。传统视觉检测研究往往把图像视作独立样本，而实际运维更关心项目内图像之间的关系、图像组与建筑区域的关系、缺陷分布与维护优先级之间的关系等。本文的系统设计正是围绕这种项目级应用需求展开。幕墙缺陷检测技术 在本文讨论的幕墙巡检场景中，图像是最直接也最容易获得的数据来源。算法面对的通常不是标准实验图，而是带有反光、阴影、倾斜视角、材料纹理和背景遮挡的现场照片。因此，缺陷检测一般要完成几件比较实际的工作：判断缺陷类型，给出缺陷位置，并尽量形成面积、长度、平整度或置信度等可解释指标。早期或轻量化场景中仍会使用传统计算机视觉方法，例如利用 Canny 边缘检测和 Hough 直线检测提取轮廓或线性结构，也可以结合 Sobel 梯度、Laplacian 方差、傅里叶频域分析和形态学开闭运算描述纹理突变、模糊程度或局部形态变化[10-11]。这类方法的优点是实现过程清楚，对大规

装 模标注数据依赖较小，在裂缝、边缘和规则构件分析中也比较容易解释；不足是对现场条件较敏感，换到另一种光照、另一类材质或另一批拍摄角度后，阈值和规则往往需要重新调整。深度学习方法把特征提取更多交给数据本身。卷积神经网络不再手工指定每一个边缘或纹理规则，而是在训练过程中逐层形成从低级纹理到高层语义的图像表征；ResNet 等残差网络进一步通过残差连接缓解深层网络训练困难，因而常被用于图像分类任务[12-13]。如果任务关注“缺陷在哪里”，目标检测模型会更直接，它可以同时给出类别和位置。基于候选区域的两阶段方法，以及以 YOLO、SSD 为代表的一阶段方法，是目标检测中较常见的两条路线；国内已有综述对 R-CNN、Faster R-CNN、YOLO 和 SSD 等代表性算法进行了梳理[14]。其中，YOLO 系列强调端到端和实时检测，在需要快速定位缺陷候选区域时较为适用[15]。当评估需要更细的边界或面积信息时，仅有检测框往往不够，语义分割和实例分割模型会更有价值。U-Net 等编码器-解码器结构是像素级分割中的典型思路，相关文献综述已经对深度学习语义分割中的区域分类、像素分类、全监督和弱监督方法进行了总结[16]。DeepLabV3+ 通过空洞卷积和编码器-解码器结构提升语义分割效果[17]；视觉 Transformer 借助自注意力机制增强长距离依赖建模能力，为视觉识别、目标检测和视觉分割提供了新的表征基础[18]；SegFormer 则进一步将 Transformer 表征用于高效语义分割[19]。此外，Mask R-CNN 等实例分割方法和 Segment Anything 分别从实例掩码预测、通用分割能力等方向扩展了区域分割的使用范围[20-21]。放到建筑结构和幕墙巡检中，裂缝检测、玻璃检测和表面缺陷识别已经有较多积累。Ye 等人利用深度学习方法进行结构裂缝检测，可为石材或混凝土裂缝识别提供参考[22]；Mei 等人研究真实场景中的玻璃检测问题，对玻璃幕墙区域分析也具有参考意义[23]。原子检测能力 核心技术路线 结构化输出 金属锈蚀锈蚀区域、置信度锈蚀像素数与占比YOLO / DeepLabV3+检测与语义分割金属锈蚀锈蚀区域、置信度锈蚀像素数与占比YOLO / DeepLabV3+检测与语义分割石材裂缝裂缝掩码、覆盖图裂缝像素数与占比YOLO / SAM / SegFormer候选区域与裂缝分割石材裂缝裂缝掩码、覆盖图裂缝像素数与占比YOLO / SAM / SegFormer候选区域与裂缝分割 石材污渍污渍区域、校正图平均与最高污渍占比YOLO 实例分割 透视校正与局部分析石材污渍污渍区域、校正图平均与最高污渍占比YOLO 实例分割 透视校正与局部分析装 玻璃平整度平整判定指标 边缘 / 直线 / 梯度 / 频谱Canny / Hough / Sobel / FFT多特征规则融合玻璃平整度平整判定指标 边缘 / 直线 / 梯度 / 频谱Canny / Hough / Sobel / FFT多特征规则融合玻璃爆裂爆裂 / 未爆裂判定分类置信度ResNet34 二分类数据清洗与增强训练玻璃爆裂爆裂 / 未爆裂判定分类置信度ResNet34 二分类数据清洗与增强训练图 2.1 五类原子检测能力核心技术路线总览 线 如图 2.1 所示，Agent 自主调度的五类原子检测能力来自团队前期积累的检测模型和实验脚本，本系统将这些模型抽象为可调度、可替换、可治理的原子检测能力。在系统链路中，原子检测能力负责给出局部视觉判断，Agent 自主任务调度方法负责选择执行的能力，LLM 分层信息整合策略负责将局部结果组织为图像总结和项目报告。本节对五类原子检测能力的基本原理进行说明。金属锈蚀检测原理 金属构件发生锈蚀后，图像中通常会出现颜色加深、边界不规则、局部纹理粗糙等特征。该检测能力需要定位锈蚀区域，并输出锈蚀区域数量、置信度、锈蚀像素数和锈蚀占比等指标。为了兼顾检测速度和区域精度，系统在这一部分保留了两条实现路径。一条是基于YOLO 的检测或分割路径，适合快速获得锈蚀候选区域，并输出缺陷框、掩码和类别置信度；另一条是基于 DeepLabV3+ 的语义分割路径，将锈蚀检测建模为二分类像素分割问题，通过编码器提取图像特征，再利用空洞卷积扩大感受野，以识别边界不规则的锈蚀区域[17]。在代码实现中，DeepLabV3+ 使用 MobileNetV2 作为轻量化骨干网络，用于降低推理成本并适配工程部署[24]。推理过程中，脚本读取原始图像后，会完成尺度变换、张量化和归一化处理，并得到锈蚀概率图。对于语义分割路径，二值掩码会根据概率分布生成：当整体概率较高时使用 Otsu 自适应阈值，其他情况下使用置信度阈值对概率图进行二值化。为减少零散噪声，脚本还会执行形态学开运算，并依据连通域或轮廓面积过滤过小区域。经过上述处理，系统可以根据掩码统计锈蚀像素数、锈蚀面积占比、区域数量和平均置信度，同时生成锈蚀覆盖图供前端展示。由此得到的锈蚀检测结果既包含可视化证据，也包含可以继续参与项目级评估的结构化指标。装 石材裂缝检测原理 石材或混凝土幕墙表面的裂缝通常较细，形态也不完全规则。现场图像中还会出现石材纹理、阴影、接缝和拍摄噪声等干扰，使裂缝边界不一定清楚。若只依赖边缘检测或固定阈值，容易把石材纹理误判为裂缝，也可能漏检对比度较低的真实裂缝。基于这一特点，裂缝检测脚本采用深度分割路线，将裂缝识别处理为像素级语义分割任务。原始系统并不是直接对整张图进行一次性判断，而是包含幕墙区域检测、区域分割和裂缝分割等步骤；其中可结合 YOLO 旋转框模型、MobileSAM 或 SegFormer 等能力，先获得相对稳定的石材候选区域，再由裂缝分割模型输出裂缝掩码[19,21]。

在裂缝分割阶段，图像经过预处理后输入分割模型，模型输出每个像素属于裂缝类别的概率。后处理部分会将概率图转化为二值掩码，并完成尺寸恢复和高亮叠加，形成裂缝覆盖图、裂缝掩码图和裂缝区域统计结果。相比只输出目标检测框，像素级掩码能够更细致地表达裂缝长度、宽度和空间走向，也便于进一步计算裂缝像素数和裂缝占比。对后续评估而言，这类结果比单纯判断“是否存在裂缝”更有使用价值。石材污渍检测原理 石材污渍与裂缝的视觉形态有所不同。裂缝多表现为细长线状结构，污渍则通常来自污染、渗水痕迹或颜色异常，呈现块状或片状变化，边界不一定清晰，并且容易受到拍摄角度和透视变形影响。若直接在整张图像上计算污渍比例，不同幕墙块的倾斜

程度会影响面积统计的准确性。为降低这一影响，污渍检测脚本采用“幕墙块分割、透视校正、局部污渍分析”的组合格

高度疑似
AIGC
(99.96%)

高度疑似

线。脚本先使用 YOLO 模型对图像中的幕墙块进行实例分割，得到每个候选幕墙块的掩码；之后根据掩码轮廓计算凸包，并通过多边形近似寻找四个角点；当角点按照左上、右上、右下、左下的顺序排列后，系统即可利用透视变换将倾斜拍摄的幕墙块校正为正视局部图像。完成透视校正后，局部幕墙块会进入传统图像处理算法，用于估计污渍面积。原始脚本调用污渍分析模块，计算每个校正块的污渍比例，并生成对应的结果图像。不同输出图像承担的作用也不相同：整体标注图用于查看所有候选幕墙块的位置，局部校正图用于确认检测区域，结果图用于展示污渍分析效果。该方法将深度学习模型用于区域定位，将传统图像处理用于比例计算，二者分工较为清楚。由此得到的“平均污渍占比”和“最高污渍占比”不只是整图层面的粗略统计，而是具有相对明确的区域含义。玻璃平整度检测原理 玻璃平整度检测关注玻璃幕墙中可能出现的局部变形、反射异常或不平整现象。与 订 裂缝、锈蚀不同，玻璃平整度问题在图像上往往缺少明确的缺陷边界。由于玻璃具有透明、反光和低纹理等特点，真实的不平整更可能表现为边缘畸变、直线偏折、梯度分布异常或频域能量变化。基于这些视觉特征，脚本采用“玻璃区域识别 + 多特征规则判 线 别”的实现路径，而不是直接使用单一分类结果。系统会先通过玻璃检测或分割模型获得玻璃区域，再将后续分析限制在局部区域内，以减少背景和窗框对判断的干扰。

在局部玻璃区域中，脚本先进行裁剪，再使用 Canny 方法提取边缘，并计算边缘数量和 Laplacian 方差，用于反映局部纹理和清晰度变化[10]。直线分析部分通过概率 Hough 变换检测线段，并统计线段角度标准差，用于判断反射线或边缘线是否出现异常偏折[11]。梯度分析利用 Sobel 算子计算梯度幅值，并统计梯度均值和梯度标准差；频域分析则对灰度图执行离散傅里叶变换，观察频谱能量差异。系统将直线、梯度和频域三个判别结果进行融合，并通过多数投票得到玻璃是否不平整的结论。玻璃爆裂检测原理 玻璃爆裂检测在系统中对应一个二分类任务，用于判断玻璃幕墙图像中是否存在破裂、爆裂或碎裂风险。脚本采用基于 ResNet34 的二分类模型，将输入图像划分为“存

在缺陷”和“不存在缺陷”两类。模型以 ResNet34 作为骨干网络，并将末端全连接层替换为二分类输出层。推理时，图像会被缩放到 224 × 224，转换为张量后按照 ImageNet均值和标准差进行归一化，再输入模型得到类别预测。虽然上层系统看到的是一个二分类接口，但该能力仍依赖较完整的数据准备和训练前处理流程。数据清洗阶段会检查图像完整性、文件重复、分辨率、标注缺失和拍摄角度；数据增强阶段使用旋转、水平翻转、缩放、亮度对比度扰动、高斯噪声、椒盐噪声和透视畸变等方式，提高模型对不同拍摄条件的适应能力；数据划分阶段则按照工程类型、区域和年份构造分层键，再将数据划分为训练集、验证集和测试集。原始脚本中还保留了基于 Canny 与 Hough 直线的辅助处理，用于过滤与横纵轴近似平行的线性

纹理干扰。由此可见，玻璃爆裂检测虽然在系统中呈现为简单分类结果，但背后仍包含 装 数据治理、数据增强和模型训练等多个环节。 综上，五类原子检测能力在技术路线和输出形态上存在差异：金属锈蚀和石材裂缝更偏向分割，石材污渍结合了实例分割与透视校正，玻璃平整度依赖多种视觉特征 订 融合，玻璃爆裂则采用深度分类模型。本文将这些差异封装在统一的原子能力接口之后，使上层 Agent 不需要感知每个算法的内部实现，而只需要基于图像内容选择任务代码并接收结构化结果。这也是本文从单点检测算法走向工程化智能评估系统的基础。 线 2.3 从单点检测到项目级评估的差距

已有幕墙缺陷检测研究通常以单一任务为目标，例如裂缝识别、玻璃检测或表面污渍检测。这类研究能够回答算法层面的问题：给定输入图像，模型是否能够准确输出目标区域。然而，项目级评估需要回答更复杂的问题：哪些图像需要执行哪些检测，多个检测结果之间如何互相补充，哪些缺陷对项目整体风险影响更大，报告中应如何表达证据与结论之间的关系。如图 2.2 所示，从单点检测到项目级评估之间至少存在四个缺口。第一是数据组织缺口，图像需要按照项目、图像组、图像和检测任务多层组织。第二是任务编排缺口，系统需要根据图像内容动态选择检测能力，而不是机械执行全部模型。第三是结果解释缺口，模型输出的像素和坐标指标需要转化为工程语言。第四是报告生成缺口，系统需要把多张图像的局部结论融合为项目级评估意见。本文的研究价值正是在这些缺口

处构建桥梁。单点检测算法 必须跨越的缺口 项目级评估问题 输入对象单张图像 输出结果 缺陷框 / 掩码 / 分类 核心回答 这张图有什么缺陷 数据组织缺口 图像进入项目 / 图像组 / 资产 任务编排缺口 材质识别与检测能力选择 结果解释缺口 像素坐标指标需转化为工程语义 报告生成缺口 局部结论需融合为建筑级报告 哪些图像需要检测 缺陷如何分布风险如何判断报告如何生成 图 2.2 从单点检测到项目级评估的跨越关系 装 如果只关注检测模型，系统会缺少业务闭环；如果只关注 Web 管理界面，系统又缺少智能决策能力。本文选择的路线是以工程系统组织检测模型，以 Agent 承担语义决 策和信息整合，从而把二者连接为完整应用。 订 AI Agent 的发展与工程范式 大语言模型出现以后，Agent 逐渐成为人工智能应用中的一种重要组织方式。与早期主要用于文本生成和问答的模型不同，Agent 更关注模型如何围绕用户目标与外部工 具、运行环境和任务反馈发生交互。ReAct 将推理过程和行动过程放在同一条链路中，使模型可以在形成思路的同时执行具体任务动作[3]。Toolformer 则进一步讨论了语言模型学习工具使用的问题，包括何时调用工具、如何传递参数以及如何利用工具返回结果[4]。从这些研究可以看出，大语言模型要进入实际任务场景，不能只依靠自然语言生成能力，还需要通过工具调用机制把语义理解转化为可执行操作。在工业界和开源社区中，Agent 也逐渐从单次问答走向工作流开发。AutoGen 通过多智能体对话组织复杂任务，使不同角色、不同能力的 Agent 可以作为可编程组件参与协作[5]。相关综述也指出，大语言模型Agent 可用于单智能体、多智能体和人机协作等场景，规划、记忆、工具使用和反馈学习是其中较为核心的能力[25]。不过在工程系统中，Agent 通常不会被设计成完全自由行动的主体，而是需要放入受控流程中运行。结构化输入输出、工具白名单和状态管理等机制，能够为 Agent 的调用边界、执行过程和 结果回收提供约束。本文正是基于这一认识，将 Agent 设计为业务链路中的受控智能节点。分类 Agent不是自由选择任意工具，而是在预定义五类任务代码中选择；总结 Agent 不是直接修改系统状态，而是通过回调接口提交总结结果；项目报告 Agent 不是自行读取数据库，而是接收由 BIZ 服务组织好的项目上下文。这样的设计符合当前 Agent 工程化趋势：让模型承担语义理解和决策任务，让工程系统负责边界和状态。 表 2.1 大语言模型应用范式的演进 范式 主要特征 局限与本文启发 文本生成 模型根据提示词直接输出自然语言 适合问答和撰写，但难以与业务状态和外部工 检索增强 先检索外部知识，再生成回答 具形成闭环 能补充上下文，但仍偏向知识问答，不直接解 决任务调度问题 装 工具调用 模型根据目标选择外部 API 或工具 适合连接模型与工程能力，但需要严格约束工 具集合和参数 工作流 Agent Agent 被放入确定业务流程，承担局部决策 更适合工程落地，本文采用该路线实现检测路 由和信息整合 订 表 2.1 展示了大语言模型应用范式从文本生成到工作流Agent 的变化。本文将 Agent视为工程工作流中的智能决策节点，而非完全自主的软件机器人。这样的设计更符合垂直行业系统的要求：业务流程必须可解释，数据状态必须可追踪，外部工具调用必须可控，失败情况必须可恢复。 线 Agent 工程化的可控性问题 Agent 引入工程系统后，可控性问题会比普通问答场景更突出。此时模型是否“答得像样”并不是唯一标准，更重要的是它

AIGC (98.9%)

中度疑似
AIGC
(83.52%)

高度疑似
AIGC
(95.25%)

中度疑似
AIGC
(81.75%)

的输出能否被后端服务稳定解析，能否与数据库状态、任务队列和人工复核流程对齐。大语言模型本身具有概率性，实际使用时可能出现格式不稳定、内容越界、幻觉或工具选择错误等情况。例如，同一类请求在不同轮次中可能出现字段缺失、字段命名变化，或者把解释性文字混入结构化结果中。已有研究对幻觉问题进行了较多讨论，其风险在于模型生成的内容表面上较为合理，但缺少事实依据，放入真实系统后会严重影响信息可靠性[26]。在工具调用场景中，这类风险还会继续向后传递。模型不仅要理解用户目标，还要判断调用时机、工具类型和参数组织方式；如果工具选择错误，后续 Worker 就可能执行不适合当前图像的检测任务。Toolformer、ToolLLM 等研究表明，大语言模型可以学习或增强工具使用能力；但从工程角度看，这也意味着工具使用能力不能只依赖模型自由发挥，而要依赖工具集合、调用路径和评估机制进行约束[4,27]。如果只是普通问答系统，上述问题更多体现为回答质量波动；如果 Agent 的输出会进一步影响数据库状态或任务调度，就可能带来错误流程。因此，结构化输出、工具白名单、状态隔离和人工复核，是 Agent 工程化中必须处理的基础问题。核心原则：Agent 承担受控决策，工程系统负责边界、状态与恢复 Agent 工程化风险工程约束机制结果可信度不足异常不可恢复状态误推进幻觉与事实偏移工具选择错误输出格式不稳定人工复核 / 评论修正 / 审核闭环状态机 / 失败记录 / 重试机制BIZ 统一推进状态 / Agent 不直接写库结构化上下文 / 系统组装数据源工具白名单 / 有限任务代码JSON Schema / 固定字段 / 解析校验Agent 工程化风险工程约束机制结果可信度不足异常不可恢复状态误推进幻觉与事实偏移工具选择错误输出格式不稳定人工复核 / 评论修正 / 审核闭环状态机 / 失败记录 / 重试机制BIZ 统一推进状态 / Agent 不直接写库结构化上下文 / 系统组装数据源工具白名单 / 有限任务代码JSON Schema / 固定字段 / 解析校验

订图 2.3 Agent 工程化风险与约束机制关系如图2.3 所示，针对上述风险，现有研究和工程实践通常会从动作、输出和流程三 线 个层面进行约束。这里的约束并不是简单减少 Agent 的能力，而是把模型适合完成的语义判断留给模型，把状态推进、数据落库和异常恢复交给工程系统处理：一是限制动作空间，让 Agent 只能在预定义工具、API 或任务代码中选择。动作空间越开放，错误工具选择后的影响就越难控制；而有限动作空间可以把模型输出收敛到工程系统能够识别和处理的范围内。ReAct 通过推理与行动交替形成较清晰的行动轨迹，AutoGen 则通过多 Agent 对话和人类参与机制组织复杂任务流程[3,5]；二是约束输出格式，使模型结果能够被程序稳定解析。自然语言回答对用户阅读比较友好，但对后端程序并不稳定，因此工程系统通常需要固定字段、类型检查和解析校验。PICARD 通过增量解析约束自回归解码，Outlines 相关研究则将文本生成表示为有限状态机转移，并借助正则表达式或上下文无关文法限制生成结构[28-29]；三是把 Agent 放入受控工作流，由工程系统负责状态推进、失败重试、日志记录和人工复核。即使 Agent 的输出格式正确，也不能说明它可以直接修改业务状态。更稳妥的方式是让 Agent 提供判断结果，由业务服务决定结果何时生效、失败如何记录、是否需要人工复核。相关综述也指出，LLM Agent 的规划、工具使用和反馈机制需要与评估和控制机制共同设计，才能进入可靠应用[25]。本文采用的思路是将Agent 决策限制在有限动作空间内。分类 Agent 只能输出五种任务代码的组合；图像总结 Agent 的结果只作为总结字段保存，不直接修改检测状态；项目报告 Agent 接收的输入由 BIZ 服务生成，输出也通过固定回调进入数据库。这样做牺牲了一部分开放性，但换来了工程系统所需的确定性和可追踪性。对于建筑幕墙评估这种专业场景，这种受控形式是 Agent 自主任务调度方法能够落地的基础。装 2.6 工程软件中的异步任务编排 长耗时任务编排是智能检测系统必须解决的问题。模型推理、外部 Agent 调用、对象上传和报告生成都可能持续数秒到数分钟，若采用同步 HTTP 请求等待全部结果，系统容易超时，用户体验也较差。成熟工程系统通常采用任务队列、Worker、回调和状态订 查询机制，将“启动任务”和“获得结果”解耦。本文系统采用类似思想，并针对 Agent 调用链路做了扩展。用户在前端启动检测后，BIZ 服务并不等待所有模型和Agent 返回结果，而是先创建检测任务、写入初始状态，再把任务投递到队列中。后续的分类、推理和总结由不同 Activity 服务分段处理，处理完成后再通过回调把结果写回业务服务。前端则通过 WebSocket 接收状态变化。这样设计以后，用户不需要停留在一次长时间的 HTTP 请求中等待，后台服务也可以把每一步的执行结果单独记录下来。异步任务编排对本系统比较关键，原因在于 Agent 调用和模型推理都存在一定不确定性。Agent 平台可能遇到网络波动、限流或输出格式异常；视觉模型可能因为图像格式、模型加载或运行环境问题失败；对象存储上传也可能受到网络状态影响。如果把这些步骤全部放在一次同步请求里，只要中间某一步失败，整个请求就会直接结束，用户也很难知道失败发生在哪个环节。采用任务状态机和回调机制后，系统可以把这些情况落到具体状态上，例如等待中、执行中、失败、成功等，前端也能据此展示进度和错误原因。从软件工程角度看，异步任务编排不只是为了避免接口超时，也是在表达本系统内部的业务依赖关系。检测主任务、检测子任务、总结任务和报告任务并不是平行关系，而是有先后条件。例如，分类失败时不应继续创建检测子任务；检测子任务全部完成后才适合进入总结阶段；如果某张图没有需要执行的检测子任务，也应直接进入相应的完成状态。把这些关系写入数据库状态和 Worker 编排逻辑后，系统才能按规则推进，而不是依赖某一次请求过程中的临时判断。LLM 信息整合与报告生成 大语言模型擅长自然语言生成，但在知识密集型任务中，输入信息的组织方式直接影响输出质量。检索增强生成通过外部知识库向模型提供上下文，缓解模型仅依赖参数知识导致的不可靠问题[30]。与此类似，工程报告生成也需要将外部数据组织成适合模型处理的上下文。如果输入过少，报告缺少依据；如果输入过多，模型容易遗漏重点或产生不稳定输出。建筑幕墙韧性评估报告属于典型的多源信息整合任务。其输入包括项目基础信息、订 图像组信息、原子检测指标、图像产物、区域级数据、用户复核评论和任务状态。直接把这些信息一次性提交给大语言模型并不合理，因为底层指标数量可能随图像数量和区域数量快速增长。本文提出 LLM 分层信息整合策略，将报告生成拆分为图像级和项 线 目级两个阶段。该策略并非简单压缩文本，而是根据工程评估逻辑先形成单图语义，再进行项目级融合。本章小结 本章围绕幕墙巡检、缺陷检测、Agent 工程范式、异步任务编排和 LLM 报告生成等内容进行了梳理。可以看到，现有研究已经为单类缺陷识别和通用 Agent 工具调用提供了较多方法基础，但与本文面向的项目级幕墙评估场景相比，仍有一些不完全匹配的地方。幕墙检测研究更多回答“某张图是否存在某类缺陷”，对图像资产如何按项目组织、检测任务如何根据图像内容调度、检测结果如何进入报告链路讨论较少；Agent 相关研究多从通用工具调用、代码执行和问答协作展开，落到建筑工程软件时，还需要进一步处理状态管理、失败恢复、权限边界和人工复核等问题；大语言模型报告生成则常关注文本生成效果，对输入信息分层、数据库状态和业务流程之间的配合说明不足。

因此，本文的切入点不是单独改进某一个检测模型，也不是把 Agent 作为额外的问答入口，而是把已有检测能力、Agent 任务调度和工程系统组织起来，形成从图像资产到韧性评估报告的完整链路。本文系统希望进一步回答三个更贴近业务流程的问题：单张图应该执行哪些检测，检测结果怎样形成可读的图像总结，多个图像结果和复核信息又怎样汇总为建筑级韧性评估报告。围绕这些问题，后续章节将从需求建模、方法设计和系统实现三个层面展开说明。装 订 线 系统需求分析 建筑幕墙韧性评估原本较多依赖人工整理和工程经验。将其转化为软件系统时，不能只从页面功能出发，还需要把现场工作中反

复出现的信息拆分为可保存、可流转和可追溯的业务对象。本文将系统中的核心对象划分为项目、图像组、图像、检测主任务、检测子任务、图像总结、人工复核和项目报告。项目用于保存建筑名称、位置、建设年份、既有问题和评估目标等背景信息；图像组对应建筑立面、楼层或采集区域；图像记录原图、尺寸、文件大小和上传状态；检测主任务以单张图像为单位，串联分类路由、五类检测子任务和图像总结；人工复核记录工程人员对检测结果的补充判断；项目报告则面向后续运维决策形成最终输出。装 从用户操作过程看，本系统也不是一组可以任意跳转的独立功能，而是围绕评估任务逐步推进的阶段化流程。用户通常需要先维护项目基础信息，再完成图像资产构建，随后启动Agent 智能检测；检测结果经过人工复核确认后，系统才进入评估报告生成阶段。各阶段之间存在明确的前后依赖，例如没有图像资产就无法启动检测，检测结果未 完成也不适合进入报告生成。基于上述对象模型和流程模型，本章进一步分析角色责任、典型用例、功能需求、非功能需求和相关约束条件。业务闭环中的角色与责任 线 如图 3.1 所示，系统中的主要角色可以分为使用者、业务服务、Agent 服务和检测模型服务。使用者关注的是项目创建、图像上传、结果查看、人工复核和报告确认；业务服务关注的是鉴权、数据持久化、状态推进和结果发布；Agent 服务关注的是语义路由、图像总结和项目报告生成；检测模型服务关注的是五类图像级原子检测能力的执行。角色划分的核心不是简单列出模块，而是明确“谁可以改变业务状态、谁只能提供判断结果”。这与第 2.5 节关于 Agent 工程化可控性问题的讨论直接对应。从业务对象角度看，图像不是孤立输入，而是项目资产的一部分；检测任务也不是一次函数调用，而是围绕图像产生的业务记录。这样建模可以支持两个重要能力：第一，用户可以在任何时间回到项目查看历史检测结果；第二，系统可以在报告阶段追溯每个结论来自哪张图像、哪个检测任务和哪个区域。图像路由任务使用者操作请求RPC 调用任务代码检测子任务项目创建、图像上传 结果查看与人工复核 状态展示 查询结果 指标与产物Reasoning检测模型服务执行原子检测总结与报告总结/报告任务 状态写入 文件与事件 数据与事件底座：MySQL 保存结构化状态 MinIO 保存图像产物 RocketMQ 发布项目事件Summary总结 Agent图像级总结与项目报告Core BIZ业务状态机任务创建与事务落库Core APIHTTP 接入 鉴权与 WebSocketClassification分类 Agent输出检测任务代码图像路由任务使用者操作请求RPC 调用任务代码检测子任务项目创建、图像上传 结果查看与人工复核 状态展示 查询结果 指标与产物Reasoning检测模型服务执行原子检测总结与报告总结/报告任务 状态写入 文件与事件 数据与事件底座：MySQL 保存结构化状态 MinIO 保存图像产物 RocketMQ 发布项目事件Summary总结 Agent图像级总结与项目报告Core BIZ业务状态机任务创建与事务落库Core APIHTTP 接入 鉴权与 WebSocketClassification分类 Agent输出检测任务代码控制原则：Agent 与模型服务只回传结构化结果，Core BIZ 统一执行业务状态推进 装 图 3.1 业务闭环中的角色与责任关系 从用户流程角度看，五个阶段并不是并列入口，而是逐步补齐评估材料的过程。项目基础信息提供报告背景，图像资产提供检测对象；检测完成以后，人工复核才有具体 订 依据，复核意见也会继续进入报告上下文。因此，系统需要设置项目进度控制，避免用户在数据尚未完整时提前进入后续阶段。与此同时，检测和报告生成都可能持续较长时间，若让用户一直等待同步请求返回，使用体验会明显下降，所以这类任务更适合以 异步方式执行。线 在角色边界上，各服务只负责自身范围内的判断或产物。分类 Agent 负责给出需要 执行的检测任务建议，数据库中的检测子任务仍由业务服务创建；Reasoning 服务负责执行模型并生成检测产物，检测主任务是否完成则应由业务服务根据子任务状态统一判断；Summary 服务负责生成图像总结和项目报告，但项目阶段推进仍需要经过业务状态机；前端可以展示进度和结果，却不应根据局部页面状态自行推断数据库中的最终状态。表 3.1 业务角色与责任边界 角色负责内容不应负责内容用户界面用户交互、状态展示、结果查看、复核输入直接修改任务状态或绕过后端判断项目是否完成Core APIHTTP 接入、登陆鉴权、WebSocket 推送复杂数据库事务和模型调度规则Core BIZ业务状态机、任务创建、事务落库、事件发布直接执行深度学习模型或自由生成报告文本Activity 服务Agent 调用、模型推理、能力执行和回调直接操作核心业务数据库Agent 平台语义路由、图像级总结、项目报告生成无约束地改变业务流程或访问内部状态角色负责内容不应负责内容用户界面用户交互、状态展示、结果查看、复核输入直接修改任务状态或绕过后端判断项目是否完成Core APIHTTP 接入、登陆鉴权、WebSocket 推送复杂数据库事务和模型调度规则Core BIZ业务状态机、任务创建、事务落库、事件发布直接执行深度学习模型或自由生成报告文本Activity 服务Agent 调用、模型推理、能力执行和回调直接操作核心业务数据库Agent 平台语义路由、图像级总结、项目报告生成无约束地改变业务流程或访问内部状态

((extend))((include))((include))((include))((include))((extend)) 表 3.1 表明，本文系统的复杂性并不只是功能数量多，而是每个功能都需要明确归属。尤其在 Agent 引入后，必须避免让 Agent 获得过大的业务权限。本系统采用“Agent输出建议，BIZ 执行业务”的方式，将核心状态推进集中在 Core BIZ 服务中，使智能能力和业务控制相互分离。这一设计虽然增加了 BIZ 的复杂度，但能够将 Agent 的语义能力和工程系统的确定性控制分离开来。典型用例分析 为了进一步明确系统边界，可以将用户使用过程抽象为图 3.2 所示的系统核心UML用例图。土木工程评估用户是系统的主要参与者，直接触发登录鉴权、项目维护、图像资产管理、Agent 智能检测、结果复核和报告生成等用例。其中，启动 Agent 智能检测 建筑幕墙韧性评估软件系统登录与身份鉴权创建与维护项目自动选择检测任务管理图像组与图像执行原子检测能力启动 Agent智能检测生成单图检测总结复核 检测结果整合项目报告上下文生成与查看评估报告失败重试与状态恢复建筑幕墙韧性评估软件系统登录与身份鉴权创建与维护项目自动选择检测任务管理图像组与图像执行原子检测能力启动 Agent智能检测生成单图检测总结复核 检测结果整合项目报告上下文生成与查看评估报告失败重试与状态恢复装 用例包含自动选择检测任务、执行原子检测能力和生成单图检测总结等内部用例；生成与查看评估报告用例包含整合项目报告上下文这一内部用例；失败重试与状态恢复作为扩展用例，用于处理检测或报告生成过程中的异常情况。((include))订 线 土木工程评估用户 图 3.2 系统核心 UML 用例图 这些用例基本对应用户从上传资料到获得评估结论的主要操作，也与第 1.2 节中提出的三个研究问题相呼应。图像资产构建主要处理数据如何进入系统、如何按项目和区域组织；Agent 智能检测关注每张图应该执行哪些检测任务，以及这些任务如何被调 度；人工复核和报告生成则把模型结果进一步转化为韧性评估表达。与普通后台管理系统相比，本系统的重点不只是增删改查页面，而是围绕长耗时任务组织业务流程。尤其是Agent 智能检测环节，背后会经过多个服务、多次回调和多种任务状态，因此需要通过状态机和异步编排来维持流程稳定。需要说明的是，UML 用例图更适合说明参与者能够使用哪些功能，以及这些功能属于系统边界内还是边界外。它并不直接表示用户必须按照怎样的顺序完成操作。实际使用时，本文系统仍按项目基础信息、图像资产构建、Agent 智能检测、人工复核确认和评估报告生成这一流程推进。每个阶段的进入条件和限制将在后续需求分析中继续说明。装 3.3 功能需求分析 功能需求需要服务于完整业务闭环，而不是只覆盖独立页面。根据前述用例，系统功能可以主要划分为项目与权限、图像资产、智能检测、人工复核与报告生成四组需求。其中，项目与图像资产需求为后续检测提供上下文和数据基础；智能检测需求承载 订 Agent 自主任务调度方法和原子检测能力；复核与报告需求则承载 LLM 分层信息整合 策略的最终输出。四组需求之间存在先后依赖，任何一组缺失都会导致“图像采集完成但评估报告难以形成”的现实问题继续存在。线 3.3.1 项目与权限需求 项目是所有业务数据

的归属边界。系统需要支持用户注册、登录、令牌刷新、头像管理、项目创建、项目删除、项目列表查询和项目详情维护。所有项目相关接口都需要进行用户身份校验，保证用户只能访问自己的项目数据。项目基础信息不仅用于页面展示，也会进入项目级报告上下文，因此建筑名称、位置、建设年份、既有问题和评估目标等字段需要能够被长期保存和读取。项目阶段推进也需要进行条件校验。系统不能允许用户在没有图像资产时进入智能检测阶段，也不能在检测任务未完成时进入人工复核阶段，更不能在报告未成功生成时将项目视为完成。阶段推进本质上是业务状态机的一部分，需要由后端统一判断，而不是由前端根据页面按钮状态自行决定。

3.3.2 图像资产需求 图像资产构建是后续Agent检测的基础。系统需要支持图像组管理、图像上传、图像移动、批量删除、批量移动、原图查看、上传状态展示和上传失败处理。图像组不仅是前端分类展示方式，也承担区域语义表达功能，例如“东立面”“裙楼区域”或“玻璃幕墙区域”等分组名称会影响项目报告中的区域化分析。由于图像数量可能较多，前端需要支持自适应卡片布局、分组折叠、批量选择和实时状态提示。后端需要通过对象存储保存原图和缩略图，通过数据库保存图像元数据、图像状态和分组关系，通过 WebSocket 推送图像状态变化。这样，图像不再是零散文件，而是具备项目归属、区域归属和处理状态的可追踪资产。

3.3.3 智能检测需求 智能检测阶段需要支持项目级启动检测、失败重试、状态查询和结果查询。启动检测时，系统应为所有已上传图像创建检测主任务。检测流程由 Worker 调度器自主推进，订用户不需要逐张图像操作。系统需要先调用分类 Agent 生成受控任务代码，再根据任务代码调度金属锈蚀、石材裂缝、石材污渍、玻璃平整度和玻璃爆裂等原子检测能力。智能检测需求还包括过程可见性。检测不是瞬时操作，用户需要知道当前图像处于分类、检测、总结、成功或失败中的哪一阶段。状态查询用于页面刷新后的状态恢

复，WebSocket 用于页面运行时的实时更新。失败重试需要清理旧任务、旧检测产物和缓存，再重新创建任务，避免旧数据影响新一次检测。

3.3.4 复核与报告需求 复核阶段需要支持读取和保存图像检测任务的复核信息，包括准确性判断和补充评论。人工复核不是对 Agent 和模型能力的否定，而是面向工程评估责任的必要补充。系统需要保留模型原始结果，同时记录人的判断，使项目报告能够体现“自动检测 + 人工确认”的协同过程。报告阶段需要支持报告状态查询、失败重试、报告结果展示和报告导出。报告生成本身属于后置任务，在用户推进项目阶段后自动触发。报告输入应包含项目基础信息、图像组信息、图像级总结、原子检测结果和人工复核意见，而不是只拼接原始底层指标。这样设计可以降低用户操作复杂度，使报告生成成为自然流程的一部分，也与第 2.7 节提出的 LLM 分层信息整合思想保持一致。

3.4 阶段化需求分析 为了使系统符合土木工程领域用户的真实工作习惯，本文将项目工作流划分为五个阶段：项目基础信息、图像资产构建、Agent 智能检测、人工复核确认和评估报告生成。每个阶段既有独立功能，又为后续阶段提供数据。项目基础信息阶段提供建筑名称、地点、年份和描述等上下文；图像资产阶段提供图像和分组；智能检测阶段产生结构化结果和图像总结；人工复核阶段补充人的判断；报告生成阶段形成最终输出。项目工作流主状态机项目基础信息Agent智能检测人工复核确认评估报告生成项目完成图像资产构建项目工作流主状态机项目基础信息Agent智能检测人工复核确认评估报告生成项目完成图像资产构建图像资产构建状态机提交 上传任务上传中上传成功上传失败重新提交上传任务图像资产构建状态机提交 上传任务上传中上传成功上传失败重新提交上传任务检测主任务状态机 主任务 Agent 创建 分类中单图总结中 主任务 失败 成功 重试子任务创建子任务检测中主任务失败检测主任务状态机 主任务 Agent 创建 分类中单图总结中 主任务 失败 成功 重试子任务创建子任务检测中主任务失败装 订 线 检测子任务状态机 子任务 模型 待执行 推理中产物上传子任务失败子任务成功推理产物完整性校验检测子任务状态机 子任务 模型 待执行 推理中产物上传子任务失败子任务成功推理产物完整性校验评估报告生成状态机报告待生成报告生成中报告生成成功报告生成失败失败重试评估报告生成状态机报告待生成报告生成中报告生成成功报告生成失败失败重试图 3.3 项目工作流与任务执行状态机总览 如图3.3 所示，阶段化需求不仅包含页面阶段之间的推进关系，还包含每个阶段内部的细粒度状态机。图像资产阶段需要处理上传成功和上传失败后的重新提交；Agent 智能检测阶段需要处理主任务的分类、子任务创建、子任务检测和图像级总结；检测子任务自身还包含模型执行、产物上传和完整性校验；报告生成阶段则需要区分生成中、生成成功和生成失败重试。只有将这些状态机统一纳入后端业务控制，系统才能在长耗时、跨服务和可失败的任务链路中保持一致。这种阶段化设计的意义在于把复杂评估任务拆分为为用户可理解的步骤。用户不需要一次性理解所有数据对象，而是在流程推进中逐步完成输入、确认和输出。对于系统而言，阶段边界也提供了校验条件。例如，没有上传图像时不能进入检测阶段；检测任务未全部成功时不能进入人工复核阶段；只有报告生成成功后，项目才能被视为最终完成。阶段化需求因此同时承担用户体验组织和业务状态约束两个作用。阶段化设计还使前端交互更清晰。图像资产阶段允许编辑、移动和删除图像；进入 装 检测阶段后，资产编辑能力应逐步收敛；进入复核阶段后，用户关注点从数据准备转为 结果确认；进入报告阶段后，用户关注点转为阅读和导出结论。不同阶段的按钮、权限和状态展示需要与业务含义一致。运行目标非功能需求工程约束真实场景试用稳定支撑可扩展性能接入一致性状态可信可观测性故障定位可部署性环境落地用户体验过程反馈进度展示、状态恢复结果解释与追溯容器编排、模型挂载服务依赖配置Request ID、任务日志 WebSocket 与 MQ 追踪回调落库、事务推进统一业务状态任务代码、注册表模型与前端可扩展运行目标非功能需求工程约束真实场景试用稳定支撑可扩展性能接入一致性状态可信可观测性故障定位可部署性环境落地用户体验过程反馈进度展示、状态恢复结果解释与追溯容器编排、模型挂载服务依赖配置Request ID、任务日志 WebSocket 与 MQ 追踪回调落库、事务推进统一业务状态任务代码、注册表模型与前端可扩展订 3.5 非功能需求分析 线图 3.4 非功能需求与工程约束关系 除业务流程能够走通之外，系统还需要考虑一些更偏工程运行层面的要求，包括可扩展性、一致性、可观测性、可部署性和用户体验等。这里的非功能需求并不是附加项，而是和第 2.5 节中讨论的 Agent 工程化问题直接相关。原因在于，Agent 输出存在一定不确定性，检测和报告生成往往不是瞬时完成，图像原图与检测产物也会占用较多存储空间，同时多个服务之间还存在回调和状态同步。如果只关注页面功能而忽略非功能需求，系统即使能够完成一次演示，也难以稳定支撑真实场景试用。为便于理解，图 3.4 将非功能需求概括为围绕真实场景试用形成的五类工程约束，分别对应能力扩展、状态可信、故障定位、环境落地和过程反馈等问题。下面对五类非功能需求进行详细说明。

可扩展性 建筑幕墙检测能力具有持续扩展需求。当前系统包含五类原检测子任务，但未来可能新增连接件松动、密封胶老化、幕墙变形、热工异常等检测能力。因此系统不能将 装 检测流程硬编码在某个单体函数中，而应通过任务代码、统一接口和注册表机制组织能力。Agent 输出也应限制在系统已注册能力范围内，以便新增能力时只需补充任务代码、模型服务、数据库字段和前端展示逻辑。可扩展性是系统持续接入新检测能力并适应真实工程场景演进的重要基础。订 一致性检测任务涉及多服务协作和异步回调。一旦状态更新顺序混乱，前端进度和数据 线 库结果就可能不一致。本文采用“先建记录、后调服务、回调落库、事务推进”的原则。即在调用外部 Activity 服务前，BIZ 服务先创建任务记录并设置状态；Activity 服务只 通过回调上报结果；BIZ 在事务中更新结果并判断是否推进主任务。该设计保证了系统状态由业务中心统

一掌握。一致性是多服务异步协作场景下业务状态可信推进的重要前提。可观测性 Agent 和模型调用存在外部依赖，失败原因可能来自网络、模型、对象存储、数据库或第三方 Agent 平台。因此系统需要完整日志和状态追踪。每个请求需要携带 Request ID 并在 HTTP、RPC 和回调链路中透传；每个任务需要记录 UUID、状态、开始时间、完成时间和运行耗时；每次 WebSocket 广播和 RocketMQ 事件也需要有日志。可观测性是长链路系统稳定运行的重要保障。可部署性 毕业设计不是脚本演示，系统需要能够在服务器上通过容器方式启动。由于系统由多个服务组成，部署需要处理镜像构建、服务依赖、环境变量、消息队列 Topic 初始化、模型文件挂载和网络端口映射。尤其是 Reasoning 服务同时依赖 Golang 运行时、Python 环境和模型权重，部署方案需要兼顾镜像体积和模型文件管理。本文采用服务镜像加模型挂载的方式，使模型文件可以独立维护，避免镜像过大。用户体验 用户体验需求主要体现在等待过程和结果解释上。检测任务耗时不可避免，因此前端需要以进度条、流程节点和图像蒙层展示状态。结果解释方面，用户需要同时看到装原图、检测产物、结构化指标和 Agent 总结。对于区域类结果，系统需要支持在总体数据和各区域数据之间切换，使用户能够从总体结论追踪到具体区域。关键挑战 订 根据上述需求分析中可以看出，本文系统不能简单理解为普通 CRUD 页面组合。它更接近一个围绕图像资产、异步任务和报告生成组织起来的业务流程系统，因此实现难点主要集中在以下几个方面。线 第一点是业务链路较长。用户上传图像后，系统还需要完成图像归档、分类路由、并发检测、图像总结、人工复核和项目报告生成等步骤。任一环节失败或状态没有及时更新，都会影响后续阶段的推进。第二个难点是服务协作较多。系统中前端负责交互，API 负责接入和鉴权，BIZ 负责业务状态，三个 Activity 服务分别处理分类、推理和总结，数据库、对象存储、缓存和消息队列又承担不同的数据支撑职责。各模块边界不同，但需要在一次检测流程中连续配合。第三，状态同步要求较高。检测状态既要持久化到数据库，又要通过 RocketMQ 触发事件，并通过 WebSocket 推送到前端。用户刷新页面后仍应看到正确进度，因此系统不能只依赖前端内存状态，而要以后端任务状态作为最终依据。第四，结果结构并不统一。五类检测任务的输出字段、区域数量和产物图像数量都不完全相同，有的输出掩码和面积占比，有的输出分类置信度或校正图。报告生成阶段还需要把这些差异化结果重新整理成统一上下文，否则后续 LLM 很难稳定生成项目级结论。第五，报告输入组织复杂。项目级报告并不是简单拼接所有检测结果，而是需要把原子指标、图像总结、图像组信息和人工复核意见组织为模型可理解的上下文。上下文过短会导致报告缺少依据，上下文过长又可能造成重点丢失。第六，跨语言能力集成复杂。视觉检测模型主要运行在 Python 环境中，而后端业务服务采用 Golang 实现。系统需要让 Python 专注模型推理，让 Golang 负责下载、上传、回调和状态管理。跨语言边界既要简洁，又要保证错误能够被 Golang 层捕获并反馈给上游状态机。装 因此，本文的工作量主要体现在跨模块的工程链路设计和 Agent 能力嵌入。系统需要同时满足“智能性”和“工程稳定性”：Agent 负责语义判断和信息整合，工程系统负责协议、状态、存储和容错。二者缺一不可。上述挑战也说明，后续技术方案不能只订 讨论 Agent 提示词，也不能只讨论工程接口，而必须把 Agent 方法与工程状态机共同设计。数据流分析 线 数据流分析用于说明系统中的数据如何从用户输入转化为可复核的检测结果和可阅读的评估报告。与普通信息管理系统不同，本文系统同时包含前端交互数据、业务状态数据、图像文件数据、模型推理结果、Agent 语义输出和实时事件数据。为了体现这些数据之间的关系，图3.5 从 FE 前端层、API 接入层、BIZ 业务层、Activity 基础能力服务和基础设施五个层次刻画系统主要数据流。如图3.5 所示，系统数据流首先从用户在前端发起项目操作和图像上传开始。前端通过 Core API 获取业务数据和上传授权，并将大体量图像文件直接写入 MinIO；Core API 负责鉴权、HTTP 接入和 WebSocket 连接维护，复杂业务状态则下沉到 Core BIZ。Core BIZ 在 MySQL 中保存项目、图像、检测主任务、检测子任务、复核记录和报告状态，在 Redis 中配合完成令牌状态和临时控制，并根据业务阶段创建分类、检测、总结和报告任务。

Classification 分类 Agent 输出检测任务代码图像路由由任务 FE 前端操作请求 Core API RPC 调用 Core BIZ 任务代码检测子任务 项目创建、图像上传 HTTP 接入 业务状态机结果查看与人工复核 状态展示 鉴权与 WebSocket 查询结果 任务创建与事务落库 指标与产物 Reasoning 检测模型服务执行原子检测总结与报告实时刷新状态广播 WebSocket 实时状态通道总结/报告任务状态读写缓存读写文件读写 Summary 总结 Agent 图像级总结与项目报告 MySQL 项目 / 图像 / 任务指标 / 复核 / 报告 Redis 令牌状态 缓存与临时控制 MinIO 原图 / 缩略图 / 检测产物结构化多源分析数据 RocketMQ 项目事件 Topic

Classification 分类 Agent 输出检测任务代码图像路由由任务 FE 前端操作请求 Core API RPC 调用 Core BIZ 任务代码检测子任务 项目创建、图像上传 HTTP 接入 业务状态机结果查看与人工复核 状态展示 鉴权与 WebSocket 查询结果 任务创建与事务落库 指标与产物 Reasoning 检测模型服务执行原子检测总结与报告实时刷新状态广播 WebSocket 实时状态通道总结/报告任务状态读写缓存读写文件读写 Summary 总结 Agent 图像级总结与项目报告 MySQL 项目 / 图像 / 任务指标 / 复核 / 报告 Redis 令牌状态 缓存与临时控制 MinIO 原图 / 缩略图 / 检测产物结构化多源分析数据 RocketMQ 项目事件 Topic

HTTP / RPC 控制流 回调控制流 对象存储文件流 MQ 事件流 WebSocket 事件流 状态与缓存读写 装 图 3.5 系统核心数据流与事件流图示 Activity 服务与 BIZ 之间形成受控的异步执行关系。Classification 服务只返回图像对应的任务代码，Reasoning 服务只返回原子检测指标和产物清单，Summary 服务只返回结构化 JSON 总结或项目报告；所有结果都通过回调进入 BIZ，由 BIZ 在事务中解析、落库并推进状态。图像原图、缩略图、检测产物和项目报告源文件等文件数据统一存储在 MinIO，模型服务在执行时读取受控授权下的原图，并在产物上传完整后再回调线 结构化结果。数据流中还存在一条独立的实时事件链路。BIZ 在状态变化后向 RocketMQ 发布项目事件，Core API 消费事件后通过 WebSocket 广播给前端，使用户能够实时看到图像上传、检测进度和报告状态变化。页面刷新或 WebSocket 断连后，前端仍然可以重新调用列表和详情接口，从 MySQL 中恢复完整状态。因此，RocketMQ 和 WebSocket 主要承担增量通知职责，数据库中的结构化状态仍然是系统最终可信来源。从该数据流可以看出，系统中并不是只有数据库记录在流动，还同时存在文件数据、结构化数据和语义数据三类内容。原图、缩略图、检测产物和报告源文件属于文件数据，主要存储在 MinIO 中，后续复核和结果展示都需要依赖这些文件。项目、图像、任务状态、指标字段、区域 JSON 和复核意见属于结构化数据，保存在 MySQL 中，用来支撑页面查询、状态追踪和流程推进。图像级总结和项目评估报告则属于语义数据，由 Agent 生成后再经过 BIZ 校验并写入数据库。三类数据的作用并不相同：文件数据保留视觉证据，结构化数据记录过程和指标，语义数据负责把底层结果转化为面向运维决策的表达。约束条件 系统设计还需要满足若干约束。第一，模型服务和业务服务应保持边界清晰。Python 模型不应直接操作数据库和对象存储业务 Key，否则后续新增模型会带来大量耦合。第二，Agent 输出必须可校验。分类结果需要解析为任务代码，报告结果需要通过固定接口回调，不能让自由文本直接控制状态。第三，检测任务需要支持失败状态和重试扩展。第四，前端状态需要能从服务端恢复，不能只依赖 WebSocket 增量消息，因为用户可能刷新页面或断线重连。第五，报告生成必须以系统已有数据为依据，不能让模型脱离项目上下文自由编造工程背景和检测结论。本章小结 本章在摘要、绪论和相关研究基础之上，将“项目级图像资产管理、Agent 自主任务调度、LLM 分层信息整合”三条主线转化为系统需求。通过业务对象和用户流程建模，本章明确了系统角色责任、典型用例、功能

需求、非功能需求、数据流和约束条件。需求分析表明，本文系统的核心难点不只是提供图像上传和结果展示功能，而是需要线在多阶段业务流程中实现Agent 自主任务调度方法、原子检测、人工复核和报告生成的闭环协同。表 3.2 需求与设计目标映射 需求类型 现实问题 设计目标 图像资产管理 图像数量多、组织混乱、难以复核 建立项目、图像组和图像三级资产结构 检测任务调度 不同材质需要不同检测能力 使用分类 Agent 输出受控任务代码 Agent 可控性 模型输出存在不确定性 通过工具集合、结构化输出和 BIZ 状态机限制业务后果 长任务执行 检测和总结耗时较长 使用 Worker、异步启动和回调推进状态 多源结果整合 底层指标难以直接形成报告 采用图像级总结到项目报告的 LLM 分层信息整合策略 用户感知 长时间执行容易造成等待焦虑 使用 WebSocket、进度条和流程节点实时展示 可追踪性 多服务链路失败难以定位 使用 UUID、状态机、日志和数据库记录追踪全流程 如表3.2 所示，本章并不是把系统功能简单列成清单，而是把前文提到的现实问题逐一一对应到后续设计中。项目级管理、任务调度、长任务执行、多源结果整合、用户感知和可追踪性，都会在系统中落到具体的工程或 Agent 设计点上。因此，第 4 章将继续讨论面向建筑幕墙多材质图像的 Agent 自主任务调度方法和 LLM 分层信息整合策略，第 5 章则说明这些需求如何进一步落实到服务架构、协议、数据库状态机、对象存储和前端可视化实现中。装 订 线 技术方案设计 本章重点讨论 Agent 方法层设计：如何在受控边界内使用 Agent 完成检测任务规划，如何将既有原子检测能力工具化，如何把多张图像、多类缺陷、多组区域和人工复核意见逐层整合为项目级评估报告。整体上，本系统采用“工程状态机约束 Agent，Agent 补足语义决策能力”的技术方案，使智能能力服务于确定性业务流程。

关键词 为了避免后续表述产生歧义，本章首先对使用的关键词进行说明。如表4.1 所示这里的术语并非孤立概念，而是对应系统中真实存在的设计对象、服务节点和数据结 构。表 4.1 技术方案设计关键词 术语 含义 Agent 自主任务调度方法 由分类Agent 根据幕墙图像中的材质和缺陷线索，输出受控检测任务代码列表，再由业务系统创建和调度后续检测子任务的方法 订 LLM 分层信息整合策略 将原子检测结果、图像级总结、人工复核和项目上下文分层组织，逐级生成图像级总结和项目级评估报告的策略 原子检测能力 面向某一类幕墙材质或缺陷的最小可调度检测能力，包括金属锈蚀、石材裂缝、石材污渍、玻璃平整度和玻璃爆裂五类 任务代码 用于连接Agent 决策与工程调度的有限动作集合，即 corrosion、crack、stain、flatness 和 spalling 受控 Agent 只能在固定业务节点被调用、只能输出固定结构、不能直接写库或推进业务状 态的 Agent 检测主任务 以单张项目图像为单位创建的检测业务记录，串联分类、子检测、图像级总结 和最终状态 检测子任务 由某个原子检测能力执行的具体任务，保存该类检测结果、区域指标、产物信息和执行状态 图像级总结 针对单张图像的结构化检测结果生成的专业文字总结，用于前端展示和项目报告上游输入 项目级评估报告 综合项目基础信息、图像组、图像检测结果、图像总结和人工复核意见形成的建筑幕墙韧性评估结论 人机协同复核 用户对 Agent 和模型结果进行准确性判断与补充评论，并将复核信息纳入项目级报告上下文的机制 总体方法框架 本文的总体方法框架面向第3章提出的系统需求，重点回答两个问题：第一，如何把项目级图像资产、多材质幕墙图像和用户真实评估目标组织为可执行的检测任务；第二，如何把原子检测事实、图像语义、人工复核和项目上下文整合为建筑级韧性评估报告。与单纯的检测模型调用不同，本文方法不把智能能力集中在某一个端到端模型中，框架价值与意义从单图检测事实到项目级韧性评估闭环，形成可追溯、可复核、可扩展、可部署、可恢复的 Agent 工程方案框架价值与意义从单图检测事实到项目级韧性评估闭环，形成可追溯、可复核、可扩展、可部署、可恢复的 Agent 工程方案而是将系统拆分为“工程约束、数据证据、原子检测、Agent 调度、LLM 整合、质量治理和目标输出”等多个层次，使每一层都有明确职责。业务需求与问题输入项目级图像资产目标输出层 单图检测总结 项目评估报告区域风险等级专业输出维护改进建议质量治理与反馈提示词规则字段白名单图像组/ 立面区域LLM 分层信息整合层原子事实层指标与产物结构化数据输入图像语义层图像级总结项目报告层多源融合输出合法性校验 输出约束 JSON 对象多材质幕墙图像输出压缩JSON 校验结构化输入报告结构层固定字段一致性约束任务代码校验非法值拦截 批量上传与状态追踪 需求牵引单图单缺陷检测难以形成项目结论Agent 自主任务调度层幕墙场景识别枚举归一去重排序 材质语义识别 任务代码生成 非幕墙过滤子任务创建策略 失败回调重试入口工具候选产物完整性校验 任务治理 Uploaded=true报告一致性校验图像组与图像数人工汇总成本高报告形成滞后原子检测工具层金属锈蚀模型注册 石材裂缝 石材污渍 玻璃平整度 产物校验 人机协同复核 准确/ 不准确/ 评论运行目录隔离用户真实目标建筑级韧性评估模型懒加载工具事实玻璃爆裂动态产物清单WebSocket 反馈进度与状态可复核证据维护优先级项目资产数据证据与上下文层项目信息区域指标图像组图像资产检测产物报告源数据复核反馈失败重试闭环检测与报告多区域风险分级处置人工复核状态约束工程约束与运行治理层可扩展接入新增模型能力报告成果归档交付 状态机 协议回调 事务落库 对象授权 失败重试 并发超时 日志追踪事件广播CronJob 清理日志观测问题定位工程底座基础支撑

MySQLRedisMinIORocketMQgRPCDockerLogConfig业务需求与问题输入项目级图像资产目标输出层 单图检测总结 项目评估报告区域风险等级专业输出维护改进建议质量治理与反馈提示词规则字段白名单图像组/ 立面区域LLM 分层信息整合层原子事实层指标与产物结构化数据输入图像语义层图像级总结项目报告层多源融合输出合法性校验 输出约束 JSON 对象多材质幕墙图像输出压缩JSON 校验结构化输入报告结构层固定字段一致性约束任务代码校验非法值拦截 批量上传与状态追踪 需求牵引单图单缺陷检测难以形成项目结论Agent 自主任务调度层幕墙场景识别枚举归一去重排序 材质语义识别 任务代码生成 非幕墙过滤子任务创建策略 失败回调重试入口工具候选产物完整性校验 任务治理 Uploaded=true报告一致性校验图像组与图像数人工汇总成本高报告形成滞后原子检测工具层金属锈蚀模型注册 石材裂缝 石材污渍 玻璃平整度 产物校验 人机协同复核 准确/ 不准确/ 评论运行目录隔离用户真实目标建筑级韧性评估模型懒加载工具事实玻璃爆裂动态产物清单WebSocket 反馈进度与状态可复核证据维护优先级项目资产数据证据与上下文层项目信息区域指标图像组图像资产检测产物报告源数据复核反馈失败重试闭环检测与报告多区域风险分级处置人工复核状态约束工程约束与运行治理层可扩展接入新增模型能力报告成果归档交付 状态机 协议回调 事务落库 对象授权 失败重试 并发超时 日志追踪事件广播CronJob 清理日志观测问题定位工程底座基础支撑MySQLRedisMinIORocketMQgRPCDockerLogConfig装

订 线图 4.1 Agent 驱动的幕墙缺陷检测与韧性评估方法框架 图4.1 是对本文方法框架的整体组织。可以先从左右两侧理解这张图：左侧对应业务需求与问题输入，包括项目级图像资产、图像组、立面区域、多材质幕墙图像、批量上传和状态追踪等对象，这些内容来自第 1 章中提到的单图检测割裂、人工汇总成本高和报告形成滞后等问题；右侧对应质量治理与反馈，用来约束 Agent 输出、任务代码、检测产物、报告一致性、人工复核、WebSocket 状态反馈、失败重试和日志观测等环节。中间部分则表示系统内部的处理主线，即从工程支撑开始，逐步形成检测事实、语义整合结果和用户最终看到的评估结论。沿图中的箭头看，业务需求先进入 Agent 自主任务调度层，系统据此判断每张图像需要执行哪些检测；项目资产进入数据证据与上下文层后，被整理为后续 Agent 和报告 生成可以使用的结构化上下文。底部的基础支撑和工程治理负责状态、协议、对象授权、事务落库和失败恢复。再往上，原子检测工具层生成可以追溯到图像和任务的检测事实，LLM 分

层信息整合层在这些结构化输入基础上生成图像级总结和项目报告。目标输出层面面向用户展示单图检测总结、项目评估报告、区域风险等级和维护改进建议。这样安排的目的，是把不确定的语义判断放在受控 Agent 节点中，把状态推进、数据管理和质量校验交给工程系统处理。框架底部是基础支撑层，主要由 MySQL、Redis、MinIO、RocketMQ、gRPC、Docker、日志和配置等组成。这些设施本身并不产生智能结论，但它们决定了系统能否稳定保存状态、存储图像产物、传递事件、完成服务间通信，并支持容器化部署。其上一层是工程约束与运行治理，包括状态机、协议回调、事务落库、对象授权、事件广播、失败重试、并发超时、日志追踪和CronJob 清理等机制。本文将这些机制单独放在一层，是因为 Agent 不能直接写库、直接推进项目状态，也不能绕过产物完整性校验；智能判断要进入业务流程，必须先经过这些工程边界。订 数据证据与上下文层保存系统决策和报告生成所需的事实依据，例如项目信息、图像组、图像资产、检测产物、区域指标、人工复核和报告源数据。原本分散在单张图像、多个任务和多类缺陷中的信息，在这一层被组织为可以查询、复核和追溯的上下文。原子检测工具层则负责封装团队已有的检测模型和图像处理脚本，将金属锈蚀、石材裂 线 缝、石材污渍、玻璃平整度和玻璃爆裂五类能力接入统一任务调度框架。为了降低不同模型之间的耦合，系统还需要配合模型注册、运行目录隔离、模型懒加载和动态产物清单等设计。Agent 自主任务调度层处理的是“哪些图像需要执行哪些检测”。它不直接给出缺陷结论，而是根据幕墙场景识别和材质语义识别结果生成有限任务代码，再通过枚举归一、去重排序、子任务创建策略和失败回调重试入口，将语义判断转换为可执行的工程动作。LLM 分层信息整合层处理的是“检测结果如何进入项目级认知”。该层并不把所有底层指标一次性丢给模型，而是把原子事实层的指标与产物、图像语义层的图像级总结、项目报告层的多源融合和报告结构层的固定字段分开处理，并通过结构化数据输入、输出压缩 JSON 校验和一致性约束降低输出漂移与事实编造风险。目标输出层最终面向用户提供单图检测总结、项目评估报告、区域风险等级和维护改进建议。右侧质量治理与反馈贯穿整个框架，主要用于检查提示词规则、字段白名单、JSON 对象合法性、任务代码合法性、产物完整性、报告一致性、人工复核、WebSocket反馈、失败重试、扩展接入和日志观测等内容。由此可见，图4.1 强调的是宏观方法组织，而不是某一次请求的技术细节。系统既不是让一个 Agent 从图像直接生成最终报告，也不是让所有模型无差别执行后再堆叠结果，而是把“证据生成”和“语义整合”分开，把“Agent 判断”和“业务状态推进”分开。分类 Agent 负责把图像语义映射为有限任务代码，Reasoning 服务负责生成可追溯的检测事实，图像总结 Agent 负责把底层指标转换为单图语义，项目报告 Agent 再结合项目背景、图像组和复核意见形成建筑级结论。该方法与第 2.5 节讨论的 Agent 工程化可控性问题相对应。Agent 具有语义理解和装 自然语言组织优势，但直接赋予 Agent 自由行动权限会带来输出不稳定、工具选择错误和状态误推进等风险。因此，本文采用“动作空间有限、输出结构固定、业务状态外置”的设计。Agent 的输出必须经过协议解析和业务校验，所有数据库写入和项目状态推进 订 均由Core BIZ 执行。总体而言，该框架的核心在于：Agent 负责受控语义决策和分层信息整合，工程系统负责边界、状态、权限、数据、质量治理和异常恢复。Agent 在系统中的角色边界 线 在本系统中，Agent 不是一个开放式对话入口，也不是可以自主操作数据库的通用代理。本系统将 Agent 限定为业务流程中的三个受控节点：分类 Agent、图像总结 Agent和项目报告Agent。三者对应从“任务调度”到“局部解释”再到“整体评估”的递进关系。表 4.2 系统 Agent 节点设计 Agent 节点 输入 输出 业务作用 分类 Agent 待检测幕墙图像 检测任务代码列表 判断该图像应执行哪些原子检测能力 图像总结 Agent 单张图像的结构化检测结果 固定字段的单图检测总结 JSON 将底层指标压缩为可读的

图像级语义 项目报告 Agent 项目基础信息、图像组、图像 固定字段的项目评估报告 JSON 形成建筑幕墙韧性评估结论总结、检测结果和复核意见 论和维护建议 表4.2 表明，本文所说的 Agent 不是“大模型能力”的笼统称呼，而是具有固定业务入口、固定输入来源和固定输出契约的智能节点。分类 Agent 不输出缺陷结论，只输出 任务代码；图像总结 Agent 不重新判断图像，只总结已完成检测结果；项目报告 Agent不读取数据库，只依据系统组织好的项目上下文生成报告。这样的角色边界使 Agent 能够参与业务流程，但不会成为业务状态的所有者。角色边界还体现在权限控制上。Agent 不能直接调用原子检测模型，不能直接访问 MySQL，不能直接写入 MinIO，也不能直接推进项目阶段。所有 Agent 输出都必须先返回 Activity 服务，再由 Activity 服务通过 gRPC 回调 Core BIZ。从设计原则上看，本文采用的是“智能节点最小授权”思想。分类 Agent 只获得图像，不获得数据库写权限；总结 Agent 只获得结构化 JSON，不获得项目状态推进权限；项目报告 Agent 只获得系统整理后的多源检测结构化数据，不直接访问数据库和对 象存储目录。这样可以降低 Agent 输出异常时的影响范围。即便某一次 Agent 输出不合 装 规，最多导致当前节点失败，不会破坏项目主数据，也不会造成跨项目数据污染。技术路线取舍分析 本小节解释为什么系统采用“受控Agent + 原子检测能力+ 分层整合”的路线，而 订 不是采用固定检测流程、端到端大模型或开放式 Agent 等更直观的方案。方案评价维度 线 表 4.3 技术方案评价维度 评价维度 设计含义 本文关注点 任务选择能力 系统能否根据图像材质和缺陷线索自动选择 检测能力 证据可追溯性 用户能否追溯结论来自哪张图像、哪个检测 任务和哪些产物 避免人工逐图选择模型，也避免所有图像无差别执行全量检测 原图、掩码、标注图、区域指标和人工复核都应被保留 状态可控性 长链路任务失败时能否定位失败节点并恢复 分类、子检测、产物上传、总结和报告生成 必须有独立状态 报告可组织性 系统能否把大量单图结果转化为建筑级评估 结论 工程可扩展性 后续能否新增检测能力、展示字段或报告字 段 用户可信性 输出是否符合工程用户对安全、责任和复核 的要求 需要支持图像组、区域差异、风险等级和维护建议 任务代码、模型注册、结构化结果和前端展示应保持松耦合 Agent 结论不能覆盖事实，人工复核必须进入最终报告上下文 建筑幕墙韧性评估不是单纯的图像分类任务。系统既要能处理多张、多组、多材质图像，又要让土木工程用户能够追溯缺陷证据、查看中间产物、补充人工意见并获得项目级报告。因此，技术方案不能只用模型精度或开发难度评价，还需要从业务角度建立 评价维度。如表4.3 所示，本文关注的是一个工程系统的综合可用性，而不是单一模型调用能力。若方案只强调端到端生成文本，可能在演示时看起来简单，但无法满足证据追踪和人工复核要求；若方案只强调数据库和流程控制，则又难以解决材质识别、任务选择和报告语言组织问题。本文技术路线正是在这两类需求之间寻找平衡。技术路线比较 本文在方案设计阶段形成了六类可选技术路线。它们在特定场景下都具有一定合理性，但适用范围和风险不同。为突出选择过程，本文从“如何选择检测任务”和“如何生成项目报告”两个关键问题出发进行比较。装 表 4.4 不同技术路线适用性比较 方案 优势 主要问题 本文取舍 固定全量检测流程 实现简单，所有图像执行相同检测，状态容易控制 订 人工选择检测能力 人的判断准确性较高，便于结合现 场经验 规则引擎路由 可解释性强，可根据文件夹、图像 组或用户选项触发检测 端到端大模型 链路短，理论上可以从图像或文本直接生成结论 线 开放式 Agent 自主性强，理论上可自行规划和调 用工具 本系统方案 任务选择具备语义能力，像素级事 实可追溯，报告生成分层可控，状态由工程系统统一推进 大量无关模型调用会浪费资源；非适用材质可能产生干扰性结果；用户难以理解为什么某类图像出现无关检测项 图像数量大时成本高、效率低；用户需要理解五类模型能力；难以支撑批量项目 规则依赖人工配置；难以处理混合材质图像；无法利用图像语义线索缺少稳定区域指标和图像产物；上下文容易超限；失败节点不可定位；结论证据链薄弱

工具边界不清晰，容易越权调用、状态误推进和输出不可控；不适合直接接管工程状态 工程链路更复杂，需要协议、状态机、Worker、回调和产物管理协同实现 不采用 作为复核补充 不作为主方案不作为主链路 不采用开放式权限采用 如表4.4所示，本文没有选择端到端大模型，核心原因不是大模型能力不足，而是建筑幕墙评估需要“可追溯、可复核、可恢复”。土木工程用户不仅需要一段自然语言，还需要知道某张图片是否检测过、执行了哪些检测、检测区域在哪里、对应产物能否查看、人工复核如何影响最终报告。如果让大模型直接从图像生成报告，系统很难解释某一结论来自哪张图片、哪类缺陷和哪项指标，也难以在失败后定位是图像理解、缺陷识别还是报告生成环节出错。固定全量检测流程也不适合本文场景。建筑幕墙图像可能是玻璃、石材、金属或多 材质组合，若所有图像都执行五类检测，一方面增加推理成本，另一方面会产生大量对用户没有意义的“未检测到某类缺陷”的结论。对于工程用户而言，关键问题不是系统是否调用了足够多模型，而是系统是否在合适图像上调用了合适模型，并能解释对应检测结果。

规则引擎方案在传统业务系统中常见，例如根据图像组名称、用户选择或固定项目模板决定检测类型。但在无人机巡检或批量图像上传场景中，图像组名称并不能可靠表示图像材质，同一图像中也可能同时包含玻璃和金属构件。纯规则方案难以处理视觉语义不确定性。本文因此引入分类 Agent，而不是完全依赖人工配置。技术路线组合逻辑 装 本系统最终采用组合式技术路线，其组合逻辑可以概括为三点。第一，视觉语义判断交给 Agent，但任务动作集合必须固定。Agent 可以判断图像中是否存在金属、石材或玻璃，但只能输出系统预定义的五类任务代码。第二，像素级事实交给原子检测能力。锈蚀区域、裂缝区域、污渍占比、平整度指标和爆裂置信度都来自专门模型或图像 订 处理流程，而不是由LLM 直接编写。第三，工程结论交给LLM 分层整合。LLM 不负责发现像素级缺陷，而负责把结构化结果组织为用户可读、专业一致的评估语言。这种组合使系统在智能性和可控性之间取得平衡。Agent 自主任务调度方法解决 线 “做哪些检测”的语义选择问题；原子检测能力工具化解决“怎样获得像素级和区域级事实”的问题；LLM 分层信息整合策略解决“如何从局部事实形成工程评估结论”的问题；工程状态机解决“如何保证流程可追踪和可恢复”的问题。也就是说，Agent 并不是替代工程系统，而是补足传统工程系统在语义判断和信息整合上的不足。该方案的代价是工程复杂度提高。系统需要维护任务代码、检测任务、子任务、总结任务、报告任务、对象存储产物、人工复核和事件通知等多个对象。但这些复杂度是为了解决真实业务痛点而引入的必要复杂度，而不是形式上的服务拆分。通过将复杂度显式建模，系统才能支撑项目级韧性评估，而不是只完成单次图像推理。工业界实践启示 从工业界智能体系统实践看，可靠的 Agent 应用通常不会让大模型直接掌控全部业务状态，而是将大模型放在受限规划、信息抽取、内容生成或工具选择节点中。任务 执行、权限控制、状态管理和数据持久化仍由传统工程系统完成。本文方案借鉴这一思路，将 Agent 定位为“受控决策节点”和“语义整合节点”，而不是完整业务流程的所有者。这种设计尤其适合建筑幕墙韧性评估场景。幕墙检测结果具有安全属性，报告结论可能影响后续维护决策和工程责任，系统不能只追求自然语言输出的流畅性，还必须保留证据链、状态链和人工复核链。因此，本文不是简单地把大模型接入前端，而是围绕Agent 输出契约、任务代码枚举、项目状态机、对象存储产物和人工复核机制构建完整方案。

Agent 自主任务调度方法 图像语义理解是否为建筑幕墙 幕墙材质识别 明显缺陷线索识别Agent 负责语义判断候选任务生成金属 $\rightarrow$ corrosion 石材 / 混凝土 $\rightarrow$ crack + stain 玻璃 $\rightarrow$ flatness + spalling系统负责输出约束受控任务集合 $R(I)$ $\{corrosion, crack, stain, flatness, spalling\}$ 的子集Core BIZ 创建检测子任务Reasoning 执行原子检测BIZ 负责状态推进任务代码治理非幕墙图像过滤 非法任务代码拦截 去重 固定顺序排序 空集合允许输入图像 I 图像语义理解是否为建筑幕墙 幕墙材质识别 明显缺陷线索识别Agent 负责语义判断候选任务生成金属 $\rightarrow$ corrosion 石材 / 混凝土 $\rightarrow$ crack + stain 玻璃 $\rightarrow$ flatness + spalling系统负责输出约束受控任务集合 $R(I)$ $\{corrosion, crack, stain, flatness, spalling\}$ 的子集Core BIZ 创建检测子任务Reasoning 执行原子检测BIZ 负责状态推进任务代码治理非幕墙图像过滤 非法任务代码拦截 去重 固定顺序排序 空集合允许输入图像 I 装 订 线图 4.2 Agent 自主任务调度方法流程 Agent 自主任务调度方法是本文核心方法之一。其目标是将“用户或工程师人工判断应该跑什么模型”的过程转化为受控的自动任务规划过程。具体而言，如图 4.2 所示，分类Agent 接收单张图片，判断图像是否属于建筑幕墙场景，并识别其中可能存在的金属、石材/混凝土和玻璃等幕墙材质，再输出对应的原子检测任务代码。任务调度问题建模 从形式上看，任务调度可以抽象为一个受限多标签选择问题。输入为项目中的单张图片，输出为任务代码集合： $R(I) \subseteq \{corrosion, crack, stain, flatness, spalling\}$ (4.1) 其中， I 表示图像， $R(I)$ 表示该图像需要执行的原子检测能力集合。与普通多标签分类不同，本文任务调度结果并不是最终用户结论，而是后续工程状态机的输入。因此，输出集合必须满足三个条件：第一，元素必须属于固定任务代码枚举；第二，集合 装 允许为空，表示图像不适合或不需要检测；第三，集合中的每个任务代码最多出现一次，并应按统一顺序归一化，以便数据库保存、前端展示和日志排查。这种问题建模的意义在于把 Agent 的开放语义理解能力压缩为工程系统可以消费 订 的离散动作。对于Agent 而言，判断图像中是否存在玻璃、石材、金属或明显缺陷线索属于语义判断；对于工程系统而言，真正可执行的动作只有五类原子检测能力。Agent 自主任务调度方法正是连接这两者的桥梁。线 4.5.2 材质语义与任务代码映射 幕墙常见材质与检测任务之间存在较明确的对应关系。金属构件主要关注锈蚀问题；石材或混凝土面板主要关注裂缝和污渍问题；玻璃幕墙主要关注平整度异常和爆裂问题。本文将这一工程经验固化为任务映射规则，并通过提示词约束 Agent 的输出。表 4.5 幕墙材质语义与检测任务代码映射关系 图像语义线索 对应任务代码 设计说明 金属幕墙、金属

连接件、明显锈蚀痕迹 石材、混凝土、类似石材面板 玻璃幕墙、玻璃反射面、明显玻璃破损 corrosion 金属锈蚀会影响构件耐久性和连接可靠性，是金属类图像的重点检测能力 crack、stain 石材类图像同时需要关注裂缝和污渍，前者偏安全与耐久性，后者偏外观和渗水线索 flatness、spalling 玻璃类图像既需要判断表面或安装状态异常，也需要判断是否存在爆裂风险 多材质组合幕墙 多类任务代码合并去重 对包含玻璃、金属和石材的图像，允许输出多个任务代码，后续并行执行非建筑幕墙、无法判断、无检测意义图像 空数组 系统不强行执行检测，避免产生无意义结果和错误报告依据 表 4.5 中的映射并不是简单关键词规则，而是提示 Agent 结合图像内容进行判断。比如，图像中只要出现大面积玻璃面板，即使没有明显爆裂，也应输出 flatness 和 spalling，因为检测任务的目的是确认已知缺陷，还包括排查潜在异常；图像中若出现石材或混凝土表面，即使裂缝不明显，也需要执行 crack 和stain，以获得可追溯检测结果。输出契约与服务端归一化 在分类路由环节，系统只接收一种输出形式：单行压缩 JSON 字符串，并且对象中只保留 task_codes 字段。该字段的值为任务代码字符串数组。例如，玻璃幕墙图像可以输出 $\{ "task_codes": ["flatness", "spalling"] \}$ ，非幕墙图像则输出装 $\{ "task_codes": [] \}$ 。这样设计不是为了追求格式上的简洁，而是为了让后续 Worker能拿到确定的任务输入。如果 Agent 额外输出自然语言解释、Markdown 代码块、中文任务名或其他字段，服务端就需要再猜测模型意图，子任务创建也会变得不稳定。因此，服务端不会默认提示词约束一定生效，而是会对 Agent 返回内容再做一次 订 归一化处理。处理

过程包括去除多余空白、解析JSON 对象、检查字段是否存在、检查 task_codes 是否为字符串数组、过滤或拒绝非法任务代码，并对合法任务代码去重后按统一顺序排序。归一化后的结果再转换为系统枚举，进入Core BIZ 的任务创建逻辑。线这个过程相当于把模型的开放输出收回到工程系统可识别的有限集合中：Agent 负责给出判断，服务端负责校验、兜底和保护业务流程。任务调度闭环设计 Agent 自主任务调度并不是调用一次Classification 服务就结束，而是检测主任务状态机中的前置环节。用户在前端启动 Agent 智能检测后，Core BIZ 会先为每张图像创建检测主任务；后台 Worker 再把主任务推进到分类中状态，获取图像访问地址，并调用 Classification 服务。分类 Agent 返回任务代码后，Classification 服务通过回调把结果交回 Core BIZ。随后 Core BIZ 在事务中写入任务代码、创建对应检测子任务，并把主任务推进到子任务检测阶段。这里需要特别区分两个边界。一方面，分类 Agent 只回答“应该做什么检测”，它不负责判断主任务是否完成，也不直接创建数据库子任务。是否进入检测阶段、是否发布状态事件，都由 Core BIZ 按照事务结果统一处理。另一方面，调度结果需要和后续检测结果绑定。主任务记录会保存本次生成的任务代码，用户以后查看结果时，可以知道系统为什么执行了这些子任务；项目报告在整合结论时，也可以据此判断某类检测是否真正执行过。异常场景与边界处理 任务调度中存在多种异常情况。第一，Agent 输出空任务列表。该情况并不一定是失败，可能表示图像不是建筑幕墙、图像质量不足、或无法可靠判断材质。系统应将其视为一种有效结果，并根据业务规则结束当前图像检测链路。第二，Agent 输出格式错误。该情况属于分类节点失败，应回调失败状态，允许用户后续重试。第三，Agent 输出合法但不符合实际材质。该情况需要通过人工复核和项目报告上下文进行纠偏，而不是在系统层面强行修改历史检测事实。本文没有采用“输出不确定时执行全部检测”的策略。原因在于，全部检测虽然看似保险，但会产生大量非适用检测结果，反而降低报告可信度。对于无法可靠判断的图像，系统更适合输出空任务，并由用户根据图像质量或项目需要重新上传、重新分组或人工补充说明。这样的策略符合土木工程场景对结论依据的审慎要求。原子检测能力工具化 原子检测能力工具化是指将团队前期积累的检测模型和图像处理脚本封装为统一可调度能力。第2.2 节已经介绍五类原子检测能力的技术基础，本节关注它们如何在本系统中成为Agent 可间接调度的工具。如图4.3 所示，工具化设计的关键不是重新提出某个检测模型，而是让不同来源、不同输入输出形式、不同产物数量的检测能力在同一工程协议下运行。统一注册机制 工具化的第一层是任务代码统一。五类能力通过统一任务代码注册，Reasoning 服务启动时读取模型目录，并检查每个任务代码下的模型文件是否存在、是否为目录、是否为Git LFS 指针文件。这样可以在服务启动阶段发现模型缺失或部署错误，而不是等到用户启动检测后才失败。工具选择与模型注册task_code 匹配检测能力 模型路径与运行参数加载动态产物清单生成注册负责能力发现运行目录隔离创建子任务工作目录 下载原图 / 区域图初始化产物目录目录负责并发隔离产物上传与完整性校验产物上传 MiniIO Uploaded=true 校验 产物数量与动态清单一致校验负责结果可信结构化结果生成缺陷状态 区域指标 图像产物 Key 检测 Summary 原始数据回调负责状态闭环Reasoning 回调 Core BIZ成功：写入子任务结果 失败：记录失败原因并等待重试原子检测执行金属锈蚀 石材裂缝 石材污渍 玻璃平整度 玻璃爆裂检测子任务工具选择与模型注册task_code 匹配检测能力 模型路径与运行参数加载动态产物清单生成注册负责能力发现运行目录隔离创建子任务工作目录 下载原图 / 区域图初始化产物目录目录负责并发隔离产物上传与完整性校验产物上传 MiniIO Uploaded=true 校验 产物数量与动态清单一致校验负责结果可信结构化结果生成缺陷状态 区域指标 图像产物 Key 检测 Summary 原始数据回调负责状态闭环 Reasoning 回调 Core BIZ成功：写入子任务结果 失败：记录失败原因并等待重试原子检测执行金属锈蚀 石材裂缝 石材污渍 玻璃平整度

玻璃爆裂检测子任务装 订 图 4.3 原子检测能力工具化执行流程 线 注册信息不仅包括任务代码，还包括模型名称、Python 执行入口和结果解析方式。通过注册机制，工程系统可以把不同模型统一看作“输入原图、输出报告和产物”的可调用工具。新增检测能力时，不需要改变 Agent 调用方式和 Worker 编排框架，只需新增任务代码、注册元数据、模型目录和 Python 检测入口。统一执行协议 工具化的第二层是执行协议统一。无论内部模型使用YOLO、ResNet、OpenCV 还是其他图像处理流程，对外都遵守相同执行约定：输入为项目图像原图，运行目录按任务代码和图像 UUID 隔离，输出为 report.json 和若干图像产物。Golang 层读取并压缩报告JSON，上传图像产物，再将结构化结果和产物清单回调给Core BIZ。Python 模型不直接访问数据库，也不直接参与业务状态推进。运行目录隔离是工具化中的重要设计。每个子任务拥有独立目录，目录中包含输入 图像、模型生成的中间产物和报告文件。这样可以避免并发任务之间相互覆盖，也便于失败后定位具体任务。对于需要重试的任务，系统可以重新创建运行目录并重新执行，而不依赖上一次执行遗留文件。通过统一执行协议，原本异构的检测模型和图像处理脚本被封装为可调度、可复核、可重试和可恢复的工程工具，为上层 Agent 调度和后续信息整合提供稳定的事实生成接口。统一结果契约 工具化的第三层是结果契约统一。不同任务的指标不同，例如锈蚀检测关注锈蚀区域数量、置信度、像素数和占比，裂缝检测关注裂缝像素与占比，污渍检测关注污渍 区域和污渍占比，平整度检测关注边缘、梯度、直线和频谱等指标，爆裂检测关注是否爆裂与置信度。但这些结果都必须被整理为可落库、可展示、可总结的结构化字段。只有具备这种契约，原子检测能力才能从实验脚本变成工程系统中的工具。表 4.6 五类原子检测能力的主要结构化指标与图像产物 订 检测能力 主要结构化指标 主要图像产物 金属锈蚀 是否锈蚀、区域数量、平均置信度、最高置信度、锈蚀像素数、锈蚀占比、区域列表 石材裂缝 是否存在裂缝、裂缝区域数量、裂缝像素数、裂缝占比、区域列表 石材污渍 是否存在污渍、污渍区域数量、平均污渍占比、线 最高污渍占比、区域列表 玻璃平整度 结果类型、异常区域数量、边缘异常、梯度异常、直线异常、频谱异常和相关数值指标 标注图、区域框、必要的检测可视化结果 裂缝浮层图、裂缝图、区域可视化结果 总体标注图、区域裁剪图、区域污渍结果图区域原图、梯度图、直线图、频谱图 玻璃爆裂 是否爆裂、置信度、分类结果 原图或分类结果相关报告产物 表4.6 展示了工具化输出的差异。系统不要求所有检测能力输出完全相同字段，而是要求它们遵守共同的外层契约：是否成功、报告JSON、产物清单、区域指标和任务状态。这样既保留每类检测模型的专业性，又保证上层 Agent 总结和前端展示能够使用统一入口读取结果。工具化与 Agent 调度的关系 原子检测能力并不直接暴露给用户，也不直接由 Agent 调用。Agent 输出的是任务代码，Core BIZ 根据任务代码创建子任务，Reasoning 服务再根据子任务代码查找已注册检测能力。这样的间接调用方式使系统能够控制执行顺序、并发、权限、失败重试和 结果落库。若让 Agent 直接调用 Python 工具，则难以保证数据库状态一致，也难以在产物上传失败时阻止任务被误判为成功。因此，本文中的“工具”不是大模型工具调用框架中的自由工具，而是工程系统管理下的受控工具。Agent 决定选择哪些工具，工程系统决定何时执行、如何执行、结果是否有效以及是否推进状态。这一点是本系统方案区别于开放式 Agent 的核心。LLM 分层信息整合策略 LLM 分层信息整合策略是本文从单图检测走向项目级韧性评估的关键设计。旧式做法通常把每个模型的检测结果直接展示给用户，用户再人工汇总；另一种极端做法是把所有检测结果一次性输入大模型并要求其生成报告。前者无法解决报告滞后问题，原子事实层缺陷状态 区域指标 置信度 / 占比图像产物 Key事实负责可追溯图像语义层图像级总结 Agent 生成总体概述、五类缺陷总结和后续建议单图负责语义压缩附件输入与

项目报告 Agent 读取多源检测结构化数据，按图像组、缺陷类型、复核意见进行多源融合项目负责多源融合结构化项目评估报告项目背景 总体结论 区域评估 风险等级 韧性评估 维护建议校验负责输出稳定输出压缩与一致性校验单行 JSON 对象 图像组与图像数校验 防止编造 前端字段化展示项目报告源数据层项目信息 图像组 检测结果 图像总结 人工复核 统计字段 检测事实与项目上下文原子事实层缺陷状态 区域指标 置信度 / 占比图像产物 Key 事实负责可追溯图像语义层图像级总结 Agent 生成总体概述、五类缺陷总结和后续建议单图负责语义压缩附件输入与项目报告 Agent 读取多源检测结构化数据，按图像组、缺陷类型、复核意见进行多源融合项目负责多源融合结构化项目评估报告项目背景 总体结论 区域评估 风险等级 韧性评估 维护建议校验负责输出稳定输出压缩与一致性校验单行 JSON 对象 图像组与图像数校验 防止编造 前端字段化展示项目报告源数据层项目信息 图像组 检测结果 图像总结 人工复核 统计字段检测事实与项目上下文装 后者容易受到上下文长度、低层指标噪声和输出不稳定影响。如图4.4 所示，本文采用分层整合方案，将信息处理划分为原子事实层、图像

语义层和项目报告层。订线图 4.4 LLM 分层信息整合策略流程 信息分层整合的必要性 项目级幕墙评估与普通文本摘要不同。输入数据既包括项目基础信息，又包括图像组、图像元数据、五类检测指标、区域列表、图像产物、图像级总结和人工复核意见。如果将所有底层 JSON 直接输入项目报告 Agent，一方面会造成上下文过长，另一方面低层细节会干扰模型对项目级风险的组织。相反，如果只把图像级总结输入项目报告 Agent，又会损失必要的结构化指标和复核依据。本文因此采用信息分层整合策略。原子检测能力首先产生可追溯事实，图像总结 Agent 将每张图像的事实压缩为语义概要，项目报告 Agent 再同时读取项目结构、检测事实、图像总结和人工复核意见等。这样既避免项目报告直接处理过多像素级或区域装级细节，又保留必要的证据字段。表 4.7 LLM 信息整合分层输入内容和输出作用 层次 输入内容 输出作用 原子事实层 五类原子检测能力输出的结构化结果，包括缺陷 保留可追溯、可展示、可复核的底层证据存在性、区域指标、置信度、占比和产物 订 图像语义层 单张图像实际执行过的检测结果 JSON 生成固定字段的图像总结，说明该图像是否存在缺陷、主要问题和维护建议 项目报告层 项目基础信息、图像组、每张图像检测结果、图像总结和人工复核意见 生成项目级韧性评估报告，给出整体状态、区域差异、风险等级和维护建议 表4.7 表明，图像级总结和项目级报告不是两个重复的摘要任务，而是两个不同抽象层次。图像级总结只回答“这一张图像说明什么”，输出字段限定为总体概述、五类缺陷总结和后续建议。项目级报告回答“整个项目说明什么”，需要综合图像组、缺陷分布、人工复核和项目背景，形成项目背景、数据集概述、总体结论、区域评估、风险等级、韧性评估、维护建议和限制说明等内容。图像级总结设计 图像级总结用于把单张图像已经产生的多类检测结果整理成前端可读的结构化文本。它的输入来自已落库的检测结果，不重新执行模型推理，也不重新判断图像是否存在某类缺陷。输出字段包括总体概述、金属锈蚀、石材裂缝、石材污渍、玻璃平整度、玻璃爆裂和后续建议。前端展示时可以根据字段是否为空决定显示哪些标签，避免把没有证据支撑的内容展示给用户。这一设计中特别需要区分“未执行检测”和“未发现缺陷”。例如，分类 Agent 没有选择金属锈蚀检测时，图像级总结不应写“未发现锈蚀”，因为系统实际并没有对该图像执行锈蚀检测。对于已经执行但未提示明显异常的任务，可以在总体概述中说明已执行任务未发现明显异常，但对应缺陷字段仍保持为空。这样处理后，图像级总结不会把检测范围之外的内容写成结论。从生成控制角度看，图像级总结需要有明确的证据边界。自然语言生成模型容易为了让文字完整而补充未执行任务的描述，这在工程报告中会造成误导。本文通过提示词规则和字段设计限制这种情况：只有存在对应材质或缺陷，并且检测结果能够支撑时，才填写对应字段；如果没有发现缺陷，后续建议可以为空；如果某类任务未执行，则不能把未执行任务等同于无缺陷结论。装 项目级报告源数据设计 项目报告生成设计的关键不在于让 Agent 写长文本，而在于如何把多源数据组织成可供 Agent 稳定阅读的上下文。Core BIZ 会先将项目基础信息、图像组、图像检测结 订 果、图像级总结和复核意见组装为结构化 JSON，并以 source.txt 文件形式存入对象 存储。项目报告 Agent 通过预签名 URL 下载该文件并作为附件读取。采用 source.txt 附件有两个原因。第一，项目级数据可能较长，直接把压缩 JSON 线 放在消息正文中容易触发上下文限制，也不利于日志阅读。附件方式可以让 Agent 明确读取完整源数据。第二，报告源数据本身是一个重要中间产物，保存在对象存储中可以 追溯报告依据。若项目报告存在争议，可以查看当时传给 Agent 的源数据，而不是只查看最终报告文本。表 4.8 项目级报告源数据设计 数据块 设计作用 项目基础信息 提供项目名称、建筑名称、地址、建成年份、描述等报告背景信息 图像组与图像列表 保留项目图像组织结构，使报告能够按立面、区域或图像组描述缺陷分布原子检测结果 提供五类检测能力的结构化事实，包括缺陷存在性、数值指标和区域列表 图像级总结 将单图多任务结果压缩为可读语义，帮助项目报告 Agent 更好理解单图重点人工复核意见 将用户对 Agent 结果的准确性判断和补充评论纳入最终报告依据 统计字段 包括图像组数量和图像数量，用于防止报告 Agent 编造数据规模 表4.8 主要说明报告源数据在进入Agent 之前需要做一次筛选。系统不会把数据库中的所有字段原样传入模型，例如内部 ID、任务主键、产物 SHA256、运行耗时以及部分图像坐标，这些字段更多用于系统内部追踪或产物校验，对项目报告本身帮助不大，因而会在组装上下文时被排除。相反，缺陷指标、区域数量、图像组结构、人工复核意见和图像级总结等信息会被保留下来，因为这些内容直接关系到报告中的风险判断和维护建议。这样处理后，报告源数据不会被底层实现字段占满，同时仍能保留形成建筑级韧性评估结论所需要的依据。项目级报告输出契约 项目级报告输出必须是结构化 JSON，而不是普通 Markdown 文本。原因在于前端需要按照字段展示项目背景、韧性评估目标、数据集概述、总体评估结论、总体风险等级、局部评估结论、局部风险等级、总体韧性评估、维护和改进

建议以及限制说明等信 息。结构化输出可以避免前端解析自然语言段落，也便于后续扩展报告打印、导出和字段化展示。项目报告 Agent 的输出还需要满足一致性约束。报告中的图像组数量和图像数量 必须来自 source.txt，不得自行估算；区域评估应基于源数据中的图像组组织，而不 订 是编造不存在的立面；风险等级需要与检测事实和人工复核保持一致；限制说明需要 指出图像检测无法完全替代现场复核。系统在接收回调时会 Agent 输出压缩为单行 JSON，并校验其为合法 JSON 对象。线 4.7.5 幻觉与上下文漂移防止机制 LLM 分层信息整合策略除了生成报告文本之外，还需要尽量控制幻觉和上下文漂移。项目级报告和普通摘要不同，报告中会出现项目名称、图像数量、立面区域、缺陷分布和风险判断等内容，这些信息一旦被模型自行补充，就会直接影响用户对建筑状态的理解。实际设计时，最需要避免的情况不是语言表述不够流畅，而是报告中出现源数据里没有的项目背景、图像组数量或风险结论。针对这一问题，本文从输入、提示词和回调校验三个位置进行约束。输入层面，系统在报告源数据中显式写入 group_count 和 image_count 等统计字段，避免模型自行根据文本描述推测数量。提示词层面，要求报告内容必须基于附件中的JSON，不允许补充不存在的项目背景或检测事实。回调层面，系统会对报告 JSON 进行压缩和合法性校验；如果必要字段缺失、字段类型不符合要求，或者整体结构无法解析，则将本次报告任务视为失败，而不是把异常文本继续写入正式报告结果。这些约束不能从理论上完全消除大模型幻觉，但可以降低幻觉进入系统结果的概率。更重要的是，系统会将报告源数据保存为 source.txt，使报告依据能够被

回看。用户如果认为报告内容不准确，可以把报告结论与源数据逐项对照，进一步判断问题来自检测结果、人工复核、源数据组织，还是Agent生成过程本身。对于工程评估场景而言，可追溯性比单纯追求语言完整度更重要，因为最终报告需要能够说明“结论依据来自哪里”。

4.7.6 信息分层整合与工程认知路径

LLM 分层信息整合策略体现了工程报告生成的认知路径。实际工程师撰写项目报告时，也不会直接从每个像素坐标开始写，而是先理解每张图像的主要现象，再比较不同区域之间的缺陷分布，最后给出整体风险和维护优先级。LLM 分层信息整合策略把这一过程固化为软件流程，使 Agent 的语言能力服务于工程语义组织，而不是替代检测事实。

从用户体验角度看，图像级总结和项目级报告也解决了不同层次用户需求。项目操作人员在检测阶段需要快速知道某张图是否存在异常，因此需要图像级总结；项目负责人或土木工程团队更关注建筑整体状态、区域风险和维护优先级，因此需要项目级报告。分层输出使同一套检测事实能够服务不同层次的业务阅读场景。

任务状态流转与人机协同机制

表 4.9	检测主任务状态枚举含义	状态枚举	业务含义
pending	主任务已创建，等待后台 Worker 消费	classifying	分类 Agent 正在判断该图像需要执行哪些原子检测能力
detecting	原子检测子任务已创建，系统正在等待检测结果和产物上传	summarizing	子任务结果已满足总结条件，图像总结 Agent 正在生成
succeeded	该图像检测链路已成功结束	failed	必要节点失败，当前图像检测链路失败，可通过重试重新创建任务

如图 4.5 和表 4.9 所示，任务状态流转是本文方法落地的骨架。每张图像对应一个检测主任务，主任务从 pending 开始，后台 Worker 消费后进入 classifying。分类成功后，如果 Agent 输出了需要执行的任务代码，系统创建对应子任务并进入 detecting；如果 Agent 输出空任务列表，说明当前图像不需要进入原子检测链路，主任务可以直接成功结束。子任务全部成功后，系统创建图像总结任务并进入 summarizing；图像总结成功后，主任务进入 succeeded；任一必要节点失败，主任务进入 failed。分类 Agent 调度 pending → classifying 输出任务代码或空集合 Agent 负责任务选择子任务检测执行 detecting Reasoning 执行原子检测并上传产物 Reasoning 只回调结果状态收敛与失败重试成功进入 succeeded 并开放复核 失败进入 failed 并等待重试 Core BIZ 统一推进状态人工复核确认查看产物、区域指标和 Agent 总结 选择准确 / 不准确并补充评论用户负责复核判断报告吸收复核上下文项目报告上下文归并检测事实 + 图像级总结 + 人工复核意见进入报告源数据图像级总结生成 summarizing 基于已落库检测结果生成图像级总结检测主任务创建分类 Agent 调度 pending → classifying 输出任务代码或空集合 Agent 负责任务选择子任务检测执行 detecting Reasoning 执行原子检测并上传产物 Reasoning 只回调结果状态收敛与失败重试成功进入 succeeded 并开放复核 失败进入 failed 并等待重试 Core BIZ 统一推进状态人工复核确认查看产物、区域指标和 Agent 总结 选择准确 / 不准确并补充评论用户负责复核判断报告吸收复核上下文项目报告上下文归并检测事实 + 图像级总结 + 人工复核意见进入报告源数据图像级总结生成 summarizing 基于已落库检测结果生成图像级总结检测主任务创建装

订 图 4.5 任务状态流转与人机协同机制流程图

状态机与 Agent 方法之间存在直接关系。分类 Agent 的输出决定是否创建子任务，Reasoning 子任务结果决定是否进入图像总结，图像总结结果决定检测主任务是否完成。也就是说，Agent 决策不是孤立文本，而是会改变后续状态流转。因此，状态推进必须由 Core BIZ 统一执行，避免 Activity 服务或前端根据局部信

息自行改变业务状态。在人机协同方面，本文没有把检测结果直接作为最终结论，而是在检测完成后设置人工复核阶段。用户可以同时查看检测产物、区域指标和 Agent 总结，再根据实际情况选择“准确”或“不准确”，也可以补充自己的判断说明。复核信息不会删除原始检测事实，也不会覆盖模型产物，而是作为后续项目报告的一部分输入。这样处理后，系统既保留自动检测形成的证据，也能记录工程人员对结果的判断过程。人工复核的作用主要体现在两个方面。一方面，幕墙维护建议可能涉及安全责任、维修成本和施工安排，土木工程场景很难完全依赖自动化结论直接决策，因此需要由人对关键结果进行确认。另一方面，复核信息可以为项目级报告提供纠偏依据。若某张图像的检测结果被用户标记为“不准确”，项目报告 Agent 在汇总时应当看到这一信息，并在整体判断中有所体现，而不是把模型输出无条件当作事实。本章小结 本章为建筑幕墙韧性评估软件系统设计了主要技术方案。方案设计中没有把所有智能能力压到一个端到端模型里，而是把任务调度、原子检测、图像级总结、项目报告 和人工复核拆成几个边界清楚的环节。分类 Agent 负责把图像语义转换为有限任务代码，Reasoning 服务负责把已有检测模型封装成可调度能力，图像总结 Agent 和项目报告 Agent 再把底层检测结果逐步整理为工程语义。人工复核则保留人的专业判断，使报告结论不只依赖自动检测结果。订 这种方案的链路相对较长，但每一步都有对应的证据来源和失败处理位置。相比固定检测流程，它可以根据图像材质选择需要执行的检测能力，减少无意义的模型调用；相比普通工程系统，它又引入 Agent 处理语义判断和信息整合，使系统不再停留于单图线单缺陷检测，而是进一步面向项目级韧性评估。第 5 章将在本章基础上继续说明这些设计如何落到多服务架构、协议、数据库、缓存、对象存储、Worker、回调、消息队列、WebSocket、CronJob 和部署配置中。系统架构与工程实现 第 4 章从方法层说明了本文如何通过 Agent 自主任务调度方法、原子检测能力工具化和 LLM 分层信息整合策略解决建筑幕墙韧性评估中的任务选择与信息融合问题。本章转向工程链路实现，重点说明上述方法如何落地为一个可运行、可部署、可观测、可恢复的软件系统。系统实现的核心不是简单把若干模型串联起来，而是围绕项目、图像、任务、产物、复核和报告构建完整的工程链路，使 Agent 能力在受控状态机中稳定发挥作用。关键技术语 本章涉及的对象更多偏向工程实现，包括服务模块、通信方式、存储对象和运维机制等。为便于阅读，现在表 5.1 中对本章关键技术语进行说明。表 5.1 系统架构与工程实现关键技术语 术语 含义 订 PSM 服务进程或模块的逻辑名称，用于区分 Core API、Core BIZ、Activity 服务、CronJob 和日志采集等运行单元 Core API 面向前端提供 HTTP、鉴权入口和 WebSocket 通道等的接入服务 Core BIZ 系统业务中心，负责状态机推进、数据库事务、对象存储授权、Worker 编排、回调处理、事件发布和定时任务等 Activity 服务 独立智能能力服务，包括 Classification、Reasoning 和 Summary，分别封装分类 Agent、原子检测模型和总结 Agent 线 Worker 后台任务执行器，异步推进检测任务和报告任务，避免用户请求长时间阻塞 回调 Activity 服务完成任务后，通过 gRPC 将状态、结果、产物清单和错误信息上报给 Core BIZ 的机制 RocketMQ 事件 Core BIZ 在图像、检测任务或报告状态变化后发布的项目事件，由 Core API 消费并转发给前端 WebSocket Scope WebSocket 推送范围，包括图像资产、检测任务和报告三类，用于隔离不同阶段的实时消息 预签名 URL Core BIZ 为前端或 Activity 服务生成的有时效对象存储访问地址，用于上传或下载图像与报告源文件 产物 Reasoning 服务生成并上传到 MinIO 的中间图像，如标注图、区域图、浮层图、梯度图、直线图和频谱图等 CronJob Core BIZ 内部定时任务，用于上传超时处理和 MinIO 脏对象清理等后台维护工作 总体架构设计与实现 系统采用前后端分离与微服务架构。前端 Core Web 负责项目工作台、项目阶段页面、图像资产管理、检测进度展示、检测结果查看、人工复核和报告展示；后端按照职责拆分为 Core API、Core BIZ、Activity Classification、Activity Reasoning、Activity 智能活动服务层 Classification 分类 Agent 图像附件任务代码 Reasoning 原子检测能力检测产物与指标 report.json Summary 总结 Agent 报告上下文 source.txt Python Worker 模型脚本 Summary 输出压缩 JSON 智能活动服务层 Classification 分类 Agent 图像附件任务代码 Reasoning 原子检测能

力检测产物与指标report.jsonSummary总结 Agent报告上下文source.txtPython Worker模型脚本Summary 输出压缩JSON 部署与运行治理层镜像发布日志采集依赖启动环境变量统一网络Docker部署与运行治理层镜像发布日志采集依赖启动环境变量统一网络DockerSummary 和ICW Common；基础设施包含MySQL、Redis、MinIO 和RocketMQ 等；部署层由 Docker Compose 统一组织容器、网络、环境变量、日志采集和依赖启动顺序。用户与前端层WebSocket实时刷新人工复核报告打印图像结果可视化阶段流程状态展示Core Web React 页面浏览器用户项目操作用户与前端层WebSocket实时刷新人工复核报告打印图像结果可视化阶段流程状态展示Core Web React 页面浏览器用户项目操作核心业务服务层Core API HTTP 接入ICW Common Proto / Enum鉴权权限请求日志Repository事务封装WebSocket Hub 回调处理聚合推进Core BIZ状态机中心事件生产状态广播Worker任务编排CronJob后台维护核心业务服务层Core API HTTP 接入ICW Common Proto / Enum鉴权权限请求日志Repository事务封装WebSocket Hub回调处理聚合推进Core BIZ状态机中心事件生产状态广播Worker任务编排C

ronJob后台维护 装 订 图 5.1 系统总体架构与服务关系 基础设施与数据层Config环境变量配置Log Sidecar分 PSM 日志RocketMQ项目事件MinIO图像与产物Redis令牌与缓存MySQL结构化状态基础设施与数据层Config环境变量配置Log Sidecar分 PSM 日志RocketMQ项目事件MinIO图像与产物Redis令牌与缓存MySQL结构化状态线 如图5.1 所示，系统总体架构可以概括为五个层次：部署与运行治理层覆盖所有服务，提供容器编排、网络隔离、环境配置、日志采集和镜像发布能力；用户与前端层负责页面交互和实时状态呈现；核心业务服务层负责接入、鉴权、状态机、任务编排、回调处理和事件生产；智能活动服务层负责分类、检测和总结等相对独立的智能能力；基础设施与数据层负责结构化状态、缓存、对象产物、事件消息和日志留存。各服务或模块的实现职责见表 5.2。该架构遵循“智能能力外置，业务状态内聚”的原则。Classification、Reasoning 和 Summary 可以独立演进各自 Agent 或模型逻辑，但不直接拥有业务状态；项目、图像、任务、复核和报告状态集中由 Core BIZ 控制；Core API 不承担复杂业务事务，只负责接入与事件转发；Core Web 只展示和触发业务动作，不自行判断最终状态。这样的边界划分避免了 Agent 与模型服务越权，也使失败恢复和日志追踪更清晰。表 5.2 系统服务划分与实现职责 服务或模块 实现职责 Core Web React 前端页面、阶段化项目流程、图像和检测结果可视化、人工复核、报告展示与打印 Core API HTTP 路由、鉴权中间件、请求日志、跨域处理、WebSocket 连接、RocketMQ 事件消费与前端广播 Core BIZ 业务状态机、数据库事务、对象存储封装、Redis 缓存、检测 Worker、报告 Worker、回调处理、事件生产和 Cronjob Activity Classification 调用分类 Agent，上传图像附件，解析受控JSON 输出，回调检测任务代码 Activity Reasoning 统一封装五类 Python 原子检测能力，负责原图下载、模型执行、产物上传和结果回调 Activity Summary 调用图像总结Agent 和项目报告Agent，压缩Agent 输出JSON，并回调总结与报告结果 ICW Common Protocol Buffers 协议、共享枚举、gRPC 调用封装、错误码、日志工具、JSON 归一化和 通用工具函数 从工程复杂度看，用户在页面上点击一次“启动 Agent 智能检测”，后端并不是执行一个简单接口。请求先由Core Web 发起，经Core API 完成鉴权后进入Core BIZ；随 装 后 Core BIZ 为图像创建检测主任务，并交给 Worker 异步推进。Worker 会先调用分类 Agent，等分类结果回调后再创建多个检测子任务；Reasoning 服务负责执行 Python 模型并上传检测产物，Summary 服务再基于检测结果生成图像级总结。与此同时，BIZ 还需要发布RocketMQ 事件，API 消费事件后通过WebSocket 把状态推送给前端。也就是 订 说，一次用户操作会被拆成多个后台步骤，系统需要在并发执行、回调延迟或某个环节失败时，仍然保持数据库状态和前端状态一致。通信与协议设计 线 系统通信分为四类。第一类是前端到 Core API 的 HTTP 请求，用于登录、项目管理、图像上传授权、检测启动、结果查询、复核更新和报告查询。第二类是 Core API、Core BIZ 与 Activity 服务之间的 gRPC 请求，用于服务间强类型调用。第三类是 Core BIZ 到Activity 服务的异步启动与Activity 服务到Core BIZ 的回调，用于长任务结果上报。第四类是 Core BIZ 发布RocketMQ 项目事件、Core API 消费事件并通过WebSocket广播给前端，用于实时状态刷新。Protocol Buffers 是内部协议的核心。系统将 Auth、ProjectCore、ProjectProfile、Pro-jectAssets、ProjectDetection、ProjectReview、ProjectReport、Socket 和 User 等 BIZ 接口定义在 Common 仓库中；Activity 侧定义 Classification、Reasoning 和 Summary 三类服务接口；前端API 类型也由 proto 生成 TypeScript 文件。通过共享协议，项目进度、图像状态、检测任务代码、主任务状态、子任务状态、复核结论、报告状态和WebSocket Scope 等均使用统一枚举，以避免服务间常量漂移。请求链路中还通过请求 ID 实现日志关联。Core API 在 HTTP 中间件生成请求 ID，gRPC 调用时通过 Metadata 透传，Activity 服务异步执行和回调时继续携带该请求 ID。这样可以在日志中关联一次用户操作、BIZ 处理、Agent 调用、模型执行和回调结果，对长链路排障具有直接价值。RocketMQ 主要承担业务状态变化后的项目事件转发。Core BIZ 在图像上传、检测主任务、检测子任务、图像级总结和项目报告等状态发生变化后，会把项目 ID、事件类型和事件载荷写入统一 Topic。Core API 订阅该 Topic 后，再根据事件类型转换成前端需要的 WebSocket Scope。这样划分后，BIZ 不需要关心当前有哪些客户端在线，只要在事务完成后发布已经发生的事实事件；API 也不参与业务事务，只负责消费事件并 装 向前端广播。长任务回调不会直接占用 HTTP 请求，页面中的图像资产、检测进度和报告状态也可以通过增量事件更新。WebSocket 连接采用“连接票据 + Scope”的设计。前端建立连接前，先通过鉴权订 HTTP 请求申请连接票据，然后携带项目ID、Scope 和Ticket 发起WebSocket 连接。这里的Scope 用来区分图像资产、检测任务和报告状态，避免某个页面收到与当前视图无 关的全部事件。API 侧Hub 按项目和Scope 维护连接集合；当 RocketMQ 消费到项目事 件后，只把消息广播给对应项目、对应 Scope 下的客户端即可。线 模块设计与实现 本节说明各服务模块的核心设计与实现，每个模块都给出职

责说明和核心时序图。Core API 服务 Core API 位于前端和业务服务之间，主要负责把页面请求接入后端系统。实现上，该服务基于Gin 注册HTTP 路由，认证、用户头像、项目核心流程、项目基础信息、图像资产、智能检测、人工复核、评估报告和 WebSocket 连接票据等接口都由这一层对外暴露。请求进入后，API 先通过鉴权中间件解析令牌中的用户身份，再通过项目访问权限中间件检查用户是否可以访问当前项目。通过校验后，请求才会被转换为 Core BIZ 的 gRPC 调用。实时状态更新也由 Core API 负责承接。Core BIZ 不直接保存前端连接信息，它只 RocketMQRocketMQ在图像状态、检测任务状态和报告状态变化后，把项目事件写入 RocketMQ。Core API订阅这些项目事件，再根据事件类型选择对应的WebSocket Scope，并广播给相应项目下的前端连接。这样划分后，BIZ 可以专注业务状态和事务处理，WebSocket 连接、事件消费和消息推送则集中在 API 层完成。Core WebCore WebCore API HTTP HTTP 请求 / 携带 Token 与 ProjectId 登陆鉴权与项目权限 Core BIZ gRPC WebSocket Hub 路由匹配 / 绑定请求参数 / 生成 RequestId 解析用户身份 / 校验项目权限 装 Userid / Project 权限结果 封装 gRPC Payload / Metadata 透传 RequestId 业务响应 / 业务错

误码 统一 ApiEnvelope / 错误映射 / 访问日志 订 HTTP 响应 业务状态变化后发布项目事件 API 消费事件并解析
 EventType / Scope 线 按 ProjectId + Scope 广播 图 5.2 Core API 服务核心时序图 如图 5.2 所示，Core API 的重点
 在于实现前端页面中同步查询和异步事件的统一。首次进入页面或手动刷新时，前端通过 HTTP 获取全量状态；任务执行
 过程中，API 通过WebSocket 推送增量事件。前端收到事件后静默刷新局部数据，避免页面反复进入加载态。Core BIZ
 服务 Core BIZ 是系统中最复杂的服务，也是工程链路的状态中心。它负责项目创建、项目阶段推进、图像组管理、图像上
 传授权、图像上报、检测任务创建、人工复核、报告任务创建、对象存储访问、Redis 缓存、RocketMQ 事件生产、
 CronJob 定时任务和后台 Worker 调度器等。所有会改变业务状态的操作都集中在 BIZ 服务中执行。BIZ 服务的核心设计
 是“服务层 + Repository 层 + Worker 层”。服务层实现 gRPC接口和业务规则，Repository 层封装MySQL、Redis、
 MiniIO、RocketMQ 和SMTP 操作，Worker 层异步推进检测和报告任务。Activity 服务回调BIZ 后，BIZ 在数据库事务
 中写入结果、判断聚合状态、创建下一阶段任务并发布项目事件。MySQLMySQLMiniIO / RedisMiniIO /
 RedisWorkerWorkerRocketMQRocketMQ Core API启动检测 / 推进阶段 / 查询状态校验项目阶段 / 幂等检查 / 构造领
 域对象开启事务：写入项目、图像、检测主任务任务 UUID / 当前状态 / 版本字段生成预签名 URL / 写入 Redis 临时控制
 对象地址 / 缓存结果提交后台任务并释放用户请求按状态启动分类、检测、总结或报告Activity 回调结果 / 失败原因 / 产物
 清单回调事务：写结果、聚合子状态、创建下一阶段任务落库成功 / 回滚失败读取 / 写入 source.txt、产物授权、URL 缓
 存对象 Key / 缓存命中 / 授权策略发布图像、检测或报告状态事件gRPC 响应或异步处理结束Core BIZ ServiceActivity
 服务Core API启动检测 / 推进阶段 / 查询状态校验项目阶段 / 幂等检查 / 构造领域对象开启事务：写入项目、图像、检测
 主任务任务 UUID / 当前状态 / 版本字段生成预签名 URL / 写入 Redis 临时控制对象地址 / 缓存结果提交后台任务并释放
 用户请求按状态启动分类、检测、总结或报告Activity 回调结果 / 失败原因 / 产物清单回调事务：写结果、聚合子状态、
 创建下一阶段任务落库成功 / 回滚失败读取 / 写入 source.txt、产物授权、URL 缓存对象 Key / 缓存命中 / 授权策略发布
 图像、检测或报告状态事件gRPC 响应或异步处理结束Core BIZ ServiceActivity服务装 订 线图 5.3 Core BIZ 服务核心
 时序图 如图5.3 所示，BIZ 服务既是请求处理者，也是长任务编排者。Worker 只负责启动 Activity 服务，真正结果通过
 回调回到 BIZ。这样可以避免 Worker 阻塞等待模型推理，也能保证所有状态变更都回到同一业务中心处理。Core BIZ
 内部还包含两个CronJob。上传中图像超时任务定期扫描长时间未完成上报的图像，将其状态置为失败并发布事件；MiniO
 脏对象清理任务每小时扫描对象存储，删除失败图像、失败子任务、失败报告和数据库已无引用对象对应的文件夹，并设
 置 30 分钟保护窗口，避免刚生成但尚未完成数据库状态同步的对象被误删。Classification 服务 Classification 服务封
 装分类Agent。BIZ 启动分类任务时传入任务UUID、图像 UUID和原图预签名下载地址。如图 5.4 所示，Classification
 服务下载并压缩图像后，通过 Agent平台文件上传接口上传附件，再通过流式对话接口获取模型输出。服务端将 Agent 输
 出归一化为 JSON 对象，解析 task_codes 任务代码，转换为共享枚举后回调 BIZ。Core BIZCore BIZClassification
 Service StartClassification(TaskUuid, ImageUuid, Url) MiniIO 图像原图 Agent 文件上传 分类 Agent 对话 装 加载
 Prompt / 校验BotId 与Token / 创建请求上下文 按预签名URL 下载原图附件 订 图像字节流 压缩图像/ 控制附件体积/ 生
 成文件名 上传图像附件 线 FileId 提示词+ FileId 流式调用分类Agent 模型输出：JSON 字符串
 CompactAgentJSONObjectString / 解析TaskCodes 非法代码拦截/ 去重/ 固定顺序排序/ 空集合允许
 ReportClassificationResult(TaskCodes / Error) 图 5.4 Classification 服务核心时序图 分类Agent 只决定任务代码，
 不创建子任务。输出异常时，Classification 服务回调 失败状态；输出成功时，BIZ 仍要进行枚举转换、任务去重和子
 任务创建。这样既利用 Agent 对幕墙材质的语义判断能力，又避免 Agent 越过业务状态机。Reasoning 服务 Core
 BIZCore BIZ Reasoning ServiceReasoning Service 能力注册表能力注册表 MiniOMiniO Python WorkerPython
 Worker 检测模型脚本检测模型脚本 StartReasoning(TaskCode, ImageUuid) 按 TaskCode 查询执行器与产物规则 模
 型入口 / 动态产物清单获取最新原图 URL 与上传授权 装 Presign Get / Artifact Policy 创建隔离运行目录 / 初始化输入
 与产物路径 下载原图到运行目录 订 原图文件 提交 Python 检测任务 懒加载模型并执行脚本 report.json / 图像产物 线
 检测结果目录 读取并压缩 report.json / 计算产物 SHA256 上传全部产物并校验 Uploaded=True Object Key /
 SHA256 / Uploaded 产物数量与动态清单校验 / 失败则阻断成功回调 ReportReasoningResult(Result JSON /
 Artifacts / Error) 图 5.5 Reasoning 服务核心时序图 Reasoning 服务是 Golang 工程层与 Python 模型层之间的桥
 梁。服务启动时注册五类原子检测能力，并根据任务代码检查模型目录中对应权重文件是否存在、是否可读取、是否为
 Git LFS 指针文件。如图 5.5 所示，任务执行时，Reasoning 服务从BIZ 获取最新原图下载地址，下载到隔离运行目录，
 再调用常驻 Python Worker 执行对应检测模块。模型执行结束后，Golang 层读取压缩后的 report.json，再从 BIZ 获取
 产物上传授权，将目录中的产物并上传到 MiniIO，并带着结果 JSON 和产物清单回调 BIZ。Reasoning 服务在真正开
 始执行检测时，才向 BIZ 申请原图下载地址和产物上传授权。这样做是因为检测任务可能在队列中等待一段时间，如果启
 动任务时就把预签名 URL 传入Reasoning，等模型实际执行时地址可能已经过期，进而导致原图下载或产物上传失败。对
 于除玻璃爆裂以外的检测任务，Reasoning 还会检查产物是否为空、是否全部上传成功；如果产物不完整，则回调失败状
 态，而不是把缺失产物的结果当作成功 任务。装 Python 侧模型采用懒加载方式运行。Golang 服务保持常驻 Worker，
 并把具体检测任务以 JSON 请求形式发送给 Python 执行；Python Worker 从标准输入读取任务后，根据task_code 定
 位到python/detectors/{task_code}/main.py，再通过动态导入加载 订 对应 detector 模块。每个 detector 都对外提
 供统一的 detect(image_path) 入口，内部可以自由使用 PyTorch、OpenCV 或其他图像处理逻辑，但输出仍约定写入
 当前任务运行目录下的report.json 和相关产物图像。首次调用某类 detector 时，Python 才加载对 应模型权重，并将模
 块缓存在内存中；后续同类任务会复用已加载 detector。这样既可 线 以保留 Python 在图像处理和深度学习模型调用方
 面的优势，也避免每次检测都重新加载模型权重带来的额外延迟。Summary 服务 Summary 服务包含图像总结和项目报
 告两类任务。如图 5.6 所示，图像总结任务接收 BIZ 组装的单图检测结果 JSON，直接调用检测总结 Agent，并要求输出
 固定七字段 JSON：总体概述、五类缺陷总结和后续建议。项目报告任务接收 BIZ 提供的source.txt预签名地址，先下载
 该附件，再将附件上传至报告 Agent，要求输出固定项目报告 JSON。Summary 服务的实现重点在于输出稳定性。
 Agent 输出需要经过 JSON 压缩和对象校验，项目报告还要校验输出中的图像组数量和图像数量是否与源数据一致，防止
 Agent 生成与项目数据不匹配的报告。数据库中保存的是压缩 JSON 字符串，前端按字 段解析和展示。Core BIZCore
 BIZSummary Service MiniIO source.txt Agent 附件上传 总结 Agent 对话 StartDetectionSummary /
 StartProjectSummary 读取任务类型 / 加载 Detection 或 Project Prompt 检测总结：使用结构化 JSON 文本输入 项目
 报告：下载 source.txt 结构化源数据文本 装 上传 source.txt 附件或复用单图文本 FileId / 输入上下文 提示词 + 文本 /

附件调用 Agent 订 检测总结 JSON 或项目报告 JSON CompactAgentJSONObjectString / JSON Object 校验 线 项目报告校验 GroupCount、ImageCount 与源数据一致 ReportDetectionSummaryResult / ReportProjectSummaryResult 图 5.6 Summary 服务核心时序图 Core Web 服务 Core Web 使用 React 和 TypeScript 实现，基于 Vite 构建。前端页面按项目阶段组织，包括项目基础信息、图像资产构建、Agent 智能检测、人工复核确认和评估报告生成。前端类型尽量复用 Common 中生成的 TypeScript 结构和枚举，避免手写常量与后端不一致。HTTP 请求统一经过 Axios 封装，处理令牌刷新、错误提示和 API 地址配置。如图 5.7 所示，前端采用“首次全量加载 + WebSocket 增量事件 + 静默刷新”的方式维护页面状态。首次进入页面时显示加载态并拉取完整数据；WebSocket 连接建立或收到事件后，不再强制展示全页面加载，而是静默拉取最新列表并更新局部状态。检测结果弹窗、进度弹窗和报告页面均基于结构化数据渲染，而不是直接展示原始 JSON。用户用户阶段页面组件 进入项目阶段页面 状态与弹窗组件 HTTP Core APICore APIWebSocketWebSocketClient 初始化页面状态 / 显示首次加载态 查询基础信息、资产、检测、报告 请求前判断 Access Token 是否需要刷新 HTTP 请求 / Payload Request 格式 装 结构化响应 / 401 则排队刷新 Token 页面数据 根据阶段 Scope 创建 Socket Ticket 并建连 订 图像 / 检测 / 报告状态事件静默刷新受影响列表，不触发全屏加载 线 查看进度 / 结果 / 复核 / 打印报告 最新列表数据 打开弹窗，默认 Agent 总结页并重置 Tab 按生成类型和枚举字段渲染指标、产物、评论 图 5.7 Core Web 服务核心时序图 ICW Common 模块 ICW Common 是跨仓库共享基础设施。它包含 Protocol Buffers 定义、生成后的 Golang/TypeScript 代码、共享枚举、错误码、日志常量、JSON 压缩归一化、gRPC 调用封装、表格化日志输出和 ID 编解码等工具。Common 模块的作用是确保多个服务在协议和

基础行为上保持一致。开发者开发者 Core Web TypeScriptCore Web TypeScriptCommon ProtoCommon Proto运行期gRPC / HTTP运行期gRPC / HTTPBuf / Protoc生成器Buf / Protoc生成器维护接口、枚举、消息结构 Golang 服务 API / BIZ / Activity 执行 Buf Generate / Protoc Go Pb、gRPC Stub、TypeScript Gen 统一枚举：任务代码、状态、Scope、事件类型 发布 gRPC 接口、错误码、JSON 工具、日志工具 同步请求响应结构、枚举与常量 go fmt / goimports / go vet / make verify src/gen/ 禁止 Lint，业务代码直接引用生成类型 装 按生成类型封装 Payload HTTP / gRPC 调用进入后端 统一错误码、状态枚举和 JSON 归一化结果 订 图 5.8 ICW Common 模块核心时序图 图 5.8 展示了 Common 模块的开发与使用链路。共享枚举对本文系统尤其重要，因为 Agent 输出的任务代码、WebSocket Scope、项目事件类型和检测状态都需要在前后 线 端与多个后端服务间保持一致。统一协议减少了重复定义，也降低了后续新增检测能力时的维护成本。数据实体设计与实现 系统数据层需要同时支撑项目阶段流转、图像资产管理、Agent 任务调度、原子检测结果、人工复核、项目报告和对象产物追踪。为避免将所有业务信息堆叠在单个 JSON 字段中，本文采用“结构化数据库 + Redis 临时状态 + MinIO 对象存储”的组合方式：MySQL 负责保存可查询、可聚合、可事务化的数据实体；Redis 负责保存令牌、缓存和短时控制状态；MinIO 负责保存图像、检测产物和报告源数据等二进制或大体积文件。三者分工明确，使系统既能支持复杂业务查询，又能控制数据库体积和对象访问权限。在 MySQL 设计中，本文按照第三范式拆分业务表，将项目、图像组、图像、检测主任务、检测子任务、复核记录和报告记录分别建模。每张表只保存与自身实体直接相关的字段，跨实体关系通过主键、外键或业务 UUID 关联，而不是在一张宽表中重复保存项目名称、图像路径、任务状态和检测结果。这样可以减少数据冗余，也能避免项目基础信息修改后多处字段不一致的问题。对于区域指标、产物清单等结构不完全固定的数据，系统仍保留 JSON 字段，但这些字段只承担局部扩展作用，不替代核心业务实体建模。数据库设计与实现 如表 5.3 所示，数据库设计围绕“项目级资产”和“图像级任务”展开。项目表保存项目名称、建筑名称、位置、建成年份、建筑描述、已知问题、评估目标、项目状态和项目进度；图像组表保存项目内图像分组名称、用户归属和排序信息，用于表达立 装 面、区域或用户自定义图像集合；图像表保存单张图像的 UUID、所属图像组、文件名、尺寸、大小、上传状态和元数据，是后续检测主任务、检测子任务和人工复核的图像级数据入口；检测主任务表保存每张图像的检测整体状态、五类子任务是否执行、子任务 ID、图像总结任务 ID 和人工复核字段；五张子任务表分别保存不同原子检测能力的专 订 属指标；图像总结任务表保存图像级总结；项目报告表保存项目级报告 JSON 和状态。表 5.3 核心数据库实体与作用 核心数据实体 作用 线 projects 保存项目基础信息、项目状态和阶段进度，是所有业务数据的归属边界 project_groups 保存项目下的图像组名称、用户归属、项目归属和排序优先级，用于组 织立面、区域或用户自定义图像集合 project_group_images 保存单张图像的 UUID、所属图像组、文件名、MIME 类型、尺寸、大小、元数据、上传状态和上传时间 project_detection_tasks 保存单张图像检测主任务、任务路由结果、主任务状态、总结任务和人工复核字段 五类检测子任务表 分别保存锈蚀、裂缝、污渍、平整度、爆裂检测结果、区域 JSON、产物 SHA256 和运行耗时 project_detection_summary_tasks 保存图像级总结任务状态和 Agent 输出结果 project_reports 保存项目级报告状态和项目报告 JSON 数据库事务用于保证状态推进的一致性。例如，分类回调成功后，BIZ 需要在同一事务内写入任务路由字段、创建子任务、关联子任务 ID、创建必要的总结任务并推进主任务状态；子任务回调后，BIZ 需要写入专属指标、保存产物 SHA256、更新子任务状态，并判断同一主任务下所有必要子任务是否完成；项目报告回调后，BIZ 需要写入报告 JSON、更新报告状态并发布报告事件。事务边界保证了前端查询到的状态与任务 记录保持一致。不同子任务表采用“公共任务字段 + 专属指标字段”的结构。公共字段便于统一查询和状态展示，专属字段便于前端按任务类型展示不同指标，也方便项目报告源数据按任务类型组装。相比所有检测结果都以一个大 JSON 字段保存，该结构更利于状态聚合、索引优化和工程维护。缓存设计与实现 Redis 在本系统中主要保存一些有效期较短、但访问频率较高的数据。安全相关状态占了一部分，例如邮箱验证码、登录失败计数、账号短时锁定、Access Token 黑名单和 Refresh Token 刷新互斥锁。邮箱验证码会按照登录、注册、重置密码等场景分别保 装 存，验证成功后立即消费，避免同一个验证码被重复使用；登录失败计数用于限制密码或验证码的连续尝试；Refresh Token 刷新互斥锁则用于处理并发刷新问题，防止同一个 Refresh Token 在短时间内被多个请求同时换新。另一部分缓存与对象访问有关。前端查看图像资产、检测结果或项目缩略图时，经 订 常会反复请求同一张原图、缩略图或检测产物。如果每次都重新生成预签名下载 URL，会增加 BIZ 与对象存储之间的重复操作。因此，系统会把部分预签名 UR

L 暂存在 Redis 中，并为其设置较短 TTL。Socket Ticket 和上传中的图像状态也放在 Redis 中保存，它线 们只用于连接建立或上传保护，不作为长期业务事实。真正需要长期保存和查询的数据仍以 MySQL 为准；删除图像、重试检测或清理对象时，BIZ 会同步删除相关 URL 缓存，避免前端继续拿到旧对象地址。对象存储设计与实现 系统使用 MinIO 保存项目缩略图、图像原图、图像缩略图、检测产物和项目报告源数据。数据库不保存二进制文件，只保存元数据、状态和必要的产物校验信息。前端上传图像时，BIZ 生成预签名 UPLOAD URL，前端将原图和缩略图直接上传到对象存储，再调用图像上报

接口确认状态。检测产物上传时，Reasoning 服务向 BIZ 获取受限前缀上传授权，在按项目、图像和任务组织的路径下上传动态数量产物。Reasoning 链路中比较容易出问题的是产物与结果不同步。模型可能已经给出了指标，但产物图像上传失败；也可能某个产物上传成功，却缺少可校验的SHA256。为避免页面出现“任务成功但看不到产物”的情况，Reasoning 服务在回调时会把产物名称、对象 Key、SHA256 和上传结果一并返回给 BIZ。BIZ 不会只根据模型是否运行结束来判断子任务成功，而是继续检查产物清单。对于需要产物的检测任务，只有全部必要产物都满足 uploaded=true，并且SHA256 非空时，BIZ 才会写入成功结果并把子任务置为成功。只要有一个必要产物缺失、上传失败或校验信息不完整，该子任务就按失败处理，成功结果也不会落库。后续用户可以通过重试重新执行检测并上传产物。对象存储和Redis 缓存也在这里配合使用：图像和检测产物可能被前端反复查看，系统会缓存部分预签名下载 URL；当图像被删除或脏对象被清理时，BIZ 同时清理对应缓存，避免已经失效的对象地址继续被使用。装 5.6 系统状态机设计与实现 本系统的流程不能只靠前端页面顺序来约束。用户看见的只是“上传图像”“开始检测”“确认复核”“生成报告”等按钮，但按钮背后还有分类路由、模型推理、产物上传、图像总结和报告生成等后台步骤。某一步提前执行、重复执行或被跳过，都会影响后续数据是否可信。因此，本文把这些过程放到后端状态机中管理。如前文图3.3 所示，系统中实际使用了项目工作流主状态机、图像资产构建状态机、检测主任务状态机、检测子任务状态机和评估报告生成状态机。这样做的目的不是把流程做复杂，而是让每个阶段都有可检查的进入条件和退出结果。项目主流程从项目初始化开始，依次推进到基础信息完成、图像资产完成、检测完成、复核完成和项目完成。每次阶段变化前，Core BIZ 都会重新读取数据库中的实际状态，而不是只接收前端提交的阶段值。比如，基础信息阶段要确认项目资料已经保存；图像资产阶段至少要有一张上传成功的图像；检测阶段要等待相关检测主任务全部结束；复核阶段要确认用户已经完成复核；报告阶段只有在报告生成成功后，项目才会被标记为完成。前端按钮主要用于提示用户当前可以做什么，最终是否允许推进，仍然由后端根据数据条件判断。检测主任务状态机对应的是单张图像从提交到总结完成的过程。主任务状态包括 pending、classifying、detecting、summarizing、succeeded 和 failed。任务刚创建时处于 pending，随后进入 classifying 调用分类 Agent。分类 Agent 返回任务代码后，BIZ 根据这些代码创建检测子任务。如果一张图像同时需要做多个检测，主任务会停留在 detecting，直到必要子任务全部结束。子任务全部成功后，系统再进入 summarizing 创建图像级总结；总结成功后，主任务才进入succeeded。分类、检测或总结任一必要环节失败时，主任务进入 failed，并保存失败原因，供前端展示和后续重试使用。检测子任务的状态相对简单，只包括 pending、succeeded 和 failed。需要注意的是，Reasoning 服务回调成功并不代表子任务一定成功，BIZ 还要继续检查产物完整性、上传状态和指标字段是否合法。评估报告生成也采用 pending、succeeded 和 failed三种状态。报告生成中，前端可以刷新查看当前进度；报告失败后，页面显示重试入口；报告成功后，则不再提供刷新或重试操作。借助这些状态记录，系统可以把多服务异步执行过程落到数据库中，后续查询、失败定位和重试处理都会更清楚。安全性设计与实现 本系统的安全性设计主要围绕几个真实使用中容易出现问题的位置展开：用户能不能被正确识别，项目数据会不会被越权访问，图像和检测产物的对象地址会不会长期暴露，长连接是否会绕过鉴权，以及 Agent 或模型服务返回的内容能不能直接影响数据库状态。由于系统涉及前端、Core API、Core BIZ、多个 Activity 服务、对象存储和消息队列，安全校验不能只放在某一个入口上，而是需要分散到请求接入、业务判断、对象授权和结果回调等环节中。用户登录采用 Access Token 与 Refresh Token 配合的方式。Access Token 是有效期较短的JWT，里面保存用户ID、邮箱、用户名、签发时间、过期时间、签发方和TI 1等信息，并使用 HMAC 签名算法 2 做完整性校验。Refresh Token 则用于换取新的访问令牌，它本身是随机生成的长期令牌，明文只返回给客户端，数据库只保存其哈希值和 Token ID。这样即使数据库中保存的记录被读取，也不能直接还原出客户端持有的 Refresh

Token。API 鉴权中间件主要校验Access Token。当前端发现访问令牌即将过期，或者业务 1 JTI 即JWT ID，是JWT 中用于标识令牌唯一编号的字段。系统可以利用该字段进行令牌追踪、吊销和黑名单校验。2 HMAC 是一种基于密钥的消息认证码算法。服务端使用密钥对消息生成签名，并在校验时重新计算签名，从而判断令牌内容是否被篡改。登录签发用户登录密码或邮箱验证码BIZ 校验身份失败计数与锁定签发Access Token生成Refresh TokenMySQL 保存 Refresh Token HashAPI 写入 HttpOnly Cookie访问校验前端请求API携带Access TokenAPI 调用Me校验JWTAccess Token有效BIZ 返回用户继续业务请求/auth/refresh读取CookieAccess Token过期或无效刷新轮换Redis 设置刷新锁防止并发刷新按Token ID + Hash查询Refresh Token轮换新令牌吊销旧Refresh Token返回新Access Token更新Cookie登录签发用户登录密码或邮箱验证码BIZ 校验身份失败计数与锁定签发Access Token生成Refresh TokenMySQL 保存 Refresh Token HashAPI 写入 HttpOnly Cookie访问校验前端请求API携带Access TokenAPI 调用Me校验JWTAccess Token有效BIZ 返回用户继续业务请求/auth/refresh读取CookieAccess Token过期或无效刷新轮换Redis 设置刷新锁防止并发刷新按Token ID + Hash查询Refresh Token轮换新令牌吊销旧Refresh Token返回新Access Token更新Cookie请求返回 401 时，再通过 /auth/refresh 使用HttpOnly Cookie 中的Refresh Token 换取新的令牌。这里没有把 Refresh Token 暴露给前端JavaScript 直接读取，一方面可以减少脚本窃取令牌的风险，另一方面也便于服务端统一控制刷新、吊销和过期处理。装图 5.9 用户登录鉴权与令牌刷新安全流程 如图5.9 所示，令牌刷新并不是在原令牌上直接延长有效期，而是采用轮换方式。BIZ 会先在Redis 中为同一个Refresh Token 设置短时刷新锁，避免浏览器并发请求同时触发刷新。刷新成功后，系统签发新的 Access Token 和Refresh Token，同时吊销旧的 Refresh Token。用户主动登出时，尚未过期的 Access Token 会按照JTI 写入Redis 黑名单，对应的 Refresh Token 也会被吊销。这样处理后，令牌即使在有效期内，也可以被服务端提前失效。项目权限没有放给前端自行判断。所有项目相关接口都会携带用户身份和项目ID，Core BIZ 根据数据库中的项目归属关系判断该用户是否可以读取、修改或推进项目阶段。前端隐藏按钮只能改善交互体验，不能作为权限依据。这样做的原因很直接：用户可以刷新页面、修改请求参数，甚至绕过页面直接调用接口，所以最终判断必须回到后端。对象存储访问也采用短时授权。图像原图、缩略图、检测产物和报告源文件都保存在 MinIO 中，前端并不持有 MinIO 的长期账号或密钥。上传图像时，BIZ 生成预签名 UPLOAD URL；查看图像或产物时，系统再生成预签名下载 URL；Reasoning 上传检测产物和 Summary 读取报告源数据时，也只拿到限定对象或限定前缀的临时授权。即使某个 URL 被复制出去，它的作用范围和可用时间也比较有限。密码和验证码的处理尽量避免保存可直接使用的敏感信息。注册和重置密码时，BIZ 会先检查密码长度和复杂度，再使用 Bcrypt 生成密码哈希写入数据库；登录时只比对哈希值，不保存明文密码。邮箱验证码按登录、注册和重置密码等场景分别生成，并保存在Redis 中。验证码通过后会消费，不能继续用于其他场景或再次提交，这样可以减少验证码复用带来的风险。WebSocket 连接没有直接把 Access Token 拼接到长连接地址中。前端需要先调用受鉴权保护的 /socket/ticket 接口，BIZ 校验用户身份和项目权限后，生成一个短时 Ticket。建立连接时，前端同时提交项目ID、Scope 和ticket，API 再向BIZ 校验票据是否有效。Scope 用于区分图像资

产、检测任务和报告状态等事件范围，避免用户打开某个页面时接收不属于当前阶段的全量消息。Agent 和模型服务的输出同样不能直接进入数据库。分类 Agent 返回的内容需要被装 解析成系统允许的任务代码，非法代码会被过滤或拒绝；图像总结和项目报告需要压 缩为合法JSON 对象后才能回调写入；项目报告还会检查图像组数量、图像数量等字段是否与源数据一致。Reasoning 服务启动时会检查模型文件，避免部署目录中只有 Git 订 LFS 指针文件而没有真实权重。上述处理不能完全消除外部服务的不确定性，但可以避免未经校验的文本、任务代码或产物状态直接改变业务事实。因此，本节的安全实现可以概括为两个思路：入口处先确认“是谁在访问、能不能 访问”，业务执行时再确认“访问的对象、返回的结果和推进的状态是否可信”。用户 线 身份、项目权限、对象授权、WebSocket Ticket 和Agent 输出校验分别处理不同风险点，最后都回到 Core BIZ 做关键判断。这样既不需要前端保存过多敏感信息，也能减少越权访问、凭证泄露和非受控输出对系统状态造成的影响。稳定性设计与实现 稳定性设计覆盖日志、异常处理、容量控制、超时控制、幂等重试、产物校验和清理机制等方面。系统的稳定性基本原则是：将长链路任务中的失败、超时和中间产物异常视为常态，通过可观测日志、受控并发、明确超时、状态机推进、事务一致性、失败重试和后台清理机制，使系统在局部异常发生时仍然能够定位问题、保留业务事实并恢复到可继续执行的状态。各服务按 HTTP、RPC、MQ、WS、CRON、CLS、RSN、SMR 等作用域范围输出结构化日志，并保留请求 ID、任务 UUID、图像 UUID、状态、耗时和错误信息。Docker Compose 中为五个后端服务配置日志采集 Sidecar，将容器日志按 PSM 写入 /root/logs/{psm}/log，并按文件大小和最大文件数量进行轮

转。实际排查问题时，按请求 ID、任务 UUID 或图像 UUID 追踪具体链路。在容量控制方面，Classification、Reasoning、Summary 和 Detection Worker 都有最大并发配置。Agent 调用和模型推理属于高耗时任务，如果不限制并发，容易压垮外部 Agent 平台、Python 模型进程、对象存储和数据库。通过并发上限和任务队列，系统在吞吐量与稳定性之间取得平衡。长任务中最常见的问题是等待时间过长或临时授权过期。因此，Agent 请求、Reasoning 总任务、产物下载、产物上传、图像上传待确认、Socket Ticket 和预签名URL 都装 设置了各自的 TTL 或 Timeout。Reasoning 服务没有在任务创建时就保存原图下载地址和上传地址，而是在真正开始执行时再向 BIZ 获取最新授权，这样可以减少队列等待造成的 URL 过期问题。对于上传中但迟迟没有上报完成的图像，上传超时 CronJob 会 订 把这类记录标记为失败，并把状态推送给前端，避免页面一直停留在“上传中”。状态更新集中放在BIZ 中处理。Classification、Reasoning 和 Summary 等Activity 服务完成任务后，只通过回调把结果交给BIZ，不直接修改数据库。BIZ 收到回调后，会 先检查任务是否还存在、当前状态是否允许被替换、产物清单是否完整，再在事务中写 线 入结果、推进状态并发布事件。Reasoning 产物的处理更严格一些，只有全部必要产物都上传成功，子任务才会被置为成功；如果产物缺失或对象不存在，即使模型已经返回指标，也不会写入成功结果。这样可以减少前端结果页出现缺图、空链接或对象不存在的情况。检测和报告都保留失败状态，并允许用户在失败后重新触发。检测任务失败时，前端可以重新启动失败任务；报告生成失败时，页面显示重试入口，由 BIZ 重新创建报告任务并调用 Summary 服务。对象存储侧还需要单独清理长期遗留文件。MinIO 脏对象清理CronJob 会扫描失败图像、失败子任务、失败报告以及没有数据库引用的对象文件夹，并在保护窗口之后再执行删除，避免正在上传或正在检测的对象被误删。通过这种方式，系统可以在失败后保留必要状态，同时控制对象存储持续膨胀。系统也提供了一些基础的可观测信息。工作台会统计项目数量、图像数量、图像组 数量、中间产物数量、检测执行次数、报告生成次数和对象存储容量。它能够让用户和维护者快速看到当前账号下的数据规模和存储占用。因此，系统稳定性并非只由单个服务的异常捕获实现，而是由状态机、事务、队列、超时、重试、日志、清理任务和容量控制共同支撑，使长链路Agent 任务在失败、超时或产物异常时仍能被定位、回滚、重试和恢复。部署设计与实现 本项目部署时使用Docker Compose 统一拉起各个服务。Compose 文件中除了Core Web、Core API 和Core BIZ 外，还包含 RocketMQ NameServer、Broker、Topic 初始化服务、三个 Activity 服务以及日志采集 Sidecar。各容器被放在同一个 Docker 网络中，服 装 务之间直接通过容器服务名访问，不需要在代码中写固定 IP。对外访问主要经过两个入口：Core API 暴露HTTP 接口，Core Web 由Nginx 托管前端静态资源并映射到 80 端口。Reasoning 服务比较特殊，它需要读取服务器上的模型权重，因此通过只读挂载方 式加载模型目录，避免模型文件被容器运行过程意外修改。服务器部署环境Docker Compose 服务网络：icw.docker.network浏览器用户HTTP / WS Core Web HTTPNginx: 80Core API HTTP / WS: 8000gRPCCore BIZ gRPC: 8001消费并广播Topic Init ICW_PROJECT_EVENT健康后初始化发布项目事件 RocketMQ Broker: 10911 路由发现 RocketMQ NameServer: 9876gRPCgRPCgRPCClassification gRPC: 8002Reasoning gRPC: 8003Summary gRPC: 8004Log Sidecar五个业务日志容器宿主机挂载与外部依赖/root/models只读模型目录注入运行参数MySQL / Redis / MinIO状态、缓存与对象环境变量 地址 TTL 并发/root/logs分 PSM 日志Docker Socket只读日志源服务器部署环境Docker Compose 服务网络：icw.docker.network浏览器用户 HTTP / WS Core Web HTTPNginx: 80Core API HTTP / WS: 8000gRPCCore BIZ gRPC: 8001消费并广播Topic Init ICW_PROJECT_EVENT健康后初始化发布项目事件RocketMQ Broker: 10911 路由发现 RocketMQ NameServer: 9876gRPCgRPCgRPCClassification gRPC: 8002Reasoning gRPC: 8003Summary gRPC: 8004Log Sidecar五个业务日志容器宿主机挂载与外部依赖/root/models只读模型目录注入运行参数MySQL / Redis / MinIO状态、缓存与对象环境变量 地址 TTL 并发/root/logs分 PSM 日志Docker Socket只读日志源订 线图 5.10 系统部署视图 如图5.10 所示，部署视图可以分为三部分：对外入口由 Core Web 和Core API 承担，内部服务通过Docker bridge 网络按服务名互访，模型目录、日志目录、Docker Socket 和数据服务则作为宿主机或外部依赖接入容器运行环境。该部署方式既保持了服务间通 信地址的稳定性，也把模型权重、日志文件和运行参数从镜像中拆分出来，便于后续更新和排查。服务启动顺序主要依靠 depends_on 和健康检查控制。RocketMQ Broker 先启动并 通过健康检查后，Topic 初始化服务再创建项目事件需要使用的 Topic。Classification、Reasoning、Summary、Core BIZ 和Core API 都会等待 Topic 初始化完成后再启动，避免服务刚启动就因为消息队列不可用而报错。Core BIZ 还会等待三个 Activity 服务启动，因为检测 Worker 后续需要调用这些服务。运行参数没有硬编码在程序中，而是集中放到环境变量里。服务地址、MySQL、Redis、MinIO、RocketMQ、JWT、邮箱验证码 Secret、Socket Ticket、对象存储 TTL、Agent Token、Agent Bot ID、任务超时和并发数量等都可以通过环境变量调整。本地调 装 试和服务器部署只需要更换配置值，不必修改代码重新编译，

也便于后续在其他机器 上复现实验环境。Reasoning 模型文件通过/root/models:/app/models:ro 方式挂载进Docker 镜像。订 这样可以避免模型权重导致镜像体积过大，也便于更新模型文件。服务启动时会检查模型文件是否存在且不是 Git LFS 指针，提前暴露部署问题。日志部署采用两层机制。容器自身使用 Docker json-file 驱动进行基础轮转，日 志采集 Sidecar 通过 Docker Socket 读取目标容器日志，并按 PSM 写入宿主机目录。该 线 方案比在每个业务容器中接入复

杂日志组件更轻量，适合当前项目规模，也便于服务器直接查看各服务日志。本章小结 本章主要说明第4章中的技术方案如何落到具体工程实现中。实现时，系统被拆分为Core Web、Core API、Core BIZ、Activity Classification、Activity Reasoning、Activity Summary 和 ICW Common 七个服务与模块。这样的拆分不是单纯为了增加服务数量，而是为了让前端交互、业务状态、Agent 调用、模型推理、报告生成和共享协议各自有明确位置。Core BIZ 负责控制任务状态和数据库事务，Activity 服务只返回结构化结果，Core API 负责 HTTP 接入、鉴权和实时推送，Core Web 负责阶段化展示、检测结果查看和人工复核。从实现过程看，难点主要不在于某一个单独接口，而在于系统的长链路协同。一次检测任务会经过HTTP 请求、gRPC 调用、Worker 入队、分类 Agent、子任务创建、Python模型执行、MinIO 产物上传、回调落库、RocketMQ 事件、WebSocket 推送和前端状态合并。项目报告任务还要在此基础上组装项目级源数据，保存多源检测结构化结果，调用附件式 Agent，校验报告 JSON，并把报告内容展示给用户。为支撑这些流程，系统在协议、状态机、对象存储、CronJob 定时任务、日志、并发控制和 Docker Compose 部署上都做了相应实现，使 Agent 能力转化为可运行、可追踪、可恢复的软件系统。装订线 系统验证与应用分析 前文已经从研究背景、需求分析、技术方案和工程实现四个层面说明了本系统的设计与实现逻辑，本章在上述基础上对系统进行验证与应用分析。本章的验证目标是本系统是否能够将 Agent 自主任务调度、原子检测能力、人工复核、LLM 分层信息整合和工程状态机连接为可运行的软件闭环。换言之，本章关注的是系统是否真正完成了“从图像资产到韧性评估报告”的工程链路，是否能够支撑实际幕墙数字运维场景中的使用需求。验证目标 装 系统验证围绕三条主线展开。第一条主线是项目级图像资产管理，即系统是否能够支持项目创建、基础信息维护、图像组组织、批量图像上传、图像状态追踪和对对象产物管理。第二条主线是 Agent 自主任务调度，即分类 Agent 的语义判断是否能够转化为受控任务代码，并进一步驱动后续原子检测任务创建、执行和状态推进。第三条主线是订 LLM 分层信息整合，即单图检测结果是否能够生成图像级总结，项目级多源信息是否能够进一步汇总为可阅读、可复核的韧性评估报告。围绕上述目标，验证工作分为功能验证、Agent 调度方法验证、LLM 分层整合策略线 验证、状态一致性验证、部署验证和验证结果展示六个方面。功能验证关注用户能否完成完整流程；Agent 调度方法验证关注分类输出是否真实影响检测任务；LLM 分层整合 策略验证关注报告生成是否建立在结构化源数据和图像级总结之上；状态一致性验证关注数据库、后端任务、消息事件和前端展示是否保持一致；部署验证关注 Docker 环境下多服务是否能够正常协同；验证结果展示则用于集中呈现上述验证结果、系统运行界面和软件效果。功能验证 功能验证按照一次完整使用过程展开，而不是只检查页面能否正常打开。验证时从注册登录进入系统，随后创建项目、填写项目基础信息、维护图像组、上传幕墙图像，并继续执行批量操作、Agent 智能检测、检测结果查看、人工复核、项目报告生成和报告打印等操作。这样安排的原因在于，本系统的核心并不是某一个单独页面，而是第3章中定义的阶段化业务流程是否能连续走通，并且每个阶段都能留下可查询、可恢复的状态记录。实际操作中，项目基础信息可以保存建筑名称、建筑位置、建成年份、建筑描述、已知问题和评估目标等内容，进入下一阶段前也能完成自动提交。图像资产阶段能够完成图像组创建、图像上传、图像移动、图像删除、原图查看和批量操作，项目、图像组、图像元数据以及对象存储路径之间能够形成明确关联。进入 Agent 智能检测阶段后，系统会为图像创建检测主任务，并交给后台异步流程处理；前端可以看到图像级进度、节点级状态、任务 ID、失败状态和重试入口。检测结束后，页面能够展示图像信息、各子任务指标、检测产物、图像级Agent 总结和人工复核区域。用户提交的复核结果会保存到数据库中，后续生成项目报告时也会被纳入上下文。装 从这一组流程可以看出，系统已经能够完成从项目资料维护到项目报告展示的端到端操作。一次检测任务背后涉及任务创建、Agent 自主任务调度、原子检测执行、产物上传、回调落库、RocketMQ 事件、WebSocket 推送和前端状态合并等多个环节。上订述 环节能够连续运行，说明系统已经具备从用户操作到后台任务、再回到

前端反馈的基本闭环。Agent 自主任务调度方法验证 线 Agent 自主任务调度方法的验证，重点放在分类 Agent 的输出能否真正改变后续检测路径上。根据第 4.5 节的设计，分类 Agent 不负责直接说明“这张图有什么问题”，而是输出受控任务代码集合；这些代码再由 BIZ 服务解析，并转换为后续检测流程中的任务执行计划。因此，本节主要检查分类输出、任务代码解析、任务代码治理、主任务执行标记、检测子任务创建以及空任务集合处理是否符合预期。在验证过程中，分类 Agent 的输出能够被系统稳定转换为检测任务。输出 crack 和 stain 时，系统创建石材裂缝和石材污渍子任务；输出 flatness 和 spalling 时，系统创建玻璃平整度和玻璃爆裂子任务；输出 corrosion 时，系统创建金属锈蚀子任务。对于空任务列表，系统不会继续创建检测子任务和图像级总结任务，而是直接把主任务推进到成功状态。这说明分类 Agent 的判断并不是页面上的提示文本，而是实际参与了后端任务编排。服务端治理逻辑也在这一部分得到验证。分类输出需要先被解析为合法 JSON 对象，任务代码必须落在系统定义的固定集合内；非法任务代码会被拦截，重复任务代码会被去重，最终任务集合还会按照固定顺序归一化。经过这些处理后，Agent 的开放语义判断被约束为工程系统可以消费的有限任务集合，也验证了本文把 Agent 设计为“受控任务规划节点”的思路。LLM 分层信息整合策略验证 LLM 分层信息整合策略主要验证两件事：一是单张图像的检测结果能否被整理成图像级总结，二是多个图像、图像组和人工复核意见能否继续汇总成项目级报告。按照第 4.7 节的设计，系统没有把所有底层区域 JSON 一次性塞给项目报告 Agent，而是装 先在图像级完成信息压缩和语义整理，再把项目基础信息、图像组、图像数量、检测任务、图像级总结、人工复核意见和报告源数据组织成项目报告上下文。原子检测完成后，系统可以把结构化检测结果、执行任务代码、图像元数据和检测产物信息整理为图像级总结输入。Summary 服务生成总结后，会通过回调交给 BIZ 保存。到了项目报告阶段，系统会进一步汇总项目级多源信息，生成报告源数据，并把这份上下文保存到对象存储中，供项目报告 Agent 以附件方式读取。项目报告 Agent 返回结果后，BIZ 会对固定JSON 结构进行压缩、解析和字段校验，校验通过后再写入数据库。从验证结果看，图像级总结主要回答“这一张图像说明什么”，项目级报告则进一步回答“整个项目说明什么”。前者把底层指标和产物信息先整理成较短的图像语义，减少项目报告Agent 直接处理大量区域JSON 的压力；后者再围绕跨图像缺陷分布、区域风险、整体韧性和维护建议形成项目级结论。该过程说明，本文提出的分层整合方式能够把单图检测结果逐步过渡到建筑级韧性评估结论。状态一致性验证 状态一致性验证关注的是数据库、后台 Worker、RocketMQ 项目事件、WebSocket实时消息和前端页面之间是否能对上。系统中并不是只有一个简单的“完成/未完成”字段，而是通过项目阶段状态、图像上传状态、检测主任务状态、检测子任务状态、图像总结状态和报告生成状态共同描述业务进度。页面刷新时，前端通过查询接口恢复数据库中的事实状态；实时运行时，前端再通过 WebSocket 接收增量事件。验证时对几种常见状态流转进行了检查。分类失败后，主任务能够进入失败状态；子任务失败后，主任务最终也会进入失败状态；所有必要子任务成功后，主任务进入图像总结阶段；图像总结成功后，主任务才被标记为成功。报告生成失败时，前端能够显示失败重试入口；项目阶段推进时，后端会检查前置阶段条件。由此可以看出，系统最终仍以数据库状态为准，WebSocket 只是加快前端更新速度。即使消息出现延迟，或者用户刷新页面，前端也能通过列表接口重新拿到当前状态。产物一致性也属于状态一致性的一部分。

Reasoning 回调成功并不意味着 BIZ 立即 把子任务写成成功状态，系统还会继续检查产物清单、上传状态和 SHA256 信息。只有必要产物全部上传成功，子任务才会进入成功状态。该验证对应第 5.6 节和第 5.8 节中 装 的设计，可以避免“数据库显示成功，但对对象存储中缺少产物”的异常情况。部署验证 部署验证在 Docker Compose 环境下进行，重点检查系统离开单机脚本后是否还能 订 作为多服务软件正常运行。验证内容包括服务启动、依赖编排、环境变量配置、RocketMQ Topic 初始化、gRPC 地址配置、对象存储访问、模型目录挂载、日志采集和 WebSocket 消息推送。参与部署的服务包括 RocketMQ NameServer 和 Broker、Classification、Reasoning、线 Summary、BIZ、API 和 Web，基础设施则包括 MySQL、Redis、MinIO、RocketMQ 和日志 Sidecar 等。验证过程中，系统能够在容器环境下完成多服务启动、链路调用和页面访问。RocketMQ Topic 会在业务服务启动前完成初始化，API 服务能够消费项目事件；Reasoning 服务通过挂载目录读取模型权重和图像处理脚本，模型更新不需要重新打包到镜像中；BIZ 和 API 可以通过环境变量连接 MySQL、Redis、MinIO 和 RocketMQ。日志采集、对象存储授权和服务依赖顺序也能够放到 Docker Compose 中统一管理，部署边界清晰。验证结果展示 本节集中展示软件运行截图，用来对应前面各项验证在软件原型系统中的实际呈现。界面截图包括登录与工作台、项目基础信息、图像资产构建、Agent 智能检测、检测结果弹窗、人工复核确认和

评估报告生成等界面。功能验证结果展示 装 订 图 6.1 用户登录界面 线 图 6.2 工作台数据概览界面 装 图 6.3 项目管理界面 订 线 图 6.4 项目基础信息界面 装 图 6.5 图像资产构建界面 订 6.7.2 Agent 自主任务调度方法验证结果 线 图 6.6 Agent 智能检测界面 装 图 6.7 金属锈蚀检测结果界面 订 线 图 6.8 石材裂缝检测结果界面 装 图 6.9 石材污渍检测结果界面 订 线 图 6.10 玻璃平整度检测结果界面 装 图 6.11 玻璃爆裂检测结果界面 订 6.7.3 LLM 分层信息整合策略验证结果 线 图 6.12 图像级 Agent 总结与人工复核界面 装 图 6.13 项目韧性评估报告界面（一） 订 线 图 6.14 项目韧性评估报告界面（二） 6.7.4 状态一致性验证结果 状态一致性验证结果主要体现在检测进度、检测子任务状态、图像级总结状态、人工复核结果和报告生成状态的连续展示中。上述界面中，图 6.6 展示了 Agent 智能检测阶段的任务进度和节点状态，图 6.12 展示了图像级总结与人工复核状态，图 6.13 和图 6.14 展示了项目报告生成后的前端结果。页面刷新和 WebSocket 增量更新均以数据库状态为准，因此这些界面共同体现了系统的状态一致性。6.7.5 部署验证结果 部署验证结果主要体现为 Docker Compose 环境下各服务能够完成启动、依赖初始化、服务名互访、对象存储访问、消息事件消费和前端页面访问。前述登录、工作台、装 项目管理、检测和报告界面均来自部署后的系统运行结果，说明 Core Web、Core API、Core BIZ、Activity 服务、RocketMQ、对象存储和日志采集等部署单元能够协同工作。从这些界面可以看到，系统已经不只是一个模型调用入口，而是形成了项目级评估 订 软件的基本形态。用户可以从工作台进入项目，维护项目资料，上传并组织图像，启动 Agent 智能检测，查看检测进度和检测结果，完成复核，最后生成项目级韧性评估报告。6.8 应用价值分析 线 表 6.1 不同实现方案对比 方案 主要特点 适用范围 本文系统体现 人工巡检 专家经验强，判断灵活 现场核验、经验判断和最终处 置决策 单模型检测 对单类缺陷识别直接有效 单张图像、单类缺陷和局部结 果判断 端到端大模型报告 自然语言生成能力强 文本整理、语义归纳和报告表 达 用系统管理图像与流程，辅助形成结构化评估依据 将模型封装为 Agent 可调度的原子检测能力 采用 LLM 分层信息整合策略和结构化落库 本文方案 可调度、可追踪、可复核、可报告 项目级图像资产管理和建筑级韧性评估 通过工程链路形成从图像资产到评估报告的软件闭环 如表 6.1 所示，本文系统将人工巡检经验、原子检测能力和大模型文本生成能力组织到同一条工程链路中。相比只关注单张图像的检测工具，系统进一步形成了项目、图像、检测、总结、复核和报告之间的连续关系；相比只生成文本的报告工具，系统保留了结构化证据、任务状态和产物追踪。结合实际部署与试用情况，系统已经能够支持建筑幕墙巡检图像的统一管理、智能检测、人工复核和报告生成，为建筑幕墙数字运维提 供了可实施的软件原型。从应用角度看，本系统的价值主要体现在幕墙巡检、缺陷检测和报告生成三个环节。对于幕墙巡检中数量较多、来源较分散的图像，系统能够按项目和图像组进行管理，后续查找、复核和区域化分析都会更方便。Agent 自主任务调度方法则减少了人工判断检测能力的步骤，系统可以根据图像内容选择合适的检测任务，而不是让用户逐项手动指定。检测结束后，LLM 分层信息整合策略把底层指标、图像级总结和复核意见逐步整理为项目报告，能够减轻人工汇总报告的工作量。实时进度和结果可视化也让用户更容易理解长任务正在执行到哪一步，失败时也能知道需要从哪里重新处理。单一检测工具通常只能回答某张图像是否存在某类缺陷，而本文系统更关注运维 场景中的连续流程：项目如何建立，图像如何组织，检测任务如何执行，结果如何复核，装 报告又如何生成。与纯 Agent 应用相比，本系统没有让 Agent 直接决定数据库状态，而是通过任务代码、状态机和接口协议进行约束。这样既保留了 Agent 在语义判断和信息整理上的作用，也使系统更接近真实软件系统的运行方式。订 6.9 本章小结 本章围绕系统功能、Agent 自主任务调度方法、LLM 分层信息整合策略、状态一致性和部署过程对系统进行了验证。验证结果说明，系统能够从项目级图像资产出发，经 线 过智能检测、人工复核和报告生成，形成建筑幕墙韧性评估报告。分类 Agent 的输出会真实改变后续检测任务，图像级总结和项目级报告也能够结构化数据基础上逐步生成。与此同时，状态机、消息队列、WebSocket 和 Docker Compose 部署共同支撑稳定的工程链路。通过本章验证，可以说明该软件原型系统能够支撑从项目级图像资产到建筑韧性评估报告的完整业务流程。总结与展望 本章对全文工作进行总结，概括本文在 Agent 自主任务调度方法、LLM 分层信息整合策略和工程系统实现方面取得的主要成果，并在此基础上展望系统后续可以继续拓展的方向。全文总结 本文围绕建筑幕墙韧性评估中的现实痛点，设计并实现了一套基于 AI Agent 的建筑幕墙韧性评估软件系统。有别于围绕单个检测模型展开的研究路线，本文重点解决现实工程中更关键的链路问题：如何把项目级图像资产、多类原子检测能力、人工复核信 息和项目级报告生成组织为一个完整、可追踪、可部署的软件原型系统，并通过 Agent 提升任务

调度和信息整合能力。论文前半部分先从幕墙巡检的实际工作出发，分析人工经验流程在效率、图像管 理和报告汇总上的不足。现有单点检测方法能够处理局部缺陷，但放到项目级评估中，订 还需要解决图像如何组织、检测任务如何选择、多个结果如何汇总成建筑级报告等问题。围绕这些问题，本文梳理了建筑立面检测、计算机视觉和 AI Agent 相关研究，并将 Agent 工具调用和多阶段工作流引入幕墙评估场景。在系统设计阶段，本文提出 Agent 线 自主任务调度方法和 LLM 分层信息整合策略，把分类 Agent 放在检测任务创建之前，把图像级总结 Agent 放在单图检测之后，再由项目级报告 Agent 生成最终评估报告。在工程实现阶段，本文完成了 Web 前端、Core API、Core BIZ、Classification、Reasoning、Summary、ICW Common 以及 MySQL、Redis、MinIO、RocketMQ、Docker Compose 配置等内容，并通过系统验证说明该软件原型系统能够支撑从项目级图像资产到建筑韧性评估报告的完整业务流程。主要成果 本文取得的主要成果包括以下三个方面。第一，提出并实现了面向多材质幕墙图像的 Agent 自主任务调度方法，使系统能够根据图像内容动态决定需要执行的检测能力。该方法将分类 Agent 的语义判断约束为有限任务代码集合，并由 Core BIZ 统

一转换为检测主任务和检测子任务，从而把开放的图像理解能力转化为可治理、可追踪的工程任务调度过程。该成果解决了多材质幕墙图像中“哪些图像需要执行哪些检测能力”的问题，使检测流程更符合幕墙材料差异和实际业务需求。第二，提出并实现了 LLM 分层信息整合策略。系统先保存原子检测结构化结果，再生成图像级总结，最后生成项目级报告。该策略将“单张图像说明什么”和“整个项目说明什么”拆分为不同语义层次，将大语言模型从简单文本生成工具扩展为“路由、压缩、汇总”的智能节点，减少了多图像、多任务、多区域信息直接汇总输入 LLM 模型带来的上下文过载和重点淹没问题，使报告生成过程更稳定、更可解释，也更符合土木工程领域用户从局部缺陷到整体评估的认知路径。第三，构建了完整工程闭环。系统通过微服务架构、gRPC 协议、异步 Worker、回装调接口、数据库状态机、对象存储、消息队列和 WebSocket 推送，使 Agent 能力从一次性调用变成可运行、可追踪、可恢复的业务流程。前端页面能够展示项目状态、图像资产、检测进度、检测产物、人工复核和项目报告，后端服务能够完成任务创建、状态推进、产物校验、失败重试和部署运行。本成果体现了本系统作为软件工程专业毕业设计的工作完整性。未来工作展望线基于本文已经完成的软件原型系统，未来工作可以从以下三个方面继续推进。第一，可以进一步扩充建筑幕墙真实场景数据集，对金属锈蚀、石材裂缝、石材污渍、玻璃平整度和玻璃爆裂五类原子检测能力进行更系统的精度评估和模型优化。本文重点在于系统链路和 Agent 工程化方法，原子检测能力主要作为工具化能力接入系统。未来若能够结合更多真实建筑、不同拍摄角度、不同光照环境和不同幕墙材料的数据进行模型训练与评估，系统的检测结果将具有更强的工程适应性。第二，可以继续增强系统工程能力。本文已经完成了容器化部署、日志采集、异步任务、消息事件、WebSocket 推送和失败重试等基础能力，后续可进一步加入分布式 Worker、任务优先级调度、模型版本管理、监控告警机制和更细粒度的权限控制。这些能力有助于系统在软件原型系统的基础上进一步走向更稳定的工程化应用。第三，可以探索系统与 BIM 平台、无人机巡检平台和建筑运维系统的对接。建筑幕墙韧性评估不仅需要识别图像中的缺陷，还需要进一步理解缺陷在建筑空间中的位置、发展趋势和维护优先级。未来若能够将图像检测结果映射到 BIM 构件、立面区域和历史维修记录中，系统将能够从图像级评估进一步扩展到建筑全生命周期的数字运维与智能管理。综上所述，本文验证了 AI Agent 在建筑幕墙韧性评估中的应用价值。Agent 不是替代工程系统，而是通过自主任务调度和分层信息整合能力补足传统工程系统在语义判断和报告生成方面的不足。本文工作为建筑幕墙智能运维提供了一条可落地的技术路线，也为 AI Agent 在垂直工程软件中的应用提供

了参考。装订线参考文献 中华人民共和国住房和城乡建设部, 中华人民共和国国家质量监督检验检疫总局. 建筑信息模型应用统一标准: GB/T 51212-2016[S]. 北京: 中国建筑工业出版社, 2016. 史晨辰, 朱小平, 王辰星, 等. 韧性城市研究综述——基于城市复杂系统视角[J/OL]. 生态学报, 2023, 43(4): 1726-1737.

<https://www.ecologica.cn/html/2023/4/stxb202203230723.htm>. DOI: 10.5846/stxb202203230723. YAO S, ZHAO J, YU D, et al. ReAct: Synergizing Reasoning and Acting in Language Models[J/OL]. 2023. arXiv: 2210.03629 [cs.CL]. <https://arxiv.org/abs/2210.03629>. DOI: 10.48550/arXiv.2210.03629. SCHICK T, DWIVEDI-YU J, DESSÌ R, et al. Toolformer: Language Models Can Teach Themselves to Use Tools[J/OL]. 2023. arXiv: 2302.04761 [cs.CL]. <https://arxiv.org/abs/2302.04761>. DOI: 10.48550/arXiv.2302.04761. WU Q, BANSAL G, ZHANG J, et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation[J/OL]. 2023. arXiv: 2308.08155 [cs.AI]. <https://arxiv.org/abs/2308.08155>. DOI: 10.48550/arXiv.2308.08155. FREIMUTH H, KÖNIG M. Planning and executing construction inspections with un-manned aerial vehicles[J/OL]. Automation in Construction, 2018, 96: 540-553. <https://doi.org/10.1016/j.autcon.2018.10.016>. DOI: 10.1016/j.autcon.2018.10.016. LI Q, PENG X, ZHONG X, et al. Quantitative identification of debonding defects in building façades based on UAV-thermography using a two-stage network integrating dual attention mechanism[J/OL]. Infrared Physics & Technology, 2024, 138: 105241. <https://doi.org/10.1016/j.infrared.2024.105241>. DOI: 10.1016/j.infrared.2024.105241. ZHANG C, WANG F, ZOU Y, et al. Automated UAV image-to-BIM registration for building façade inspection using improved generalised Hough transform[J/OL]. Automation in Construction, 2023, 153: 104957. <https://doi.org/10.1016/j.autcon.2023.104957>. DOI: 10.1016/j.autcon.2023.104957. 冷烁, 胡振中. 基于 BIM 的人工智能方法综述[J/OL]. 图学学报, 2018, 39(5): 797-805. <https://cdn.sciengine.com/doi/pdf/93AD401571CF4A7AB125385E72BA5D84>. DOI: 10.11996/j.gj.2095-302X.2018050797. CANNY J. A Computational Approach to Edge Detection[J/OL]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 1986, PAMI-8(6): 679-698. <https://doi.org/10.1109/TPAMI.1986.4767851>. DOI: 10.1109/TPAMI.1986.4767851. DUDA R O, HART P E. Use of the Hough transformation to detect lines and curves in pictures[J/OL]. Communications of the ACM, 1972, 15(1): 11-15. <https://doi.org/10.1145/361237.361242>. DOI: 10.1145/361237.361242. LECUN Y, BENGIO Y, HINTON G. Deep Learning[J/OL]. Nature, 2015, 521(7553): 436-444. <https://doi.org/10.1038/nature14539>. DOI: 10.1038/nature14539. [13] HE K, ZHANG X, REN S, et al. Deep Residual Learning for Image Recognition[C/OL] //2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2016: 770-778. <https://doi.org/10.1109/CVPR.2016.90>. DOI: 10.1109/CVPR.2016.90. [14] FAN L L, ZHAO H W, ZHAO H Y, et al. 基于深度卷积神经网络的目标检测研究综述[J/OL]. Guangxue Jingmi Gongcheng/Optics and Precision Engineering, 2020, 28(5): 1152-1164. <https://ojs.lightpublishing.cn/zh/article/doi/10.3788/OPE.20202805.1152>. DOI: 10.3788/OPE.20202805.1152. REDMON J, DIVVALA S, GIRSHICK R, et al. You Only Look Once: Unified, Real-Time Object Detection[C/OL] //2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2016: 779-788. <https://doi.org/10.1109/CVPR.2016.91>. DOI: 10.1109/CVPR.2016.91. 田萱, 王亮, 丁琪. 基于深度学习的图像语义分割方法综述[J/OL]. 软件学报, 2019, 30(2): 440-468. <https://www.jos.org.cn/jos/article/abstract/5659>. DOI: 10.13328/j.cnki.jos.005659. CHEN L C, ZHU Y, PAPANDREOU G, et al. Encoder-Decoder with Atrous Separable Convolution for Semantic Image Segmentation[C/OL] //2018. arXiv: 1802.02611 [cs.CV]. <https://arxiv.org/abs/1802.02611>. DOI: 10.48550/arXiv.1802.02611. 周丽娟, 毛嘉宁. 视觉 Transformer 识别任务研究综述[J/OL]. 中国图象图形学报, 2023, 28(10): 2969-3003. <https://www.cjig.cn/zh/article/doi/10.11834/jig.220895>. DOI: 10.11834/jig.220895. XIE

E, WANG W, YU Z, et al. SegFormer: Simple and Efficient Design for Se-mantic Segmentation with Transformers[C/OL]//RANZATO M, BEYSELZIMER A, DAUPHIN Y, et al. Advances in Neural Information Processing Systems: vol. 34. Curran 装 Associates, Inc., 2021: 12077-12090. <https://arxiv.org/abs/2105.15203>. DOI: 10.48550/arXiv.2105.15203. 苏丽, 孙雨鑫, 苑守正. 基于深度学习的实例分割研究综述[J/OL]. 智能系统学报, 2021, 17(1): 16-31. <https://tis.hrbeu.edu.cn/oa/darticle.aspx?id=202109043>. DOI: 10.11992/tis.202109043. KIRILLOV A, MINTUN E, RAVI N, et al. Segment Anything[C/OL]//Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV). 2023: 4015-4026. 线 <https://arxiv.org/abs/2304.02643>. DOI: 10.48550/arXiv.2304.02643. YE X W, JIN T, CHEN P Y. Structural crack detection using deep learning-based fully convolutional networks[J/OL]. advances in structural engineering, 2019, 22(16): 3412-3419. <https://doi.org/10.1177/1369433219836292>. DOI: 10.1177/1369433219836292. MEI H, YANG X, WANG Y, et al. Don't Hit Me! Glass Detection in Real-World Scenes[C/OL]//2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2020: 3684-3693. <https://doi.org/10.1109/CVPR42600.2020.00374>. DOI: 10.1109/CVPR42600.2020.00374. SANDLER M, HOWARD A, ZHU M, et al. MobileNetV2: Inverted Residuals and Linear Bottlenecks[C/OL]//2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2018: 4510-4520. <https://doi.org/10.1109/CVPR.2018.00474>. DOI: 10.1109/CVPR.2018.00474. XI Z, CHEN W, GUO X, et al. The Rise and Potential of Large Language Model Based Agents: A Survey[J/OL]. 2023. arXiv: 2309.07864 [cs.AI]. <https://arxiv.org/abs/2309.07864>. DOI: 10.48550/arXiv.2309.07864. HUANG L, YU W, MA W, et al. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions[J/OL]. ACM Transactions on Information Systems, 2025, 43(2): 1-55. <https://doi.org/10.1145/3703155>. DOI: 10.1145/3703155. QIN Y, LIANG S, YE Y, et al. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs[J/OL]. 2023. arXiv: 2307.16789 [cs.AI]. <https://arxiv.org/abs/2307.16789>. DOI: 10.48550/arXiv.2307.16789. SCHOLAK T, SCHUCHER N, BAH DANAU D. PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models[J/OL]. 2021. arXiv: 2109.05093 [cs.CL]. <https://arxiv.org/abs/2109.05093>. DOI: 10.48550/arXiv.2109.05093. WILLARD B T, LOUF R. Efficient Guided Generation for Large Language Models 线 [J/OL]. 2023. arXiv: 2307.09702 [cs.CL]. <https://arxiv.org/abs/2307.09702>. DOI: 10.48550/arXiv.2307.09702. LEWIS P, PEREZ E, PIKTUS A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[J/OL]. 2021. arXiv: 2005.11401 [cs.CL]. <https://arxiv.org/abs/2005.11401>. DOI: 10.48550/arXiv.2005.11401. 附录 附录从工程视角补充说明系统涉及的 PSM、接口清单与环境变量配置。 A. 系统模块与 PSM 划分 本附录归档系统模块与 PSM 划分。 PSM 划分覆盖 Web 前端、后端业务服务、Agent 与推理能力服务、消息队列、日志采集、定时任务、共享协议模块与外部基础设施等, 便于从部署和运行维护角度理解系统边界。 表 A.1 系统模块与 PSM 划分 PSM 服务名称 职责 装 icw.core.web Core Web 前端服务 提供项目操作、图像管理、检测复核和报告展示界面 icw.core.api Core API 网关服务 提供 HTTP 与 WebSocket 接入, 处理鉴权、路由、响应和事件转发 订 icw.core.biz Core BIZ 业务服务 负责项目编排、状态推进、数据存储、任务调度和事件发布 icw.activity.classification 图像分类 Agent 服务 识别幕墙材质, 生成原子检测任务代码 icw.activity.reasoning 原子检测推理服务 执行五类原子检测, 上传产物并回调结果 icw.activity.summary 总结 Agent 服务 生成图像级总结与项目级韧性评估报告 线 icw.common 共享协议与通用模块 提供跨服务 Proto 协议、枚举、错误、日志和 JSON 工具等 icw.cron.pending_image_timeout 上传中图像超时任务 扫描上传超时图像并置为失败 icw.cron.minio_dirty_object_cleanup MinIO 脏对象清理任务 清理失败或失联对象, 保留三十分窗口 icw.rocketmq.namesrv RocketMQ NameServer 提供 RocketMQ 路由发现能力 icw.rocketmq.broker RocketMQ Broker 存储并投递项目事件消息 icw.rocketmq.init RocketMQ 初始化任务 初始化项目事件 Topic icw.log.core_api Core API 日志采集服务 采集 Core API 容器日志并独立归档轮转 icw.log.core_biz Core BIZ 日志采集服务 采集 Core BIZ 容器日志并独立归档轮转 icw.log.activity_classification Classification 日志采集服务 采集 Classification 容器日志并独立归档 icw.log.activity_reasoning Reasoning 日志采集服务 采集 Reasoning 容器日志并独立归档 icw.log.activity_summary Summary 日志采集服务 采集 Summary 容器日志并独立归档 icw.docker.logging Docker 日志轮转策略 配置容器日志文件大小和保留数量 icw.docker.network Docker Compose 网络 连接 Web、后端服务、消息队列和日志等 续下页 续表 A.1 PSM 服务名称 职责 icw.data.mysql MySQL 数据库 存储用户、项目、图像、任务、复核和报告等结构化数据 icw.data.redis Redis 缓存 存储验证码、令牌、连接票据和短期缓存 icw.data.minio MinIO 对象存储 存储图像、检测产物和报告源文件 B. HTTP 与 RPC 接口清单 本附录按照运行时 PSM 对 HTTP 与 RPC 接口进行归档, Core API 直接对前端暴露 HTTP 与 WebSocket 接口, 其余服务主要通过 gRPC 进行服务间调用。 装 B.1 icw.core.api 服务接口清单 表 B.1 icw.core.api HTTP 接口清单 方法 API 路径 接口说明 订 POST /auth/login 登录 POST /auth/logout 登出 GET /auth/me 获取当前登录用户信息 POST /auth/refresh 刷新访问令牌 线 POST /auth/register 注册账号 POST /auth/reset-password 重置密码 POST /auth/send-email-code 发送邮箱验证码 POST /project/core/advance 推进项目流程阶段 POST /project/core/create 创建项目 POST /project/core/delete 删除项目 GET /project/core/dashboard 获取账号维度工作台统计 GET /project/core/list 获取项目列表 DELETE /project/profile/thumbnail 删除项目缩略图 GET /project/profile/detail 获取项目基础信息 GET /project/profile/thumbnail 获取项目缩略图 POST /project/profile/update 更新项目基础信息 POST /project/profile/thumbnail 上传项目缩略图 GET /project/assets/list 获取项目图像资产列表 续下页 续表 B.1 方法 API 路径 接口说明 POST /project/assets/group/create 创建图像组 POST /project/assets/group/delete 删除图像组 POST /project/assets/group/move 移动图像组 POST /project/assets/group/update 更新图像组 POST /project/assets/image/delete 删除图像 GET /project/assets/image/original 获取项目原图访问地址 POST /project/assets/image/move 移动图像到指定图像组 POST /project/assets/image/report 上报图像上传结果 POST /project/assets/image/upload 申请图像上传策略 GET /project/detection/result 获取单张图像检测结果 装 GET /project/detection/list 获取项目检测任务列表 POST /project/detection/retry 重试项目智能检测 POST

/project/detection/start 启动项目智能检测 GET /project/review/detail 获取图像检测人工复核信息 POST
/project/review/update 更新图像检测人工复核信息 订 GET /project/report/detail 获取项目韧性评估报告 POST
/project/report/retry 重试项目评估报告生成 GET /socket/setup 建立 WebSocket 连接 POST /socket/ticket 创建
WebSocket 连接票据 线 DELETE /user/avatar 删除用户自定义头像 GET /user/avatar 获取用户头像 POST
/user/avatar 上传用户自定义头像 icw.core.biz 服务接口清单 表 B.2 icw.core.biz RPC 接口清单 RPC 服务 RPC 方法
接口说明 AuthService Login 用户登录并签发令牌 AuthService Logout 用户登出并注销刷新令牌 AuthService Me 校
验访问令牌并返回用户信息 AuthService Refresh 刷新访问令牌和刷新令牌 AuthService Register 注册用户账号 续下
页 续表 B.2 RPC 服务 RPC 方法 接口说明 AuthService ResetPassword 通过邮箱验证码重置密码 AuthService
SendEmailCode 发送邮箱验证码 ProjectCoreService AdvanceProject 推进项目流程阶段 ProjectCoreService
CheckProjectAccess 校验项目访问权限 ProjectCoreService CreateProject 创建项目主记录 ProjectCoreService
DeleteProject 删除项目主记录 ProjectCoreService GetProjectDashboard 获取工作台统计数据 ProjectCoreService
ListProjects 获取用户项目列表 ProjectProfileService DeleteProjectThumbnail 删除项目缩略图
ProjectProfileService GetProjectProfile 获取项目基础信息 装 ProjectProfileService GetProjectThumbnail 获取项
目缩略图访问地址 ProjectProfileService UpdateProjectProfile 更新项目基础信息 ProjectProfileService
UploadProjectThumbnail 生成项目缩略图上传策略 ProjectAssetsService CreateProjectGroup 创建项目图像组
ProjectAssetsService DeleteProjectGroup 删除项目图像组 订 ProjectAssetsService DeleteProjectImage 删除项
目图像 ProjectAssetsService GetProjectAssets 获取项目图像资产列表 ProjectAssetsService
GetProjectImageOriginal 获取项目原图访问地址 ProjectAssetsService MoveProjectGroup 移动项目图像组 线
ProjectAssetsService MoveProjectImage 移动项目图像 ProjectAssetsService ReportProjectImage 上报图像上传
结果 ProjectAssetsService UpdateProjectGroup 更新项目图像组 ProjectAssetsService UploadProjectImage 生成
项目图像上传策略 ProjectDetectionService GetImageDetectionResult 获取单张图像检测结果
ProjectDetectionService GetProjectDetectionTasks 获取项目检测任务列表 ProjectDetectionService
GetReasoningArtifactPolicy 获取推理产物上传策略 ProjectDetectionService ReportClassificationResult 接收分类
Agent 回调结果 ProjectDetectionService ReportDetectionSummaryResult 接收图像级总结 Agent 回调结果
ProjectDetectionService ReportReasoningResult 接收原子检测推理回调结果 ProjectDetectionService
RetryProjectDetection 重试项目智能检测 ProjectDetectionService StartProjectDetection 启动项目智能检测
ProjectReviewService GetProjectDetectionReview 获取图像检测人工复核信息 ProjectReviewService
UpdateProjectDetectionReview 更新图像检测人工复核信息 ProjectReportService GetProjectReport 获取项目评估
报告 续下页 续表 B.2 RPC 服务 RPC 方法 接口说明 ProjectReportService RetryProjectReport 重试项目评估报告生成
ProjectReportService ReportProjectSummaryResult 接收项目报告 Agent 回调结果 SocketService
CreateSocketTicket 创建 WebSocket 连接票据 SocketService ValidateSocketTicket 校验 WebSocket 连接票据
UserService DeleteAvatar 删除用户头像 UserService GetAvatar 获取用户头像 UserService UploadAvatar 上传用
户头像 icw.activity.classification 服务接口清单 装 表 B.3 icw.activity.classification RPC 接口清单 RPC 服务 RPC 方法
接口说明 ClassificationService Start 启动分类任务 订 icw.activity.reasoning 服务接口清单 表 B.4
icw.activity.reasoning RPC 接口清单 RPC 服务 RPC 方法 接口说明 线 ReasoningService Start 启动推理任务
icw.activity.summary 服务接口清单 表 B.5 icw.activity.summary RPC 接口清单 RPC 服务 RPC 方法 接口说明
SummaryService StartDetectionSummary 启动图像检测总结任务 SummaryService StartProjectSummary 启动
项目总结任务 C. 环境变量配置 本附录对Core Web、Core API、Core BIZ、Classification、Reasoning 和Summary
六个服务的环境变量进行整理。表中值为部署环境中的配置值，通过配置文件传入镜像。icw.core.web 服务环境变量配
置 表 C.1 icw.core.web 环境变量 环境变量名称 配置值 说明 VITE_API_BASE_URL http://47.76.71.221:8000 Core
API 基础地址 icw.core.api 服务环境变量配置 环境变量名称配置值说明GIN_MODEReleaseGin 运行模式
ICW_CORE_API_ADDR0.0.0.0:8000Core API HTTP 服务地址ICW_CORE_BIZ_ADDRicw-core-biz:8001Core BIZ
gRPC 服务访问地址ROCKETMQ_NAMESRV_ADDR47.76.71.221:9876RocketMQ NameServer 地址
ROCKETMQ_PROJECT_EVENT_TOPICICW_PROJECT_EVENT项目事件主题名称
ROCKETMQ_CONSUMER_GROUPicw_core_api_ws_dispatcherAPI WebSocket 事件消费者组环境变量名称配置值说
明GIN_MODEReleaseGin 运行模式ICW_CORE_API_ADDR0.0.0.0:8000Core API HTTP 服务地址
ICW_CORE_BIZ_ADDRicw-core-biz:8001Core BIZ gRPC 服务访问地址
ROCKETMQ_NAMESRV_ADDR47.76.71.221:9876RocketMQ NameServer 地址
ROCKETMQ_PROJECT_EVENT_TOPICICW_PROJECT_EVENT项目事件主题名称
ROCKETMQ_CONSUMER_GROUPicw_core_api_ws_dispatcherAPI WebSocket 事件消费者组表 C.2 icw.core.api
环境变量 装 订 icw.core.biz 服务环境变量配置 表 C.3 icw.core.biz 环境变量 线 环境变量名称 配置值 说明
ICW_CORE_BIZ_ADDR 0.0.0.0:8001 Core BIZ gRPC 服务地址 ICW_ACTIVITY_CLASSIFICATION_ADDR icw-
activity-classification:8002 Classification 服务gRPC 地址 ICW_ACTIVITY_REASONING_ADDR icw-activity-
reasoning:8003 Reasoning 服务 gRPC 地址 ICW_ACTIVITY_SUMMARY_ADDR icw-activity-summary:8004
Summary 服务 gRPC 地址 MYSQL_USERNAME mysql-user MySQL 用户名 MYSQL_PASSWORD mysql-2250758
MySQL 密码 MYSQL_ADDR 47.76.71.221:3306 MySQL 服务地址 MYSQL_DATABASE icw_data_mysql MySQL 数
据库名称 REDIS_ADDR 47.76.71.221:6379 Redis 服务地址 REDIS_PASSWORD redis-2250758 Redis 密码
REDIS_DB 0 Redis 数据库编号 SMTP_HOST smtpdm.aliyun.com SMTP 邮件服务器主机 续下页 续表 C.3 环境变量
名称 配置值 说明 SMTP_PORT 465 SMTP SSL 端口 SMTP_PASSWORD M7nB9cV3xZ1IK4jH SMTP 发信密码
SMTP_FROM_NAME 建筑幕墙韧性评估软件系统 邮件发信显示名称 SMTP_FROM_EMAIL
noreply@mail.minmuslin.cn 邮件发信邮箱 MINIO_ENDPOINT http://47.76.71.221:9000 MinIO 服务地址
MINIO_ACCESS_KEY minio-user MinIO 访问密钥 MINIO_ACCESS_SECRET minio-2250758 MinIO 访问密钥
Secret MINIO_BUCKET icw.data.minio MinIO 业务存储桶 ROCKETMQ_NAMESRV_ADDR 47.76.71.221:9876
RocketMQ NameServer 地址 ROCKETMQ_PROJECT_EVENT_TOPIC ICW_PROJECT_EVENT 项目事件主题名称 装

JWT_SECRET icw-jwt-secret JWT 签名密钥 EMAIL_CODE_SECRET icw-email-code-secret 邮箱验证码签名密钥
 EMAIL_CODE_TTL_MINUTES 1 邮箱验证码有效期 LOGIN_FAIL_TTL_MINUTES 10 登录失败计数有效期
 ACCESS_TOKEN_TTL_MINUTES 10 Access Token 有效期 订 REFRESH_TOKEN_TTL_MINUTES 480 Refresh Token
 有效期 AVATAR_GET_TTL_MINUTES 480 用户头像下载链接有效期 AVATAR_UPLOAD_TTL_MINUTES 1 用户头像上传
 策略有效期 PROJECT_IMAGE_GET_TTL_MINUTES 60 项目图像下载链接有效期 线
 PROJECT_IMAGE_UPLOAD_TTL_MINUTES 1 项目图像上传策略有效期 SOCKET_TICKET_TTL_MINUTES 1
 WebSocket 连接票据有效期 PROJECT_IMAGE_PENDING_TIMEOUT_MINUTES 1 图像上传中状态超时时间
 PENDING_IMAGE_TIMEOUT_JOB_CRON "0 * * * * *" 上传中图像超时任务 Cron
 MINIO_DIRTY_OBJECT_CLEANUP_JOB_CRON "0 0 * * * *" MinIO 脏对象清理任务 Cron
 DETECTION_WORKER_MAX_CONCURRENCY 20 检测任务 Worker 最大并发数 icw.activity.classification 服务环境
 变量配置 表 C.4 icw.activity.classification 环境变量 环境变量名称 配置值 说明
 ICW_ACTIVITY_CLASSIFICATION_ADDR 0.0.0.0:8002 Classification gRPC 服务地址 ICW_CORE_BIZ_ADDR icw-
 core-biz:8001 Core BIZ gRPC 服务地址 续下页 续表 C.4 环境变量名称 配置值 说明
 CLASSIFICATION_TASK_MAX_CONCURRENCY 20 分类任务最大并发数 AGENT_SECRET_TOKEN pat_***** 分类
 Agent 调用密钥 AGENT_BOT_ID 7633469965525614634 分类 Agent Bot 标识 AGENT_USER_ID
 icw_activity_classification 分类 Agent 用户标识 AGENT_IMAGE_SIZE 512 分类前图像压缩尺寸
 AGENT_REQUEST_TIMEOUT_MINUTES 10 Agent 请求超时时间 icw.activity.reasoning 服务环境变量配置 表 C.5
 icw.activity.reasoning 环境变量 环境变量名称配置值说明
 ICW_ACTIVITY_REASONING_ADDR 0.0.0.0:8003 Reasoning gRPC 服务地址 ICW_CORE_BIZ_ADDR icw-core-
 biz:8001 Core BIZ gRPC 服务地址 REASONING_MODEL_DIR/app/models 本地模型文件根目录
 REASONING_RUNTIME_DIR/app/.runtime 推理运行时临时目录 REASONING_TASK_MAX_CONCURRENCY 20 推理任
 务最大并发数 REASONING_TASK_TIMEOUT_MINUTES 30 单个推理任务超时时间
 ARTIFACT_DOWNLOAD_TIMEOUT_MINUTES 5 检测输入文件下载超时时间
 ARTIFACT_UPLOAD_TIMEOUT_MINUTES 10 检测产物上传超时时间 环境变量名称配置值说明
 ICW_ACTIVITY_REASONING_ADDR 0.0.0.0:8003 Reasoning gRPC 服务地址 ICW_CORE_BIZ_ADDR icw-core-
 biz:8001 Core BIZ gRPC 服务地址 REASONING_MODEL_DIR/app/models 本地模型文件根目录
 REASONING_RUNTIME_DIR/app/.runtime 推理运行时临时目录 REASONING_TASK_MAX_CONCURRENCY 20 推理任
 务最大并发数 REASONING_TASK_TIMEOUT_MINUTES 30 单个推理任务超时时间
 ARTIFACT_DOWNLOAD_TIMEOUT_MINUTES 5 检测输入文件下载超时时间
 ARTIFACT_UPLOAD_TIMEOUT_MINUTES 10 检测产物上传超时时间 装 订 线 icw.activity.summary 服务环境变量配置
 表 C.6 icw.activity.summary 环境变量 环境变量名称 配置值 说明 ICW_ACTIVITY_SUMMARY_ADDR 0.0.0.0:8004
 Summary gRPC 服务地址 ICW_CORE_BIZ_ADDR icw-core-biz:8001 Core BIZ gRPC 服务地址
 DETECTION_SUMMARY_TASK_MAX_CONCURRENCY 20 图像检测总结任务最大并发数
 DETECTION_SUMMARY_AGENT_SECRET_TOKEN pat_***** 图像检测总结 Agent 调用密钥
 DETECTION_SUMMARY_AGENT_BOT_ID 7637717111535730723 图像检测总结 Agent Bot 标识
 DETECTION_SUMMARY_AGENT_USER_ID icw_activity_summary 图像检测总结 Agent 用户标识
 PROJECT_SUMMARY_TASK_MAX_CONCURRENCY 20 项目报告总结任务最大并发数 续下页 续表 C.6 环境变量名称
 配置值 说明 PROJECT_SUMMARY_AGENT_SECRET_TOKEN pat_***** 项目报告 Agent 调用密钥
 PROJECT_SUMMARY_AGENT_BOT_ID 7637717197741342726 项目报告 Agent Bot 标识
 PROJECT_SUMMARY_AGENT_USER_ID icw_activity_summary 项目报告 Agent 用户标识
 AGENT_REQUEST_TIMEOUT_MINUTES 10 Agent 请求超时时间 装 订 线 谢辞 十八岁刚进入同济大学时，我以为二十
 二岁会是某种写好的答案，但是真到了这一天，才发现那是一道更复杂、更迷茫、也更没有标准答案的题目。首先，衷心
 感谢我的指导教师黄杰老师。感谢黄老师在选题确定、系统设计、论文撰写和整体方向把握上给予的指导与帮助。老师的指
 导帮助我不断收束主线，使我最终明确了核心技术路线与工作创新点，也让我能够把一个庞杂的软件系统整理成清晰的论文
 结构。毕业设计能够从最初的想法推进到最终成稿，离不开老师在关键阶段的指导、支持与包容。感谢沈坚老师。高级语
 言程序设计是我在本科阶段收获最大的课程，也是我真正 装 走进代码世界的起点。沈老师扎实而严谨的教学，为我打下了
 C/C++ 编程基础，也帮助我较早培养起了计算机思维。后来无论是写课程项目、完成毕业设计，还是进入工作后编写需要
 长期稳定运行、服务真实用户的生产代码，我都能感受到那段程序入门训 订 练留下的影响。那些当时看似只是语法、调试
 和作业的训练，后来都慢慢变成了工程实践中的底气。感谢我的家人。感谢你们在我本科四年求学过程中始终给予理解、
 支持和信任。无 论是课程压力、竞赛准备，还是实习求职、本科就业，你们都没有用某一种固定答案要 线 求我，而是尊重
 我的判断，支持我去寻找更适合自己的道路。感谢 iGEM 2024 Tongji-Software 的所有队友。那是一段非常特别的经
 历：我们从最初的想法出发，一起讨论、开发、部署，最后在巴黎斩获国际金奖。竞赛本身带给我的不只是奖项，更是一次
 跨学科协作、产品构思和团队推进的完整训练。它让我意识到，编程能力只是软件工程的一部分，更重要的是如何把想法组
 织起来、把问题讲清楚，并最终把系统交付出去。感谢同济大学。感谢西南十二楼党建室、通达馆和济事楼 319 学生技术
 俱乐部，为我提供了学习、熬夜和赶项目的场所。许多代码、报告、答辩材料和技术方案，都是在这些地方一点点完成的。
 也感谢一直陪伴我的她。大学四年里，有许多焦虑、低落和自我怀疑的时刻，也有许多因为项目跑通、面试通过、论文完成
 而放松下来的瞬间，谢谢你愿意听我讲那些并 不总是有趣的技术问题，见证这些具体而琐碎的时刻。最后，感谢一直努力
 的自己。回顾本科四年，我庆幸没有只把大学当成绩点和排名的竞技场。工程能力的培养不能只依赖课堂学习，更需要长期
 主动实践、持续自学和真实项目的反复锤炼。正是在课程之外的探索与投入中，我从 C/C++ 写到 Java、Golang、
 TypeScript，从单机系统写到分布式系统，从简单接口写到完整部署，逐渐建立起自己的技术体系。曾经我总觉得大学时
 光还有很长，很多事情以后再做也来得及，真正写到毕业论文最后一页时，才发现四年其实过得很快。这里有让我疲惫的时
 刻，也有让我被推动着成长的时刻；有遗憾，也有幸运。它并没有给我一条标准答案，而是让我在不断选择中 逐渐看清自
 己。装 谨以此文，献给我的本科四年。愿自己在未来仍能保持清醒的自我认知，保持对技 术的热情与好奇心，也保持继续
 学习和解决真实